IMPLEMENTASI REINFORCEMENT LEARNING PADA ROBOT HEXAPOD

LAPORAN TUGAS AKHIR

Oleh:

Muhammad Harish

102121014

FAKULTAS TEKNOLOGI INDUSTRI

PROGRAM STUDI TEKNIK ELEKTRO

UNIVERSITAS PERTAMINA

2025

Hak Cipta milik Universitas Pertamina Dilindungi Undang-Undang © Copyright of Universitas Pertamina

1. Dilarang mengutip karya tulis ini, kecuali:
   a. menyebutkan sumber sesuai kaidah kecendekiaan;
   b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
   c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Implementasi Reinforcement Learning Pada Robot Hexapod

Muhammad Harish
10212114

Pertamina

2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

IMPLEMENTASI REINFORCEMENT LEARNING PADA ROBOT HEXAPOD

LAPORAN TUGAS AKHIR

Oleh:

Muhammad Harish

102121014



FAKULTAS TEKNOLOGI INDUSTRI

PROGRAM STUDI TEKNIK ELEKTRO

UNIVERSITAS PERTAMINA

2025

Hak Cipta milik Universitas Pertamina

Dilindungi Undang-Undang

© Copyright of Universitas Pertamina

Universitas Pertamina

1. Dilarang mengutip karya tulis ini, kecuali:
   a. menyebutkan sumber sesuai kaidah kecendekiaan;
   b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
   c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

LEMBAR PENGESAHAN

Judul Tugas Akhir	:	Implementasi Reinforcement Learning Pada Robot Hexapod
Nama Mahasiswa	:	Muhammad Harish
Nomor Induk Mahasiswa	:	102121014
Program Studi	:	Teknik Elektro
Fakultas	:	Teknologi Industri
Tanggal Lulus Sidang Tugas Akhir	:	11 Juli 2025

Jakarta, 06 Agustus 2025

Mengesahkan,

Dosen Pembimbing 1

Dr. Eng. Muhammad Abdillah, S.T., M.T.

NIP. 116153

Dosen Pembimbing 2

Dr. Eng. Novanto Yudistira, S.Kom., M.Sc.

NIP. 20120183311101001

Dosen Penguji 1

Wahyu Agung Pramudito, B. Eng., Ph.D

NIP. 116093

Dosen Penguji 2

Dr. Eng. Ananta Adhi Wardana, S.T., M.Eng

NIP. 199102162022103101

Mengetahui,

Ketua Program Studi Teknik Elektro

Dr. Eng., Muhammad Abdillah., S.T., M.T

NIP. 116153

Universitas Pertamina
1. Dilarang mengutip karya tulis ini, kecuali:
a. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
b. menyebutkan sumber sesuai kaidah kecendekiaan;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
Hak Cipta milik Universitas Pertamina
Dilindungi Undang-Undang
Copyright of Universitas Pertamina

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa Tugas Akhir berjudul **Implementasi Reinforcement Learning Pada Robot Hexapod** ini adalah benar-benar merupakan hasil karya saya sendiri dan tidak mengandung materi yang ditulis oleh orang lain kecuali telah dikutip sebagai referensi yang sumbernya telah dituliskan secara jelas sesuai dengan kaidah penulisan karya ilmiah.

Apabila dikemudian hari ditemukan adanya kecurangan dalam karya ini, saya bersedia menerima sanksi dari Universitas Pertamina sesuai dengan peraturan yang berlaku.

Demi pengembangan ilmu pengetahuan, saya menyetujui untuk memberikan kepada Universitas Pertamina hak bebas royalti noneksklusif (non-exclusive royalty-free right) atas Tugas Akhir ini beserta perangkat yang ada. Dengan hak bebas royalti noneksklusif ini Universitas Pertamina berhak menyimpan, mengalih media/format-kan, mengelola dalam bentuk pangkatan data (database), merawat, dan mempublikasikan Tugas Akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya

Jakarta, 08 Agustus 2025

Yang membuat pernyataan,



Muhammad Harish

ABSTRAK

Muhammad Harish. 102121014. Implementasi *Reinforcement Learning* pada Robot Hexapod

Robot hexapod merupakan robot berkaki enam yang dirancang untuk dapat melintasi medan tidak beraturan secara stabil. Namun, kompleksitas kendali akibat 18 derajat kebebasan pada kakinya menimbulkan tantangan besar dalam desain kontrol gerak. Perancangan ini mengembangkan sistem kontrol adaptif dengan mengintegrasikan metode kinematika (inverse kinematic, body kinematic, dan trajectory planning) dengan algoritma Reinforcement Learning (RL). Tiga algoritma RL—Proximal Policy Optimization (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), dan Soft Actor-Critic (SAC)—dilatih menggunakan simulator PyBullet. Hasil pelatihan selama 100.000 langkah menunjukkan bahwa algoritma SAC menghasilkan reward kumulatif terbaik sebesar -1300, mengungguli TD3 (-2200) dan PPO (-4400), serta mampu menyelesaikan seluruh rintangan simulasi dengan rata-rata durasi 90 langkah per episode. Model SAC kemudian diimplementasikan pada robot fisik BumbleBee dan diuji pada tiga jenis medan nyata: tanjakan, turunan, dan tangga. Robot berhasil menyelesaikan setiap rintangan dengan rata-rata waktu 26 detik, kestabilan orientasi tubuh (pitch dan roll) tetap terjaga dalam rentang ±0,3 radian, serta tidak mengalami kegagalan (success rate 100%). Hasil ini menunjukkan bahwa integrasi RL dan kinematika mampu meningkatkan efisiensi pergerakan dan adaptivitas robot hexapod secara signifikan di medan kompleks.

Kata kunci: Robot hexapod, Reinforcement Learning, Soft Actor-Critic, inverse kinematic, kendali adaptif

Hak Cipta milik Universitas Pertamina. Copyright of Universitas Pertamina
1. Dilarang mengkopi karya tulis ini kecuali:
a. menyebutkan sumber sesuai kaidah ilmiah;
b. penggunaan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah atau penelitian;
c. penggunaan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

ABSTRACT

Muhammad Harish. 102121014. Implementasi *Reinforcement Learning* pada Robot Hexapod

Hexapod robots are six-legged machines designed for stable locomotion across unstructured terrains. However, the 18 degrees of freedom in their legs pose significant control complexity. This study develops an adaptive control system by integrating kinematic methods (inverse kinematics, body kinematics, and trajectory planning) with Reinforcement Learning (RL) algorithms. Three RL algorithms Proximal Policy Optimization (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC) were trained in a PyBullet simulation. After 100,000 training steps, SAC achieved the best cumulative reward of -1300, outperforming TD3 (-2200) and PPO (-4400), and completed all obstacle challenges with an average of 90 steps per episode. The trained SAC model was then deployed on the physical BumbleBee robot and tested on three real-world terrains: slope climbing, downhill, and stairs. The robot successfully completed each challenge in an average of 26 seconds, maintained body stability within ±0.3 radians, and achieved a 100% success rate. These results demonstrate that combining RL and kinematics significantly improves the locomotion efficiency and adaptability of hexapod robots in complex environments.

Keywords: Hexapod robot, Reinforcement Learning, Soft Actor-Critic, inverse kinematics, adaptive control

Pertamina
Universitas
Hak Cipta milik Universitas Pertamina
Perpustakaan Universitas Pertamina
© Copyright of Universitas Pertamina
1. Dilarang mengutip karya tulis ini kecuali:
a. menyebutkan sumber sesuai kaidah ilmiah;
b. penggunaannya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
c. penggunaan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat, karunia, dan kemudahan-Nya, sehingga penulis dapat menyelesaikan tugas akhir dengan judul: **Implementasi Reinforcement Learning Pada Robot Hexapod** sebagai salah satu syarat untuk memperoleh gelar Sarjana Teknik di Program Studi Teknik Elektro, Universitas Pertamina. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada:

1. Kedua orang tua tercinta, Syamsul Bahri dan Endang Candra Dewi, atas doa, cinta, dan dukungan yang tiada henti dalam setiap langkah penulis.
2. Dr. Eng. Muhammad Abdillah, S.T., M.T., selaku Pembimbing I, dan Dr. Eng. Novanto Yudistira, S.Kom., M.Sc., selaku Pembimbing II, yang telah membimbing dengan penuh kesabaran, memberikan ilmu dan arahan selama proses penyusunan tugas akhir ini.
3. Dr. Arie Sukma Jaya, S.T., M.Eng., IPM., sebagai pembina UKM Robotik, yang selalu memberi inspirasi dan motivasi dalam pengembangan robotika.
4. Seluruh dosen dan staf Program Studi Teknik Elektro Universitas Pertamina yang telah membekali penulis dengan ilmu dan pengalaman berharga selama masa perkuliahan.
5. Rekan-rekan seperjuangan Teknik Elektro angkatan 2021 yang telah menjadi teman belajar, berbagi semangat, dan saling mendukung hingga akhir.

Penulis menyadari bahwa tugas akhir ini masih jauh dari sempurna. Oleh karena itu, penulis terbuka terhadap segala masukan dan saran yang membangun demi penyempurnaan di masa mendatang. Harapan penulis, semoga karya ini dapat memberikan manfaat dan menjadi bagian dari kontribusi kecil dalam dunia pendidikan dan teknologi. Semoga Allah SWT meridai setiap langkah yang telah ditempuh, dan menjadikannya amal baik yang membawa keberkahan.

Jakarta, 19 Juni 2025

Penulis

DAFTAR ISI

LEMBAR PENGESAHAN i
LEMBAR PERNYATAAN ii
ABSTRAK iii
ABSTRACT iv
KATA PENGANTAR v
DAFTAR ISI vi
DAFTAR GAMBAR viii
DAFTAR TABEL x
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 2
1.4 Tujuan Perancangan 3
1.5 Manfaat Perancangan 3
BAB II 5
TINJAUAN PUSTAKA 5
2.1 Robot Hexapod 5
2.2 Desain dan Hardware Robot Hexapod 5
2.2.1 Desain Robot Hexapod BumbleBee 6
2.2.2 Aktuator: Servo Dynamixel AX-12A dan AX-18A 6
2.2.3 Kontroler dan Komputasi: NVIDIA Jetson Nano 7
2.2.4 Sensor: Gyroscope (IMU) BNO055 8
2.2.5 Perangkat Lunak: PyBullet 9
2.3 Inverse Kinematic dan Body Kinematic pada Robot Hexapod 9
2.4 Trajectory dan Gait Pattern pada Robot Hexapod 10

Universitas Pertamina
Hak Cipta milik Universitas Pertamina
Penelitian, Pendidikan dan Penulisan © Copyright Universitas Pertamina
1. Dilarang mengutip karya tulis ini, secara:
a. menyebutkan sumber sesuai dengan kaidah kepenulisan ilmiah,
b. pengutipan hanya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian,
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

2.5 Konsep Dasar Reinforcement Learning 11
2.6 Algoritma RL untuk Robotika (PPO, SAC, TD3) 11
2.6.1 Proximal Policy Optimization (PPO) 11
2.6.2 Soft Actor-Critic (SAC) 11
2.6.3 Twin Delayed Deep Deterministic Policy Gradient (TD3) 12
2.7 Penelitian Terkait 12
BAB III 15
KONSEP PERANCANGAN 15
3.1 Diagram Alir Perancangan 15
3.2 Pertimbangan Perancangan 16
3.3 Analisis Teknis 17
3.4 Peralatan dan Bahan 17
3.5 Formulasi Reinforcement Learning untuk Kontrol Gait Hexapod 18
BAB IV 21
HASIL DAN PEMBAHASAN 21
BAB V 48
KESIMPULAN DAN SARAN 48
DAFTAR PUSTAKA 50
Lampiran 1. Lapangan Kontes Robot SAR Indonesia (KRSRI) 2024 54
Lampiran 2. Robot Hexapod Bumblebee 55
Lampiran 3. Hasil Testing Model *Reinforcement Learning* 56
Lampiran 4. Program Environment Reinforcement Learning 59

Universitas Pertamina
Hak Cipta Universitas Pertamina
Hindungi Undang-Undang Hak Cipta Copyright of Universitas Pertamina
1. Dilarang mengutip karya tulis ini, kecuali:
a. menyebutkan sumber untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
b. penggunaan hanya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

DAFTAR GAMBAR

Gambar 2. 1 Robot Hexapod Berdasarkan Jumlah DoF (Degrees of Freedom) kaki robot (a) 2-DoF [13], (b) 3-DoF [14], (c) 4-DoF [15] (d) 5-DoF [16]. 5
Gambar 2. 2 Desain Robot Hexapod BumbleBee 6
Gambar 2. 3 Servo Dynamixel AX-12A dan AX-18A 6
Gambar 2. 4 NVIDIA Jetson Nano 8
Gambar 2. 5 Sensor IMU BNO055 8
Gambar 2. 6 Konsep Trajectory pada Robot Hexapod. Adapted from [26] 10
Gambar 2. 7 Jenis Gait Pattern pada Robot Hexapod. Adapted from [28] 10
Gambar 3. 1 Flowchart Tahap Pelaksanaan 15
Gambar 3. 2 Arsitektur Sistem 16
Gambar 3. 3 Sistem *Hardware* Robot hexapod 17
Gambar 4. 1 Bagian Utama BumbleBee 21
Gambar 4. 2 Bagian Kaki BumbleBee 21
Gambar 4. 3 BumbleBee 22
Gambar 4. 4 BumbleBee full-view 22
Gambar 4. 5 Model URDF Robot Hexapod 24
Gambar 4. 6 Permodelan Inverse Kinematic pada Kaki Robot 24
Gambar 4. 7 Hubungan Sudut Servo pada Kaki Robot 25
Gambar 4. 8 Penentuan Sudut Coxa Kaki Robot 25
Gambar 4. 9 Penentuan Sudut Femur dan Tibia Kaki Robot 26
Gambar 4. 10 Environment Simulasi Robot 28
Gambar 4. 11 Hasil Training Episode Length 30
Gambar 4. 12 Hasil Training Cumulative Reward 30
Gambar 4. 13 Hasil Testing (PPO vs SAC vs TD3) terhadap Reward 32
Gambar 4. 14 Hasil Testing (PPO vs SAC vs TD3) Terhadap State (Roll) 32
Gambar 4. 15 Hasil Testing (PPO vs SAC vs TD3) Terhadap State (Pitch) 32
Gambar 4. 16 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope 37
Gambar 4. 17 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope 37
Gambar 4. 18 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope 38
Gambar 4. 19 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope 38
Gambar 4. 20 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope 38
Gambar 4. 21 Hasil Pengujian Real Robot (State) pada Rintangan Climbing Slope 39

Universitas Pertamina
tanpa adanya izin dari Universitas Pertamina.

Gambar 4. 22 Dokumentasi Pengujian Real Robot pada Rintangan Climbing Slope 40
Gambar 4. 23 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope 40
Gambar 4. 24 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope 41
Gambar 4. 25 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope 41
Gambar 4. 26 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope 41
Gambar 4. 27 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope 42
Gambar 4. 28 Hasil Pengujian Real Robot (State) pada Rintangan Down Slope 42
Gambar 4. 29 Dokumentasi Pengujian Real Robot pada Rintangan Down Slope 43
Gambar 4. 30 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs 44
Gambar 4. 31 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs 44
Gambar 4. 32 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs 44
Gambar 4. 33 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs 45
Gambar 4. 34 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs 45
Gambar 4. 35 Hasil Pengujian Real Robot (State) pada Rintangan Climbing Stairs 45
Gambar 4. 36 Dokumentasi Pengujian Real Robot pada Rintangan Climbing Stairs 46

Universitas Pertamina
Hak Cipta Dilindungi Undang-Undang
© Copyright of Universitas Pertamina
1. Dilarang menggunakan karya tulis ini, kecuali:
a. penelitian, penulisan karya ilmiah atau penelitian;
b. penulisan laporan untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
c. penulisan dan menyebutkan sumber sesuai kaidah kecendekiaan;
2. Dilarang mengumumkan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

DAFTAR TABEL

Tabel 3. 1 Desain *Observation Space (State)* 18
Tabel 3. 2 Desain *Action Space* 18
Tabel 3. 3 Desain *Reward Function* 19
Tabel 3. 4 Desain *Reward Function* 19
Tabel 4. 1 Spesifikasi BumbleBee 23
Tabel 4. 2 Parameter Kontrol BumbleBee 27
Tabel 4. 3 Desain Nilai *Reward Function* 29
Tabel 4. 4 Perbandingan PPO, TD3, dan SAC Serta Dampaknya ke Robot 35
Tabel 4. 5 Perbandingan Reward PPO, TD3, dan SAC 36

Hak Cipta Universitas Pertamina
Dilindungi Undang-Undang
© Copyright of Universitas Pertamina
Universitas Pertamina.
Dilarang mengutip karya tulis ini, kecuali:
1. Pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
2. Penyebutkan sumber sesuai kaidah kecendekiaan;
Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
Pengutipan tidak merugikan Universitas Pertamina.


Universitas
Pertamina

BAB I
PENDAHULUAN

1.1 Latar Belakang

Salah satu jenis robot yang dapat menjawab tantangan eksplorasi lingkungan yang sulit dijangkau dan berbahaya adalah robot hexapod [1]. Robot ini memiliki pendekatan secara biologis atau *bio-inspired* yang membuatnya lebih mudah untuk menjalankan misi dikarena jumlah kaki dan derajat kebebasan atau *degrees of freedom* yang dimilikinya [2]. Akan tetapi, fleksibilitas yang dimiliki membuat tingkat kompleksitas robot menjadi tinggi. Cara paling mudah untuk menggerakkan robot ini adalah dengan mengubah setiap sendi dengan sudut tertentu sehingga menciptakan gerakan yang dinamis [3]. Metode ini sangat tidak efektif terlebih gerakan robot tidak akan stabil dan program yang tidak efesien karena memberikan mobilitas yang rendah terhadap robot [4].

Beberapa penelitian sebelumnya telah menerapkan beberapa metode kinematika seperti *inverse kinematic* dan *body kinematic* yang dapat mentransformasikan koordinat kartesian menjadi sudut untuk kemudian diterapkan pada kaki robot. *Inverse kinematic* diperlukan karena dapat mempermudah kalkulasi sudut pada setiap sendi kaki robot dengan acuan ujung kaki robot atau *end effector* [5]. Sedangkan *Body Kinematic* diperlukan untuk mengatur posisi pusat massa robot (*Body*) [6]. Penerapan *gait* dan *trajectory* juga memiliki peran penting agar robot dapat bergerak secara dinamis. *Gait* merupakan pola langkah robot yang menentukan cara robot bergerak [7]. Sedangkan *trajectory* yang merupakan lintasan gerakan kaki robot selama siklus langkah, yang terdiri dari *swing phase* (fase kaki mengayun di udara) dan *stance phase* (fase kaki bertumpu pada permukaan) [8].

Disamping kombinasi kinematika dan pola gerakan robot yang dapat menghasilkan gerakan robot lebih dinamis, metode ini memiliki kekurangan yaitu proses pengaturan pola gerakan (gait) dan pengaturan kestabilan robot masih harus dilakukan secara manual sehingga tidak praktis. Hal ini menyebabkan robot tidak bersifat adaptif yang dapat mengatasi lingkungan yang kompleks dan menentukan gerakan robot yang sesuai dengan rintangan yang dilewati. Disamping itu, beberapa penelitian telah membuktikan bahwa pola gerakan robot hexapod dapat dioptimasi dengan menerapkan metode *reinforcement learning* (RL) [9]. Dengan menggunakan RL yang digabungkan dengan parameter gerak robot, maka robot dapat menerapkan *reward* sebagai fungsi *feedback* dari *action* yang diambil oleh robot [10]. Metode ini menghasilkan gerakan robot yang stabil dan proses training yang cepat [11].

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Bertolak dari permasalahan mobilitas robot hexapod dan potensi penerapan RL yang dapat mejawab tantangan tersebut, **Implementasi *Reinforcement Learning* pada Robot Hexapod** berfokus pada penerapan metode *inverse kinematic*, *body kinematic*, dan *trajectory planning* yang diintegrasikan dengan *Reinforcement Learning* (RL). Integrasi metode kinematik dan *trajectory planning* dapat menghasilkan gerakan robot (*gait*) yang dinamis. Sedangkan RL digunakan sebagai input kontrol pada robot sehingga dapat beradaptasi pada rintangan. Cakupan perancangan dilakukan secara bertahap mulai dari pembuatan simulasi, *hardware* robot, kinematika robot, pembuatan dan *tuning* model RL, serta integrasi antar seluruh sistem tersebut untuk di impelementasikan pada arena Kontes Robot SAR Indonesia (KRSRI) 2024 yang merupakan ajang kontes robotika nasional di Indonesia yang berfokus pada robot Search and Rescue (SAR) [12].

Meskipun kinematika memberikan kontrol presisi dan RL menawarkan kemampuan adaptasi, integrasi keduanya untuk mengurangi kompleksitas ruang aksi (*action space*) dalam RL sambil mempertahankan gerakan yang stabil pada robot fisik di medan kompleks seperti arena KRSRI masih menjadi tantangan. Perancangan ini bertujuan untuk mengisi celah tersebut melalui pendekatan integratif yang menggabungkan kontrol berbasis kinematika dengan pembelajaran berbasis pengalaman atau *reinforcement learning*. Diharapkan dengan dilakukannya penelitian ini, robot hexapod dapat semakin dioptimalkan dalam hal stabilitas, efisiensi pergerakan, dan adaptasi terhadap berbagai medan terutama pada ajang Kontes Robot Indonesia.

1.2 Rumusan Masalah

1. Bagaimana merancang dan mengimplementasikan metode inverse kinematic, body kinematic, dan trajectory planning untuk menghasilkan gerakan yang lebih stabil dan dinamis pada robot hexapod?
2. Bagaimana penerapan Reinforcement Learning (RL) dapat membantu robot hexapod beradaptasi terhadap berbagai jenis rintangan Kontes Robot SAR Indonesia (KRSRI) 2024 seperti jalan menanjak, jalan menurun, dan tangga?

1.3 Batasan Masalah

1. Pengembangan model RL dilakukan pada lingkungan simulasi berbasis Python dengan menggunakan PyBullet menggunakan metode Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3).
2. Lingkungan uji coba robot hexapod merupakan *setting* dari arena Kontes Robot SAR Indonesia (KRSRI) 2024.

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

1.4 Tujuan Perancangan

1. Mengembangkan dan mengimplementasikan metode inverse kinematic, body kinematic, dan trajectory planning untuk meningkatkan stabilitas dan efisiensi pergerakan robot hexapod.
2. Mengintegrasikan *Reinforcement Learning* (RL) sebagai kontrol adaptif pada robot hexapod.
3. Menguji performa robot dalam menghadapi berbagai medan dan memastikan pola gerakan dapat disesuaikan secara otomatis.rintangan.

1.5 Manfaat Perancangan

1. Memberikan kontribusi dalam pengembangan sistem kendali gerakan adaptif untuk robot hexapod berbasis kinematika dan model kecerdasan buatan (RL).
2. Meningkatkan efisiensi pergerakan dan stabilitas robot hexapod dalam menghadapi medan yang tidak terstruktur.
3. Mendukung inovasi dalam bidang robotika, khususnya pada penerapan navigasi otonom dan pengambilan keputusan berbasis kecerdasan buatan.
4. Mengembangkan robot hexapod yang dapat beradaptasi pada berbagai rintangan untuk membantu memberikan inovasi di sektor industri dan bencana.

Universitas Pertamina
Hak Cipta dilindungi Undang-Undang
© Copyright of Universitas Pertamina
1. Dilarang mengutip karya ini kecuali:
2. Dilarang memperbanyak atau mendistribusikan karya ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. Penggunaan untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
b. Penyebutan sumber sesuai kaidah kecendekiaan;
c. Penggunaan tidak merugikan Universitas Pertamina.


Universitas
Pertamina

BAB II
TINJAUAN PUSTAKA

2.1 Robot Hexapod

Robot hexapod merupakan sebuah robot yang terinspirasi dari gerakan serangga dengan enam kaki. Kelebihan robot ini dibanding robot berkaki lain adalah kemampuan yang memungkinkan mobilitas tinggi di medan tidak terstruktur. Keunggulannya terletak pada stabilitas statis (tetap seimbang meski 3 kaki tidak menyentuh tanah) dan kemampuan adaptasi topografi yang kompleks [1]. Setiap kaki biasanya memiliki 3 derajat kebebasan (*Degrees of Freedom/DoF*), yaitu sendi coxa (pangkal kaki), femur (paha), dan tibia (betis), yang memungkinkan gerakan maju, mundur, dan mengangkat kaki. Bentuk umum dari robot hexapod berdasarkan DoF setiap kaki robot adalah sebagai berikut.



(a) 2-DoF Hexapod Robot (b) 3-DoF Hexapod Robot (c) 4-DoF Hexapod Robot (d) 5-DoF Hexapod Robot

Gambar 2. 1 Robot Hexapod Berdasarkan Jumlah DoF (Degrees of Freedom) kaki robot (a) 2-DoF [13], (b) 3-DoF [14], (c) 4-DoF [15] (d) 5-DoF [16].

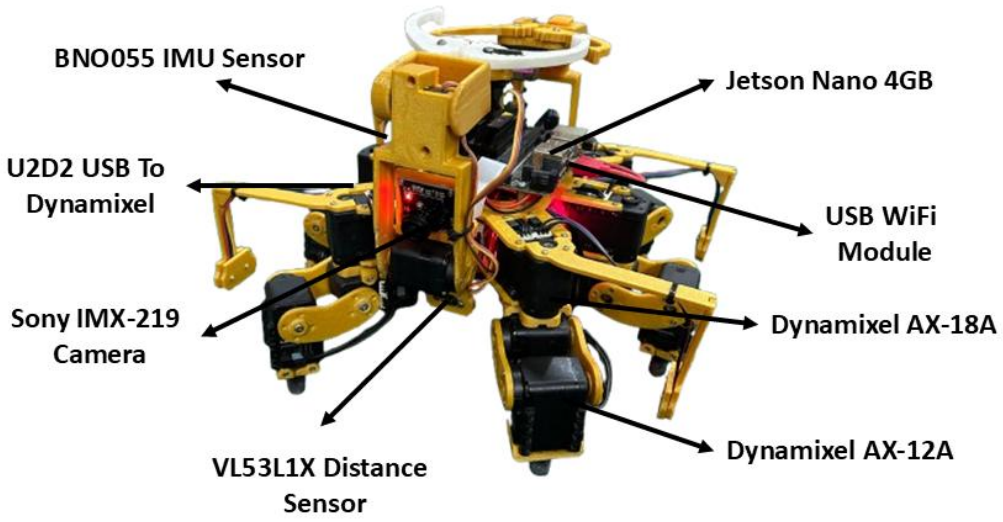
Perbedaan mendasar antara jumlah DoF pada kaki robot hexapod adalah pada kemampuannya menjaga keseimbangan tubuh (*body stability*) saat melintasi medan kompleks, seperti bebatuan, tangga, atau lereng curam. Semakin banyak jumlah DoF yang terdapat pada robot hexapod akan menjadikan robot semakin stabil. Akan tetapi terdapat *trade-off* yaitu kompleksitas kontrol, energi, dan biaya pembuatan robot. Umumnya 3-DoF robot hexapod merupakan pilihan terbaik karena dapat memberikan kestabilan yang baik dengan kompleksitas dibawah 4-DoF dan 5 DoF. Hexapod 3-DoF untuk setiap kakiberarti memiliki total 18 DoF (6 kaki × 3 sendi) yang harus disinkronkan [14]. Oleh karena itu, pada perancangan ini akan menggunakan hexapod dengan 3 DoF untuk setiap kaki (18 DoF total).

2.2 Desain dan Hardware Robot Hexapod

Robot hexapod yang digunakan dalam penelitian ini menggunakan beberapa komponen utama yang mendukung kestabilan, kecepatan, dan akurasi dalam pengambilan keputusan dan pergerakan. Berikut adalah penjelasan masing-masing komponen.

2.2.1 Desain Robot Hexapod BumbleBee

BumbleBee merupakan nama robot hexapod dirancang dengan struktur modular yang terinspirasi dari serangga, dengan 6 kaki simetris dan 3 sendi per kaki (coxa, femur, dan tibia). Setiap kaki memiliki 3 derajat kebebasan (*3-DOF*) untuk memungkinkan gerakan 3D seperti maju, mundur, dan rotasi. Material utama yang digunakan adalah dan PLA cetak 3D untuk mengurangi berat tanpa mengorbankan kekuatan. Berikut merupakan desain 3D dari robot hexapod BumbleBee.



Gambar 2. 2 Desain Robot Hexapod BumbleBee

2.2.2 Aktuator: Servo Dynamixel AX-12A dan AX-18A

Servo Dynamixel merupakan aktuator pintar yang dirancang khusus untuk aplikasi robotika yang membutuhkan kontrol presisi tinggi. Dynamixel AX-12A digunakan pada bagian tibia, sedangkan AX-18A digunakan pada bagian coxa dan femur karena memiliki torsi yang lebih besar. Kedua tipe servo ini mendukung komunikasi serial half-duplex TTL dengan protokol yang dapat dikonfigurasi secara fleksibel [17][18].



Gambar 2. 3 Servo Dynamixel AX-12A dan AX-18A

Kelebihan utama dari Dynamixel adalah kemampuannya untuk mengukur posisi, kecepatan, dan suhu secara real-time, serta mendukung control loop tertutup. Hal ini memungkinkan integrasi yang lebih optimal dengan sistem kontrol berbasis Reinforcement Learning (RL) dibandingkan servo konvensional. Berikut merupakan spesifikasi dari akutuator yang digunakan.

1. Dynamixel AX-12A (untuk sendi tibia):
    - Torsi: 1.5 Nm pada 12V, cukup untuk gerak sendi dengan beban ringan.
    - Resolusi sudut: 0.29° dengan kontrol posisi *closed-loop*.
    - Kecepatan rotasi: 0.196 detik per 60°, lebih lambat dibanding AX-18A.
    - Material gear: Plastik, lebih ringan namun kurang tahan terhadap beban berat atau penggunaan jangka panjang.
    - Keunggulan: Harga lebih ekonomis, cocok untuk bagian robot dengan beban dan frekuensi gerak rendah seperti *tibia*.
2. Dynamixel AX-18A (untuk sendi coxa dan femur):
    - Torsi: 1.8 Nm pada 12V, memberikan tenaga lebih besar untuk sendi utama.
    - Kecepatan rotasi: 0.114 detik per 60°, lebih cepat dan responsif dibanding AX-12A.
    - Resolusi sudut: 0.29° dengan kontrol posisi *closed-loop*, seperti AX-12A.
    - Material gear: Metal, memberikan ketahanan lebih tinggi terhadap keausan dan beban berat.
    - Keunggulan: Lebih tahan lama dan kuat, ideal untuk sendi utama seperti *coxa* dan *femur* yang membutuhkan performa tinggi.

2.2.3 Kontroler dan Komputasi: NVIDIA Jetson Nano

Jetson Nano merupakan sistem komputer berbasis GPU dari NVIDIA yang mendukung pemrosesan paralel dan cocok untuk pengembangan model kecerdasan buatan seperti RL. Dengan GPU 128-core Maxwell dan RAM 4GB, Jetson Nano dapat menjalankan inference *machine learning* secara real-time. Jetson Nano mendukung berbagai pustaka deep learning seperti TensorFlow, PyTorch, serta platform NVIDIA seperti CUDA dan TensorRT. Hal ini memungkinkan pengguna untuk melakukan inferensi dari model deep learning secara optimal. Studi yang sama menunjukkan bahwa integrasi Jetson Nano dengan kamera dan algoritma deteksi objek memungkinkan sistem untuk melakukan pengenalan objek secara akurat di lingkungan nyata. Hasil eksperimen menunjukkan akurasi tinggi dalam mendeteksi beberapa objek seperti manusia dan kendaraan, serta efisiensi dalam penggunaan sumber daya [19].

Pertamina
Universitas
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
c. pengutipan tidak merugikan Universitas Pertamina.
b. pengutipan hanya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
a. menyebutkan sumber sesuai kaidah ketentekanan;
1. Dilarang menggunakan karya tulis ini, kecuali:
Hak Cipta Universitas Pertamina
Dilindungi Undang-Undang
Copyright Universitas Pertamina



Gambar 2. 4 NVIDIA Jetson Nano

2.2.4 Sensor: Gyroscope (IMU) BNO055

IMU (*Inertial Measurement Unit*) BNO055 adalah sensor 9-DOF (*Degrees of Freedom*) yang menggabungkan giroskop, akselerometer, dan magnetometer. Sensor ini menggunakan *sensor fusion* (algoritma *Kalman Filter*) untuk menghasilkan estimasi orientasi kuaternion atau Euler angle dengan akurasi tinggi (±0,3°) [22]. Data orientasi dan kecepatan sudut dari BNO055 menjadi input kritis dalam menjaga stabilitas robot, terutama pada medan tidak rata atau saat manuver dinamis. Dalam kerangka *Reinforcement Learning* (RL), IMU berfungsi sebagai sumber *feedback* untuk mengoreksi gerakan robot secara adaptif. Misalnya, nilai *roll* dan *pitch* dari BNO055 dapat digunakan RL untuk menyesuaikan body kinematic agar robot tetap seimbang. Data orientasi dari IMU BNO055 menjadi krusial sebagai state observation untuk reward function yang memberikan penalti pada ketidakstabilan, sebuah komponen penting dalam melatih agen RL.



Gambar 2. 5 Sensor IMU BNO055

Hak Cipta milik Universitas Pertamina
Universitas Pertamina
1. Dilarang mengutip karya tulis ini kecuali:
a. menyebutkan sumber sesuai dengan kaidah penulisan ilmiah;
b. pengutipan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan kepentingan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
Copyright of Universitas Pertamina

2.2.5 Perangkat Lunak: PyBullet

Simulasi dilakukan menggunakan PyBullet, sebuah physics engine open-source yang menyediakan lingkungan 3D real-time dan sering digunakan untuk pengujian RL sebelum diterapkan di robot fisik [23]. Keunggulan PyBullet adalah kecepatan simulasi yang tinggi serta dukungan terhadap pengumpulan data reward dan action dalam format yang kompatibel dengan banyak algoritma RL seperti PPO dan SAC yang telah terbukti diterapkan pada beberapa simulasi robot. Penelitian yang dilakukan oleh Panerati et al. menunjukkan bahwa pybullet dapat diterapkan untuk melakukan simulasi Quadcopter Multi-agent yang dapat diintegrasikan dengan metode reinforcement learning menggunakan Open AI Gym [24].

2.3 Inverse Kinematic dan Body Kinematic pada Robot Hexapod

Kompleksitas kontrol akibat banyaknya derajat kebebasan (degrees of freedom/DOF) pada setiap kaki menjadi tantangan utama dalam mencapai gerakan yang stabil dan efisien. Sehingga pendekatan konvensional dengan menggerakkan setiap sendi secara manual terbukti tidak optimal karena memicu ketidakstabilan dinamis dan mobilitas terbatas [3]. Oleh karena itu, metode kinematika seperti *inverse kinematic* dan *body kinematic* menjadi dasar dalam merancang sistem kontrol robot hexapod.

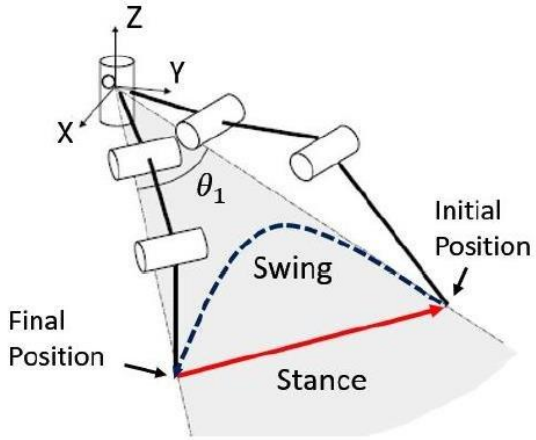
Inverse kinematic (IK) berperan krusial dalam mentransformasikan koordinat kartesian ujung kaki (*end-effector*) menjadi sudut sendi (*joint angle*) yang diperlukan untuk mencapai posisi target. Pada robot hexapod, penerapan IK memungkinkan penghitungan sudut sendi femur, tibia, dan coxa secara matematis berdasarkan posisi ujung kaki relatif terhadap tubuh robot, sehingga menyederhanakan kontrol gerakan tanpa perlu menghitung setiap sendi secara terpisah. Studi oleh Muslimin et al. menunjukkan bahwa solusi IK menggunakan pendekatan matematis dapat menghasilkan error hingga 0% terhadap posisi target kaki robot yang dituju [5].

Body kinematic mengatur orientasi dan translasi tubuh robot (*body frame*) relatif terhadap kaki-kakinya untuk menjaga kestabilan pusat massa (*center of mass*). Metode ini memastikan distribusi beban merata pada kaki yang sedang dalam fase *stance*, sekaligus mengompensasi perubahan sudut tubuh saat robot menghadapi kemiringan medan. Penelitian oleh Kurniawan et al. mengintegrasikan data IMU (*Inertial Measurement Unit*) dengan *body kinematic* untuk mengoreksi orientasi tubuh secara real-time, Berdasarkan hasil yang diperoleh, rata-rata kesalahan pada sumbu X untuk setiap kaki sebesar 1,89%, pada sumbu Y sebesar 2,6%, dan pada sumbu Z mencapai 12,4% [6].

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

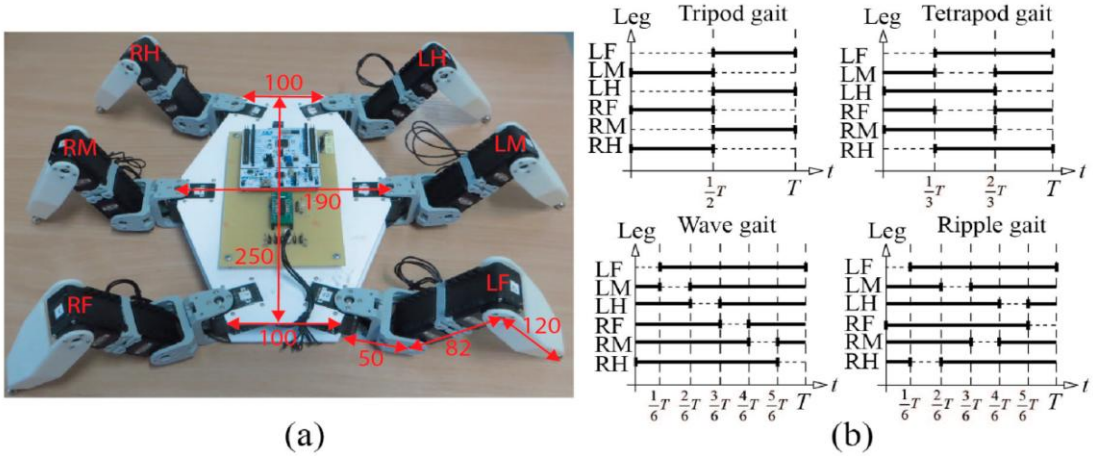
2.4 Trajectory dan Gait Pattern pada Robot Hexapod

Perencanaan lintasan (trajectory) kaki robot melibatkan dua fase utama: swing phase (gerakan mengayun di udara) dan stance phase (gerakan menahan beban di permukaan). Swing phase dirancang menggunakan interpolasi polinomial (contoh: cubic spline) untuk menghasilkan lintasan halus yang meminimalkan getaran, sementara stance phase difokuskan pada translasi tubuh robot ke arah gerakan. Penelitian yang dilakukan oleh Xia et al. menunjukkan bahwa metode perencanaan trajektori yang tepat dapat secara signifikan mengurangi kompleksitas dalam perancangan algoritma control [8].



Gambar 2. 6 Konsep Trajectory pada Robot Hexapod. Adapted from [26]

Pola langkah (gait) menentukan urutan dan sinkronisasi gerakan kaki untuk mencapai kecepatan atau stabilitas tertentu. Pada robot hexapod, pola umum seperti tripod gait (tiga kaki bergerak bersamaan) cocok untuk medan datar, sedangkan wave gait (gerakan kaki berurutan) lebih stabil di medan tidak rata. Studi menambahkan variabel kecepatan dan tinggi langkah (stride height) dalam algoritma gait untuk meningkatkan adaptasi di medan berbatu. Namun, pola gait konvensional bersifat statis, sehingga kurang responsif terhadap perubahan medan secara tiba-tiba [27].



Gambar 2. 7 Jenis Gait Pattern pada Robot Hexapod. Adapted from [28]

2.5 Konsep Dasar Reinforcement Learning

Reinforcement Learning (RL) adalah paradigma pembelajaran mesin di mana agen (robot) belajar mengambil keputusan optimal melalui interaksi dengan lingkungan untuk memaksimalkan cumulative reward [29]. Proses ini didasarkan pada konsep trial-and-error, di mana agen mengeksekusi action berdasarkan observasi state lingkungan, lalu menerima feedback berupa reward atau penalty. Secara formal, RL dimodelkan sebagai Markov Decision Process (MDP) yang terdiri dari tuple (S, A, P, R, γ) (S,A,P,R,γ), dengan **S** sebagai himpunan state, **A** sebagai himpunan action, **P** sebagai fungsi transisi, **R** sebagai fungsi reward, dan **γ** sebagai faktor diskon untuk future reward [30].

Kelebihan RL terletak pada kemampuannya dalam menyelesaikan permasalahan *sequential decision-making* tanpa memerlukan data pelatihan berlabel, sehingga sangat cocok untuk aplikasi robotika yang dinamis dan tidak terstruktur. Namun, tantangan utama dalam RL adalah *trade-off* antara *exploration* dan *exploitation*, dimana *agent* harus menyeimbangkan antara mencoba *action* baru dan memanfaatkan pengetahuan yang telah diperoleh. Dalam konteks robot *hexapod*, RL digunakan untuk mengoptimasi parameter gerakan (*gait*), menjaga stabilitas tubuh, serta beradaptasi terhadap medan melalui proses pelatihan berbasis simulasi maupun interaksi dunia nyata.

2.6 Algoritma RL untuk Robotika (PPO, SAC, TD3)

Beberapa algoritma RL modern telah terbukti efektif dalam mengatasi kompleksitas kontrol robotika, termasuk Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), dan Twin Delayed Deep Deterministic Policy Gradient (TD3).

2.6.1 Proximal Policy Optimization (PPO)

PPO adalah algoritma *policy-based* yang memaksimalkan *objective function* dengan membatasi perubahan kebijakan (*policy*) agar tidak melampaui batas (*trust region*), sehingga menjamin stabilitas pelatihan [31]. Keunggulan PPO terletak pada efisiensi komputasi dan kemudahan implementasi, menjadikannya populer untuk optimasi gerak robot hexapod. Studi oleh [32] menunjukkan bahwa PPO mampu menghasilkan reward yang lebih tinggi hingga dua kali lipat dibandingkan metode actor critic lainnya seperti SAC. Namun, PPO rentan terhadap *local optima* jika *reward function* tidak dirancang dengan hati-hati.

2.6.2 Soft Actor-Critic (SAC)

SAC adalah algoritma *off-policy* yang mengintegrasikan entropi kebijakan (*policy entropy*) ke dalam fungsi *reward* untuk mendorong eksplorasi lebih luas [32]. Pendekatan

ini ideal untuk robotika karena mampu menangani *action space* kontinu (seperti sudut sendi) dan mengurangi risiko *overfitting* pada lingkungan simulasi. Kelemahan SAC adalah kebutuhan komputasi yang tinggi akibat kompleksitas arsitektur *double Q-network*.

2.6.3 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 merupakan pengembangan dari DDPG yang mengatasi masalah *overestimation bias* dengan menggunakan dua *critic network* dan memperlambat update *target network* [33]. Algoritma ini cocok untuk tugas robotika yang memerlukan presisi tinggi, seperti navigasi di medan sempit. Untuk mengatasi kelemahan tersebut, Fujimoto et al. (2018) [33] mengembangkan algoritma TD3 (Twin Delayed DDPG) yang merupakan perbaikan langsung dari DDPG. TD3 menggabungkan beberapa teknik untuk mengurangi kesalahan aproksimasi fungsi dan bias estimasi, di antaranya: Clipped Double Q-learning, Delayed Policy Updates, dan Target Policy Smoothing Regularization.

Ketiga teknik tersebut berhasil meningkatkan performa dan stabilitas dalam berbagai tugas kontrol robotik, termasuk lingkungan MuJoCo seperti Hopper, Walker2d, dan Ant. Evaluasi eksperimen menunjukkan bahwa TD3 secara konsisten mengungguli DDPG dan metode populer lainnya seperti PPO dan SAC dalam hal akurasi dan kecepatan konvergensi. Meskipun TD3 dilaporkan mengatasi kelemahan DDPG, dan SAC unggul dalam eksplorasi, performa ketiganya dalam tugas lokomosi hexapod dengan action space hibrida (gabungan kinematika) perlu diuji secara empiris untuk menentukan yang paling efektif.

2.7 Penelitian Terkait

Penelitian tentang robot hexapod telah berkembang pesat dalam beberapa tahun terakhir. Studi awal oleh [3] dan [4] mengembangkan kontrol dasar dengan mengatur sudut sendi secara manual, namun metode ini menghasilkan gerakan yang tidak stabil dan kurang efisien. Perkembangan berikutnya memperkenalkan metode kinematika seperti inverse kinematic [5] dan body kinematic [6] yang meningkatkan presisi perhitungan gerak, tetapi masih memerlukan tuning manual untuk pola gait sehingga kurang adaptif terhadap medan yang dinamis.

Perkembangan terbaru dalam kontrol robot hexapod menunjukkan potensi Reinforcement Learning (RL) sebagai solusi optimasi gerakan. Studi oleh [9] dan [10] berhasil menerapkan RL untuk menghasilkan gait yang stabil dengan waktu konvergensi relatif cepat dalam lingkungan simulasi. Namun, implementasi ini menghadapi dua tantangan utama: (1) Sebagian besar implementasi RL masih terbatas pada lingkungan simulasi dan belum diuji coba pada medan nyata yang kompleks, dan (2) Kompleksitas

Pertamina
Universitas Pertamina
Hak Cipta milik Universitas Pertamina
Perpustakaan Universitas Pertamina
Copyright of Universitas Pertamina
1. Dilarang mengutip atau menyebarkan tesis ini untuk:
a. kepentingan komersial;
b. penggunaan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah atau penulisan
c. penggunaan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

kontrol tingkat sendi (joint-level control) yang menghasilkan dimensionalitas ruang aksi tinggi, sehingga memperlambat proses training dan menyulitkan konvergensi kebijakan yang optimal.

Oleh karena itu, penelitian ini hadir untuk mengisi celah tersebut dengan mengusulkan integrasi menyeluruh antara metode kinematika (inverse kinematic dan body kinematic) dengan Reinforcement Learning. Pendekatan hybrid ini diharapkan dapat menghasilkan gerakan yang lebih stabil, efisien, dan adaptif, khususnya untuk aplikasi Search and Rescue dalam Kontes Robot Indonesia (KRSRI) 2024 dimana robot harus mampu menghadapi berbagai medan yang tidak terduga. Implementasi tidak hanya dilakukan dalam simulasi tetapi juga pada hardware nyata, sehingga memberikan kontribusi praktis dalam pengembangan robot hexapod.

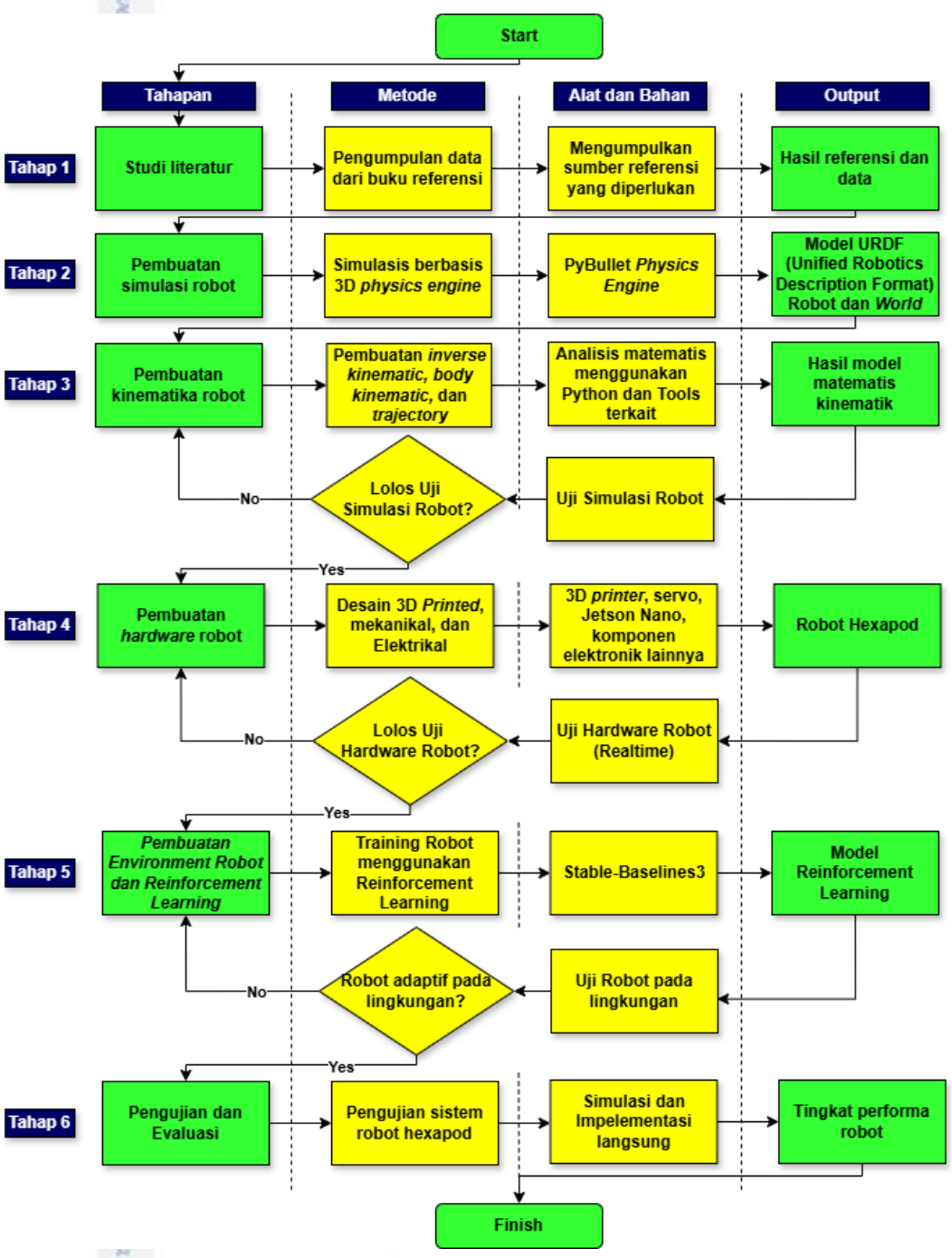
Hak Cipta Universitas Pertamina
Dilindungi Undang-Undang
© Copyright of Universitas Pertamina
Universitas Pertamina
1. Dilarang mengutip karya tulis ini, kecuali:
a. untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah, penyusunan laporan, penulisan kritik atau tinjauan suatu masalah; dan
b. pengutipan tidak merugikan kepentingan yang wajar Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

BAB III
KONSEP PERANCANGAN

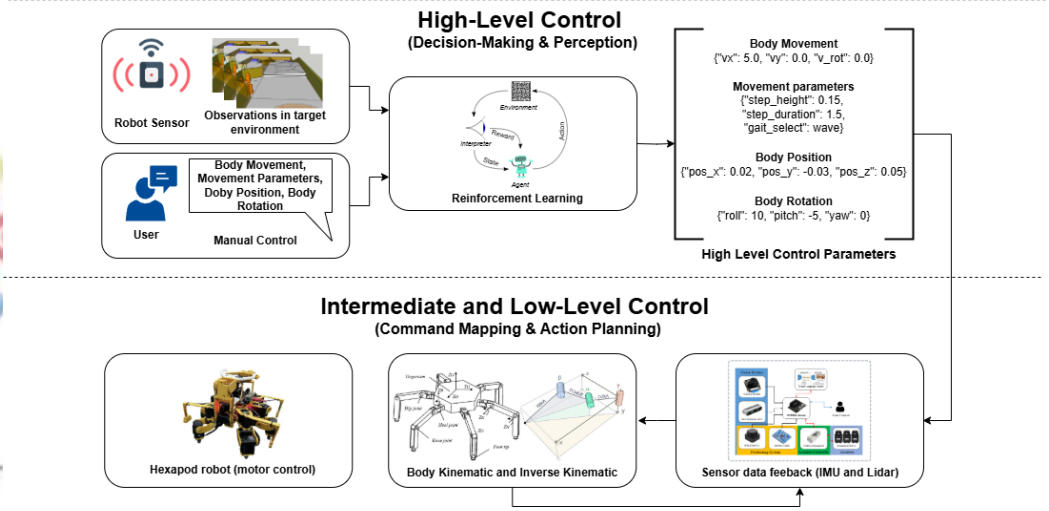
3.1 Diagram Alir Perancangan

Proses perancangan sistem robot hexapod akan dilaksanakan di Universitas Pertamina dengan waktu pelaksanaan selama empat bulan. Berikut merupakan tahap pelaksanaan perancangan sistem.



Gambar 3. 1 Flowchart Tahap Pelaksanaan

b. pengutipan hanya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang memublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 3. 2 Arsitektur Sistem

Arsitektur sistem terdiri dari tiga level kontrol yang bekerja secara berurutan untuk menggerakkan robot berdasarkan input gambar dan teks. *High-Level Control* berfungsi sebagai pengambil keputusan dan persepsi lingkungan *reinforcement learning*. Model ini menerima input dari parameter observasi yang digunakan pada *reinforcement learning* untuk kemudian memberikan aksi yang berisi parameter gerakan seperti kecepatan linear, rotasi, posisi tubuh, dan parameter langkah. Parameter ini kemudian dikirim ke level berikutnya untuk diproses lebih lanjut.

Intermediate and Low-Level Control berperan sebagai penerima output parameter kontrol. Algoritma *Body Kinematics* dan *Inverse Kinematics* menghitung pergerakan berdasarkan parameter yang diberikan. Dengan pendekatan ini, robot dapat bergerak secara otomatis mengikuti instruksi yang diberikan oleh pengguna, tanpa perlu pengendalian manual secara langsung.

3.2 Pertimbangan Perancangan

Perancangan robot hexapod berbasis *visual language model* untuk kendali gerakan adaptif dan navigasi otonom memiliki empat pertimbangan perancangan sebagai berikut.

a. Desain hardware robot didasarkan pada peraturan robot pada Kontes Robot SAR Indonesia (KRSRI) 2024 yaitu ukuran robot harus berada didalam lorong arena lapangan seperti pada Lampiran 1. Ukuran lebar Lorong adalah 45 cm sehingga dimensi panjang atau lebar robot tidak boleh melebihi 45 cm. Robot yang akan digunakan dapat dilihat pada Lampiran 2.

b. Rintangan yang akan diuji pada robot berdasar pada KRSRI 2024 yang meliputi jalan pecah, jalan miring, jalan berlumpur (jalan yang diisi kelereng), jalan berbatu, tangga, dan tanah longsor.

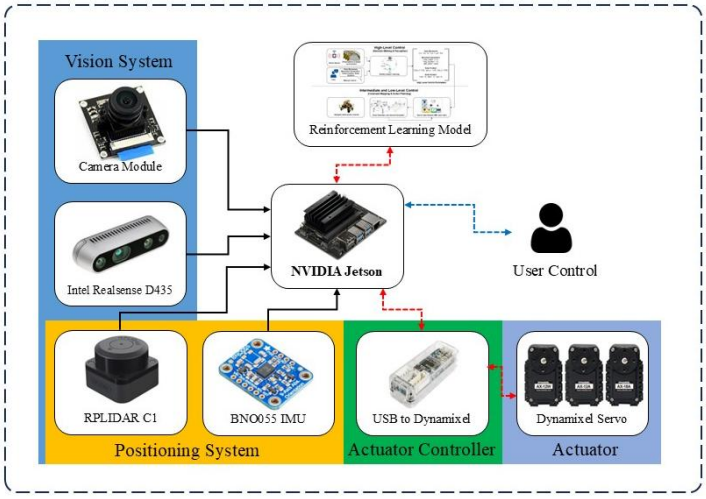
c. Algoritma program robot akan diterapkan dengan menggunakan bahasa pemrograman Python dan di integrasikan dengan ROS (Robot Operating System).
d. Pengembangan model RL berdasar pada penggunaan dan *tuning* model dengan metode PPO, SAC, dan TD3.

3.3 Analisis Teknis

Tahapan pengujian dan evaluasi diperlukan untuk mengetahui tingkat performa sistem robot hexapod dalam menjaga kestabilan dan kecepatan gerak dari hasil integrasi seluruh komponen sistem yang terdiri dari pembuatan *hardware*, *software*, serta model *machine learning*. Parameter yang akan diuji diantaranya analisis gerak robot berdasarkan hasil permodelan matematis *inverse kinematic, body kinematic,* dan *trajectory planning*. Analisis performa RL seperti analisis evaluasi *reward* dan *episode*. Selain itu uji fungsional sistem juga akan diuji untuk memastikan bahwa sistem telah memenuhi spesifikasi yang telah ditentukan serta bekerja dengan baik dan benar. Pengujian fungsional dilakukan dengan menganalisis kestabilan gerak robot berdasarkan sensor IMU (*Inertial Measurement Unit*), kecepatan robot dalam menyelesaikan misi rintangan, dan kemampuan robot dalam memahami perintah yang diberikan oleh *user*.

3.4 Peralatan dan Bahan

Peralatan dan bahan yang diperlukan untuk terwujudnya sistem robot hexapod terbagi menjadi dua bagian yaitu pertalatan dan bahan pembuatan *hardware* dan *software*. Peralatan pembuatan *hardware* terdiri dari perangkat keras yang dibutuhkan akan sistem dapat bekerja sesuai dengan yang ditentukan yaitu dapat mendeteksi dan melaporkan kejadian tindak kejahatan. Perkiraan komponen yang akan digunakan dalam sistem robot hexapod ditunjukkan pada Gambar 3.3 sebagai berikut.



Gambar 3. 3 Sistem *Hardware* Robot hexapod

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
© Copyright of Universitas Pertamina

Kemudian pada bagian peralatan dan pembuatan *software* dibutuhkan beberapa *tools* pendukung seperti aplikasi Fusion 360 untuk melakukan desain 3D, *Visual Studio Code* sebagai editor program, *Jupyter Notebook* dan *Google Colab* sebagai editor pembuatan model, dan beberapa software tambahan lainnya yang berpotensi meningkatkan kinerja dan analisis dari sistem robot hexapod.

3.5 Formulasi Reinforcement Learning untuk Kontrol Gait Hexapod

Untuk melatih robot hexapod agar dapat berjalan secara adaptif menggunakan Reinforcement Learning (RL), diperlukan pemodelan interaksi robot-lingkungan menggunakan Markov Decision Process (MDP). Komponen kunci pada MDP adalah: Observation Space (State), Action Space, dan Reward Function. Agen RL (menggunakan algoritma PPO, SAC, atau TD3) akan dilatih dalam lingkungan simulasi PyBullet untuk mempelajari kebijakan (policy) yang memetakan observasi ke aksi, dengan tujuan memaksimalkan cumulative reward. Bobot reward ditentukan secara iteratif. Awalnya disetel seimbang, lalu disesuaikan untuk menekankan stabilitas setelah pengamatan robot sering jatuh di awal pelatihan. Berikut merupakan rancangan desain Observation Space (State), Action Space, dan Reward Function yang dapat dilihat pada Tabel 3.1, 3.2, dan 3.3.

Tabel 3. 1 Desain *Observation Space (State)*

No	Observation Parameter	Min	Max	Keterangan
1	roll	-1.54	1.54	Rotasi sumbu X yang terdeteksi
2	pitch	-1.54	1.54	Rotasi sumbu Y yang terdeteksi
3	terrain_type	0	20	Jenis/kelas permukaan tanah

Tabel 3. 2 Desain *Action Space*

No	Action Parameter	Min	Max	Keterangan
1	x	-1	1	Posisi sumbu X
2	y	-1	1	Posisi sumbu Y
3	roll	-1	1	Rotasi sumbu X (roll)
4	pitch	-1	1	Rotasi sumbu Y (pitch)
5	step_height	0.03	0.1	Tinggi langkah kaki

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Tabel 3. 3 Desain *Reward Function*

No	Komponen	Bobot	Keterangan
1	stability_penalty	w1	Penalti untuk menjaga kestabilan tubuh (roll dan pitch)
2	energy_penalty	w2	Penalti atas penggunaan energi (misal: gerakan motor)
3	stuck_counter	W3	Penalti jika agen terlalu lama diam (macet), dikalikan dengan w4
Total Reward	reward = w1*stability + w2*energy + w3*(-stuck_counter)		

Pada observation space *terrain_type* pada penelitian didapatkan berdasarkan nilai IMU dari kemiringan robot (*pitch, roll*). Akan tetapi, pada implementasi dunia nyata, deteksi jenis rintangan (seperti tangga atau tanjakan) dilakukan secara tidak langsung melalui feedback orientasi dari sensor IMU. Sistem RL belajar mengenali kondisi ini berdasarkan perubahan nilai pitch dan roll, dan menyesuaikan parameter langkah serta jenis gait secara adaptif. Untuk pengembangan selanjutnya, modul deteksi visual atau depth sensor dapat ditambahkan untuk meningkatkan kemampuan persepsi medan secara eksplisit. Berikut merupakan pembagian *terrain_type* terhadap nilai kemiringan robot (*pitch, roll*).

Tabel 3. 4 Desain *Reward Function*

Terrain Type	Threshold Roll (°)	Threshold Pitch (°)
0	0 < roll < 5	0 < pitch < 5
1	5 ≤ roll < 10	5 ≤ pitch < 10
2	10 ≤ roll < 15	10 ≤ pitch < 15
3	15 ≤ roll < 20	15 ≤ pitch < 20
4	20 ≤ roll < 25	20 ≤ pitch < 25
5	roll ≥ 25	pitch ≥ 25

Pertamina
Universitas Pertamina
Copyright © Universitas Pertamina
1. Dilarang menggunakan karya tulis ini kecuali:
a. menyebutkan sumber sesuai kaidah penulisan karya ilmiah atau penelitian;
b. pengutipan untuk keperluan pendidikan
c. pengutipan dalam rangka penulisan karya ilmiah atau penelitian;
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

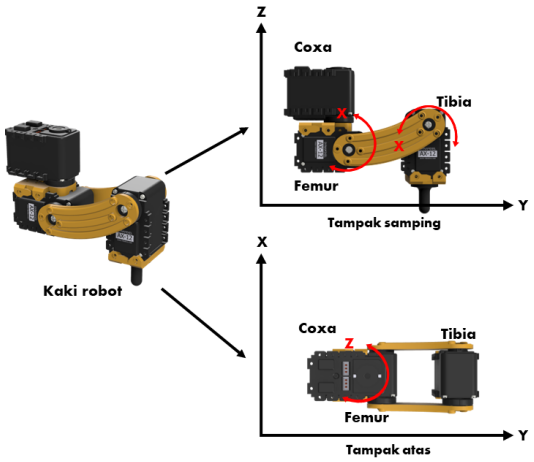
BAB IV
HASIL DAN PEMBAHASAN

BumbleBee merupakan robot hexapod pencari dan penyelamat yang memanfaatkan algoritma Reinforcement Learning (RL) untuk mengatur parameter pergerakannya seperti posisi tubuh, rotasi, tinggi langkah, dan jenis gait secara adaptif berdasarkan kondisi lingkungan pasca bencana yang kompleks; dengan memanfaatkan data dari sensor IMU, RL memungkinkan robot untuk belajar menjaga keseimbangan, efisiensi energi, serta kemampuan navigasi dalam mendeteksi dan mengevakuasi korban ke zona aman, menjadikan BumbleBee lebih fleksibel dan andal dibandingkan sistem kontrol statis konvensional.

Desain BumbleBee terdiri atas beberapa bagian utama yaitu: baseplate badan atas dan bawah, kaki robot, dan gripper seperti pada Gambar 4.1. Pada bagian kaki robot terdiri dari coxa, femur dan tibia seperti yang ditunjukkan pada Gambar 4.2. Robot ini didesain dengan menggunakan aplikasi Autodesk Fusion 360. Material robot yang berbahan dasar *Polylactic Acid* (PLA) dari hasil 3D printing objek yang telah dibentuk sesuai dengan desain yang telah dibuat.



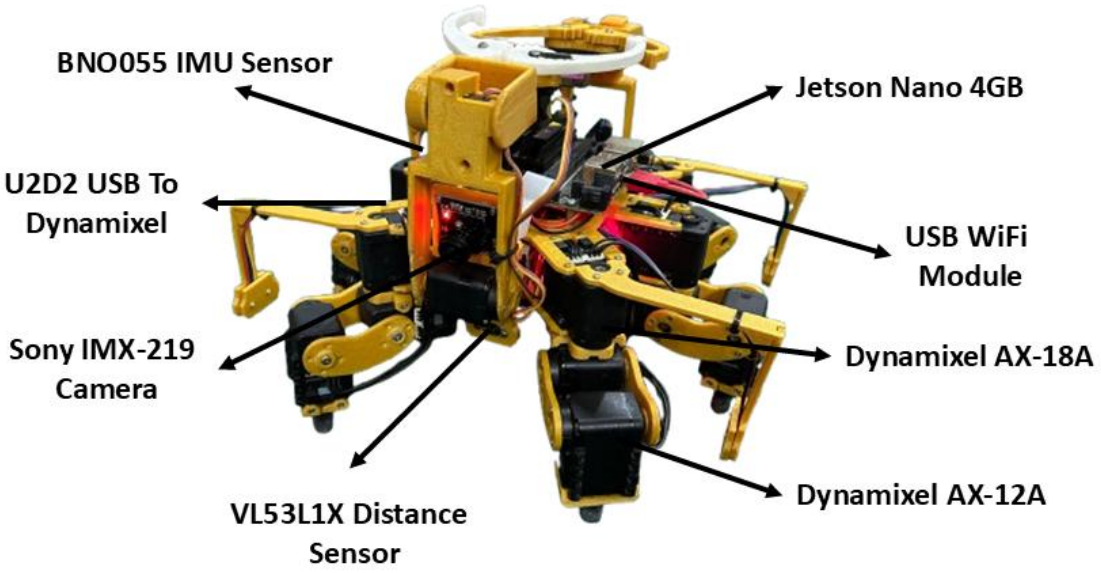
Gambar 4. 1 Bagian Utama BumbleBee



Gambar 4. 2 Bagian Kaki BumbleBee

Universitas Pertamina
Hak Cipta Universitas Pertamina dilindungi Undang-Undang
© Copyright of Universitas Pertamina
1. Dilarang menyebarluaskan karya tulis ini, kecuali:
Pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
Pengutipan tidak merugikan kepentingan Universitas Pertamina.
2. Dilarang memublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

BumbleBee didesain memiliki enam kaki, dua *gripper*, dan beberapa komponen utama untuk mendukung penyelesaian misi. Peletakan enam kaki pada robot yang menyerupai bentuk laba-laba untuk menjaga kestabilan robot dalam berjalan dan menyelesaikan misi. *Gripper* didesain untuk dapat melakukan misi penyelamatan korban pada arena yang dipasang pada bagian depan robot. Komponen utama pada robot terbagi atas *controller*, sensor, dan aktuator. Hasil desain dari BumbleBee dapat dilihat pada Gambar 4.3 dan 4.4. Kemudian untuk desain spesifikasi dari robot dapat dilihat pada Tabel 4.1.



Gambar 4. 3 BumbleBee



Gambar 4. 4 BumbleBee full-view

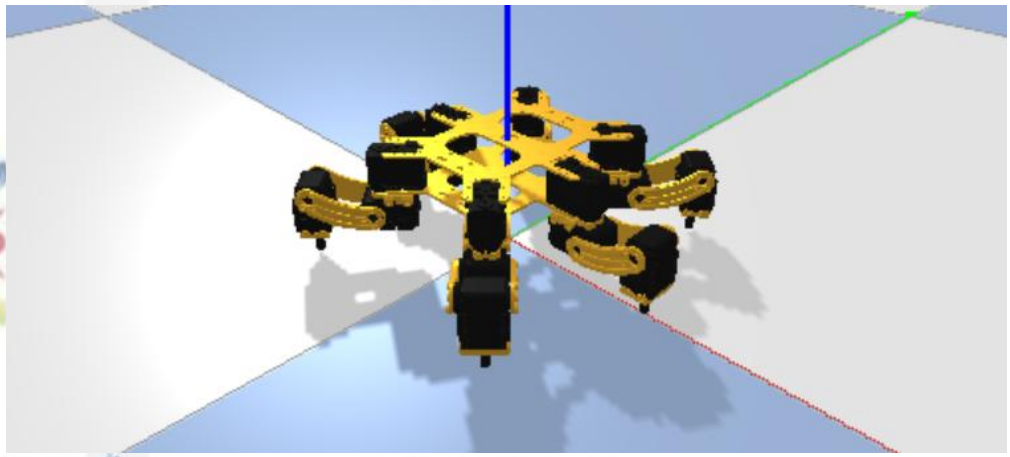
Pertamina
Universitas
Hak Cipta Dilindungi Undang-Undang
1. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
2.

Tabel 4. 1 Spesifikasi BumbleBee

Spesifikasi	
Nama Robot	BumbleBee
Fungsi	Melakukan pencarian korban bencana
Dimensi	Panjang 300mm x Lebar 280mm x Tingi 170mm
Material	*Polylactic Acid* (PLA)
Berat	2 kg
Processor	Jetson Nano B01 4GB
Memori	4GB LPDDR4 RAM
Kapasitas	32 GB Micro SD Card
Modul *Gyroscope*	BNO055 Gyroscope Sensor
Modul Kompas	BNO055 Compass Sensor
Motor	Dynamixel AX-12A and AX-18A Smart Servo
Interface	128x64 Pixel OLED LCD Display
Suhu Kerja	0-50ºC
Konektivitas	2.4 GHz and 5.0 GHz IEEE 802.11b/g/n/ac wireless LAN, Bluetooth 5.0, BLE Gigabit Ethernet, 2 × USB 3.0 ports 2 × USB 2.0 ports
Tegangan Kerja	12V DC 10A

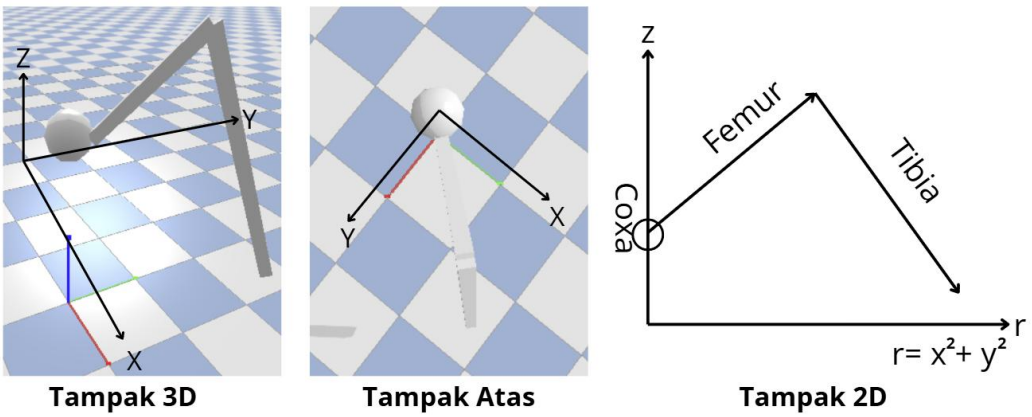
Dalam perancangan ini, sistem simulasi pergerakan robot hexapod dikembangkan menggunakan PyBullet sebagai *physics engine* tiga dimensi untuk menghasilkan simulasi gerak yang realistis. PyBullet dipilih karena kemampuannya dalam menyediakan simulasi fisika yang realistis, ringan untuk dijalankan pada Jetson Nano, dan kompatibel dengan pustaka RL seperti Gym dan Stable-Baselines3. Selain itu, PyBullet mendukung URDF, memungkinkan integrasi langsung dengan model robot fisik, serta menyediakan rendering visualisasi untuk mempermudah debugging selama proses pelatihan. Model URDF yang digunakan pada simulasi merupakan duplikasi dari model real robot yang bertujuan untuk mempermudah proses simulasi dan training. Berikut merupakan model URDF yang digunakan pada PyBullet untuk melakukan simulasi robot hexapod.

Pertamina
Universitas Pertamina
Hak Cipta milik Universitas Pertamina
Dilindungi Undang-Undang
© Copyright of Universitas Pertamina
1. Dilarang menggandakan karya tulis ini, kecuali:
a. menyebutkan sumber sebagai kaidah kecendekiaan
b. penggunaan untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
c. penggunaan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



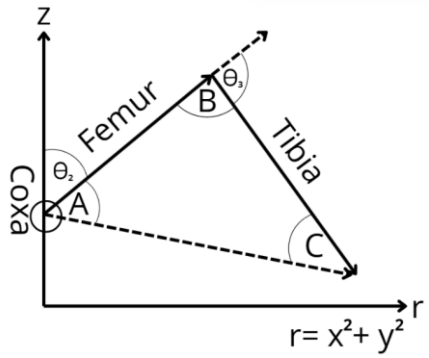
Gambar 4. 5 Model URDF Robot Hexapod

Selanjutnya, inverse kinematics diterapkan untuk menentukan sudut yang dibutuhkan pada masing-masing sendi kaki (coxa, femur, dan tibia) agar kaki dapat mencapai target posisi yang telah dihitung berdasarkan hasil body kinematics dan lintasan gerakan kaki (trajectory). Lintasan ini dibentuk berdasarkan metode sinusoidal untuk menghasilkan pergerakan naik-turun (lift-swing) yang halus pada fase langkah, dikombinasikan dengan komponen translasi dan rotasi tubuh yang diatur dalam parameter gerak. Metode *body kinematics* digunakan untuk menghitung transformasi posisi dan orientasi tubuh robot terhadap posisi dasar masing-masing kaki, dengan mempertimbangkan translasi dan rotasi tiga derajat kebebasan (DoF) pada sumbu x, y, dan z. Berikut merupakan persamaan inverse kinematic dan body kinematic yang digunakan pada robot hexapod.



Gambar 4. 6 Permodelan Inverse Kinematic pada Kaki Robot

Inverse kinematic merupakan proses matematis untuk menentukan nilai sudut masing-masing aktuator (servo motor) berdasarkan posisi yang diinginkan dari end-effector, dalam hal ini ujung kaki robot, yang dinyatakan dalam koordinat kartesian (X, Y, Z). Proses ini merupakan kebalikan dari *forward kinematic*, yang menghitung posisi end-effector berdasarkan sudut masing-masing sendi.



Gambar 4. 7 Hubungan Sudut Servo pada Kaki Robot

Pada sistem kaki robotik yang terdiri dari tiga derajat kebebasan (3-DOF), terdapat tiga buah servo yang berperan dalam mengatur pergerakan kaki, yaitu:

1. Servo Coxa (θ_1): Berfungsi untuk mengatur rotasi horizontal dari kaki terhadap tubuh robot, menentukan orientasi kaki dalam bidang X-Y.
2. Servo Femur (θ_2): Mengatur sudut elevasi femur (bagian tengah kaki), yang mempengaruhi tinggi dan jangkauan kaki pada sumbu Z.
3. Servo Tibia (θ_3): Mengontrol sudut tibia (bagian ujung kaki), yang menentukan posisi vertikal akhir dari end-effector.

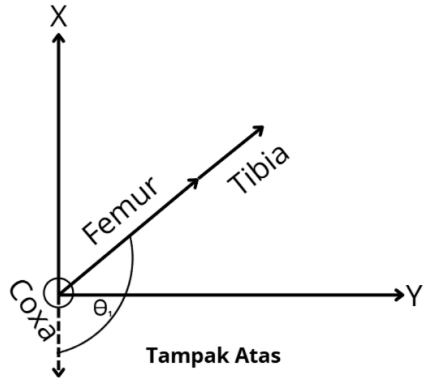
Sudut Coxa (θ_1) merupakan sudut antara sumbu tubuh robot dan garis lurus dari titik asal (pusat rotasi Coxa) ke proyeksi posisi target kaki pada bidang horizontal (X-Y). Dengan kata lain, sudut ini menentukan arah kaki menyamping terhadap sumbu utama tubuh. Secara geometris, sudut ini dapat ditentukan dengan fungsi trigonometri sebagai berikut:

$$\theta_1 = 90° + \arctan\left(\frac{x}{y}\right) \tag{4.1}$$

θ_1: Sudut rotasi coxa terhadap sumbu vertikal.

x: Posisi target pada sumbu X.

y: Posisi target pada sumbu Y.



Gambar 4. 8 Penentuan Sudut Coxa Kaki Robot

Hak Cipta milik Universitas Pertamina. Dilindungi Undang-Undang. Copyright of Universitas Pertamina
1. Dilarang menggunakan karya tulis ini untuk kepentingan komersial tanpa izin tertulis;
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
c. Pengutipan tidak merugikan Universitas Pertamina.

Kemudian sudut femur dihitung berdasarkan posisi vertikal (Z) dan jarak horizontal dari target terhadap titik pangkal kaki. Perhitungan ini melibatkan beberapa tahap:

Hitung jarak horizontal r:

$$r = \sqrt{x^2 + y^2} \tag{4.2}$$

Hitung jarak total d dari pangkal ke target:

$$d = \sqrt{r^2 + z^2} \tag{4.3}$$

Hitung sudut a_1:

$$a_1 = \arctan\left(\frac{z}{r}\right) \tag{4.4}$$

Hitung sudut A dengan hukum cosinus:

$$A = \arccos\left(\frac{femur^2 + d^2 - tibia^2}{2 \cdot femur \cdot d}\right) \tag{4.5}$$

Sehingga, didapatkan sudut femur:

$$\theta_2 = 90° - (A + a_1) \tag{4.6}$$

θ_2: Sudut antara femur dan sumbu horizontal.

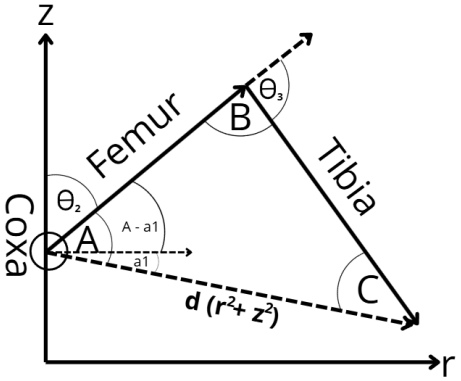
z: Posisi target pada sumbu vertikal.

r: Jarak horizontal dari coxa ke target.

d: Jarak total dari coxa ke posisi target.

$femur$: Panjang segmen femur.

$tibia$: Panjang segmen tibia.



Gambar 4. 9 Penentuan Sudut Femur dan Tibia Kaki Robot

Sedangkan sudut tibia dihitung menggunakan hukum cosinus dari segitiga yang dibentuk oleh segmen femur, tibia, dan jarak d:

$$B = \arccos\left(\frac{femur^2 + tibia^2 - d^2}{2 \cdot femur \cdot tibia}\right) \tag{4.7}$$

Universitas Pertamina
Hak Cipta Milik Universitas Pertamina
Dilindungi Undang-Undang
© Copyright of Universitas Pertamina
1. Dilarang menyutip karya tulis ini, kecuali:
a. menyebutkan sumber sesuai kaidah keilmiahan;
b. pengutipan hanya untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Sehingga sudut tibia adalah:

$$\theta_3 = 180° - B \tag{4.8}$$

θ_3: Sudut antara femur dan tibia.

B: Sudut segitiga antara femur dan tibia.

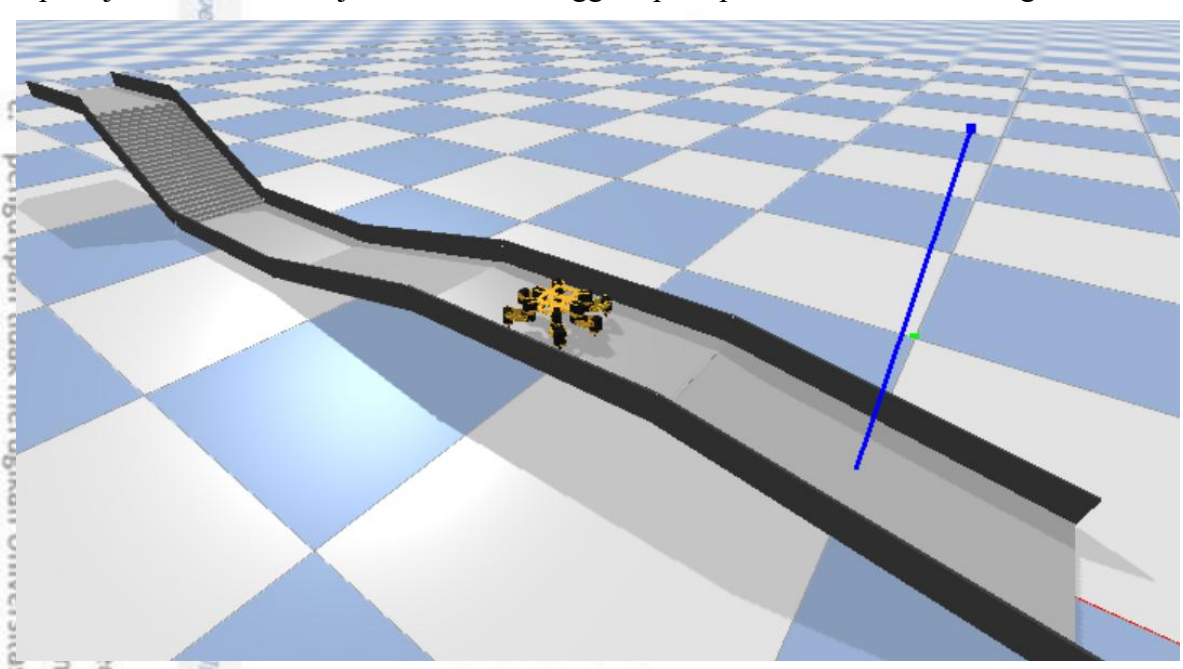
Selain menerapkan teknik kinematika pada robot, sistem ini juga mengimplementasikan kontrol gait menggunakan *phase shifting* dengan beberapa mode langkah seperti *wave gait, tripod gait, dan bipod gait* untuk menghasilkan pola koordinasi gerakan antar kaki yang berbeda. Berdasarkan pendekatan ini, sistem menghasilkan antarmuka parameter yang dapat dikontrol berupa: *Body Position* yang terdiri dari pos_x, pos_y, dan pos_z dalam rentang [-1, 1] meter; *Body Rotation* yang mencakup roll, pitch, dan yaw dalam rentang [-1, 1] radian; serta *Movement Parameters* seperti step_height (0 hingga 1 meter), step_duration (0 hingga 2 detik), dan gait_select yang dapat dipilih antara *wave*, *tripod*, atau *bipod* gait, sehingga memungkinkan kontrol fleksibel terhadap perilaku pergerakan hexapod secara dinamis dan real-time.

Tabel 4. 2 Parameter Kontrol BumbleBee

Kategori	Parameter	Rentang Nilai	Satuan	Deskripsi
Body Position	pos_x	-5 to 5	cm	Posisi tubuh pada sumbu X
	pos_y	-5 to 5	cm	Posisi tubuh pada sumbu Y
	pos_z	-5 to 5	cm	Posisi tubuh pada sumbu Z (ketinggian)
Body Rotation	roll	-20 to 20	derajat	Rotasi tubuh terhadap sumbu X
	pitch	-20 to 20	derajat	Rotasi tubuh terhadap sumbu Y
	yaw	-20 to 20	derajat	Rotasi tubuh terhadap sumbu Z
Movement Parameters	step_height	0 to 5	cm	Tinggi langkah kaki saat bergerak
	step_duration	0 to 2	detik	Durasi satu siklus langkah (semakin kecil, semakin cepat)
	gait_select	wave, tripod, bipod	-	Jenis pola langkah kaki yang digunakan dalam gerakan

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Berdasarkan pada parameter robot yang terdapat pada Tabel 4.2 diatas, maka gerakan robot dapat dikontrol secara otomatis sehingga memungkinkan robot menyesuaikan gerakannya secara cerdas. Oleh karena itu, diperlukan penerapan metode Reinforcement Learning (RL). Pendekatan ini melatih robot untuk menemukan strategi kontrol yang optimal, dan hasil pelatihan awal menggunakan RL akan dibahas selanjutnya. Lingkungan simulasi robot akan didasarkan pada rintangan KRSRI 2024 yang terdiri dari rintangan seperti jalan datar, menanjak, turun, dan tangga seperti pada Gambar 4.10 sebagai berikut.



Gambar 4. 10 Environment Simulasi Robot

Lingkungan simulasi pada perancangan ini dikembangkan menggunakan *library PyBullet* dan *Gym* yang dirancang khusus untuk mendukung pelatihan algoritma *Reinforcement Learning* (RL). Algoritma ini mengimplementasikan semua komponen utama dari RL seperti *observation space*, *action space*, serta *reward function* untuk mencapai kestabilan dan efisiensi gerak robot. *Observation space* terdiri dari parameter *roll* dan *pitch* tubuh robot, yang diukur langsung dari orientasi tubuh (*quaternion*) dan dikonversi ke euler angle. Kedua nilai ini cukup untuk merepresentasikan kestabilan robot selama pelatihan. Sementara itu, action space didefinisikan sebagai perintah gerak tingkat tinggi dalam bentuk translasi (x, y) dan rotasi (*roll*, *pitch*) tubuh robot, yang kemudian ditransformasikan ke posisi target kaki menggunakan metode body kinematic dan inverse kinematic. Program python yang digunakan pada environment robot dapat dilihat pada Lampiran 4.

Salah satu inovasi penting dari lingkungan ini adalah kemampuan adaptasi gait secara otomatis berdasarkan kondisi medan. Dalam implementasinya, fungsi *calculate_reward()* tidak hanya menghitung penalti atas postur tidak stabil dan konsumsi energi, namun juga

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

mengubah parameter langkah seperti tinggi langkah (*step_height*), durasi langkah (*step_duration*), dan jenis gait (*gait_select*) secara real-time berdasarkan tingkat kestabilan tubuh dan deteksi stuck (*macet*). Sebagai contoh, jika robot mendeteksi *roll* dan *pitch* berada dalam rentang ±5°, serta tidak mengalami stuck, maka sistem akan mengaktifkan mode cepat dengan langkah yang rendah dan singkat (step height = 0.03, step duration = 0.3 detik). Sebaliknya, jika medan terdeteksi berat, robot akan beralih ke gait yang lebih lambat dan stabil untuk menghindari kegagalan.

Reward function pada sistem ini dirumuskan sebagai kombinasi dari tiga komponen utama, yaitu:

- Stability penalty berdasarkan akumulasi derajat kemiringan tubuh (roll dan pitch),
- Energy penalty berdasarkan estimasi daya (torque × velocity) dari seluruh joint,
- Stuck penalty berdasarkan jumlah langkah di mana robot tidak bergerak signifikan.

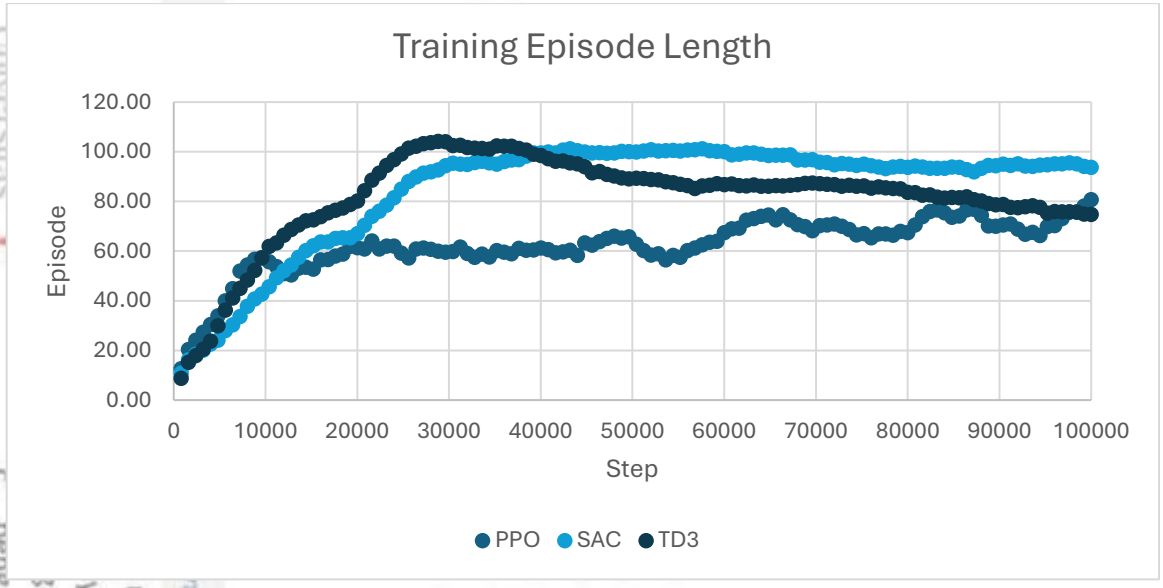
Dengan pendekatan ini, sistem tidak hanya memaksimalkan performa RL dalam hal reward, namun juga mempertahankan interpretabilitas kontrol fisik, menjadikannya cocok untuk diimplementasikan pada robot riil. Perilaku adaptif terhadap lingkungan ini merupakan aspek kunci yang membuat model hasil pelatihan mampu menyelesaikan semua rintangan nyata secara konsisten dengan stabilitas tinggi. Berikut merupakan table bobot reward yang ditetapkan pada *environment training*.

Tabel 4. 3 Desain Nilai *Reward Function*

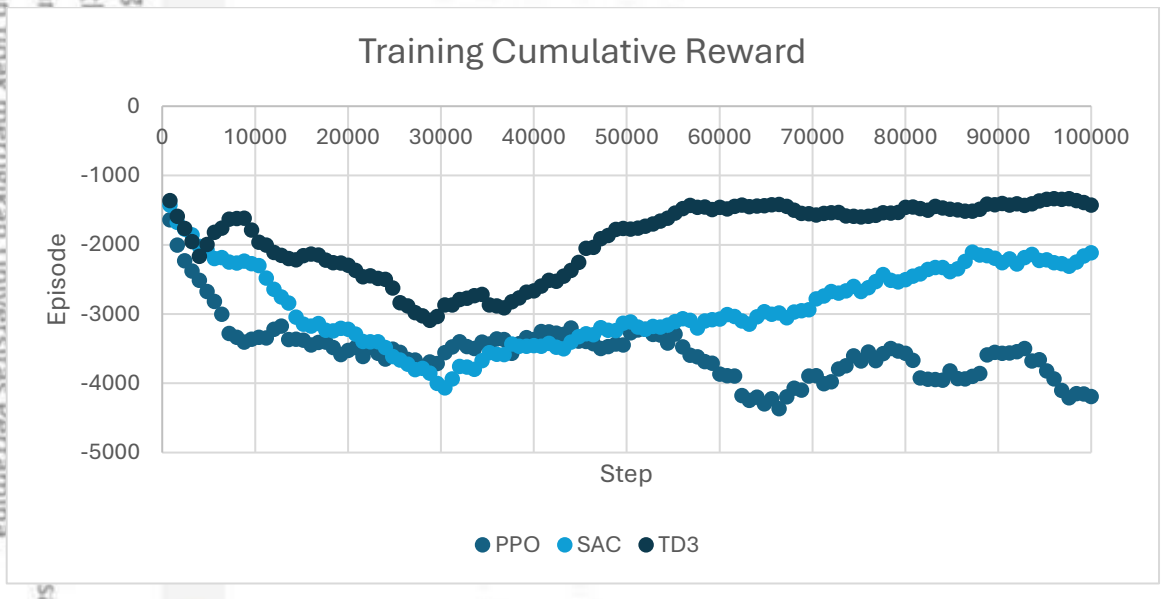
No	Komponen	Bobot	Nilai	Keterangan
1	stability_penalty	w1	2	Penalti untuk menjaga kestabilan tubuh (roll dan pitch)
2	energy_penalty	w2	0.5	Penalti atas penggunaan energi (misal: gerakan motor)
3	stuck_counter	W3	2	Penalti jika agen terlalu lama diam (macet), dikalikan dengan w4
Total Reward		reward = w1*stability + w2*energy + w3*(-stuck_counter)		

Berdasarkan hasil pengujian menggunakan environment yang telah dibuat menggunakan PyBullet dan Stable-Baselines3, didapatkan hasil pengujian (*training*) sebagai berikut.

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 11 Hasil Training Episode Length



Gambar 4. 12 Hasil Training Cumulative Reward

Hasil pelatihan awal selama 100.000 langkah interaksi (Gambar 4.11 dan 4.12) mengonfirmasi bahwa ketiga algoritma (PPO, SAC, TD3) berhasil mempelajari perilaku lokomosi dasar, terlihat dari peningkatan panjang episode dan perbaikan *cumulative reward*. Dalam hal optimalisasi *cumulative reward* sesuai fungsi yang dirancang, TD3 menunjukkan performa akhir terbaik (-1300), mengungguli SAC (-2200) dan PPO (-4400). Namun, SAC memperlihatkan stabilitas durasi episode yang paling unggul di akhir pelatihan (90-95 langkah/episode), menunjukkan potensi gait yang lebih *robust* dalam menghindari terminasi, meskipun PPO dan TD3 sempat mencapai puncak episode length yang lebih tinggi sebelumnya. Observasi ini mengindikasikan adanya *trade-off* potensial, di mana TD3 lebih efektif mengoptimalkan metrik performa gabungan dalam *reward function*, sementara SAC

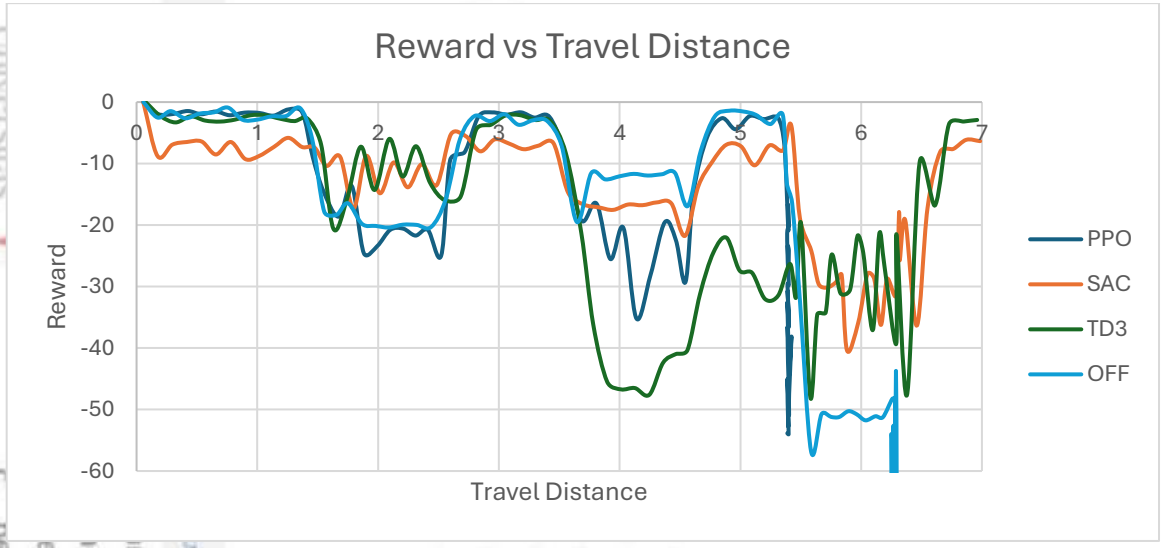
lebih memprioritaskan durasi stabilitas. Model hasil *training*, selanjutnya akan diuji dengan pada environment simulasi yang sama untuk mengetahui performa robot dalam kondisi waktu *realtime*.

Berdasarkan hasil testing yang dilakukan, terlihat bahwa ketiga algoritma menunjukkan performa yang berbeda-beda dalam menghadapi berbagai rintangan. SAC menonjol sebagai algoritma terbaik dengan keberhasilan melewati semua rintangan dan total reward tertinggi dan waktu tercepat. Hal ini disebabkan oleh kemampuannya menjaga stabilitas melalui kontrol yang halus terhadap orientasi tubuh robot (*state_roll* dan *state_pitch*), serta adaptasi dinamis pada tinggi langkah (*action_step_height*). Selain itu, algoritma SAC juga menunjukkan konsistensi dalam mengoptimalkan reward, dengan *stability_penalty* yang terkontrol dan *distance_penalty* rendah, menandakan gerakan yang efisien.

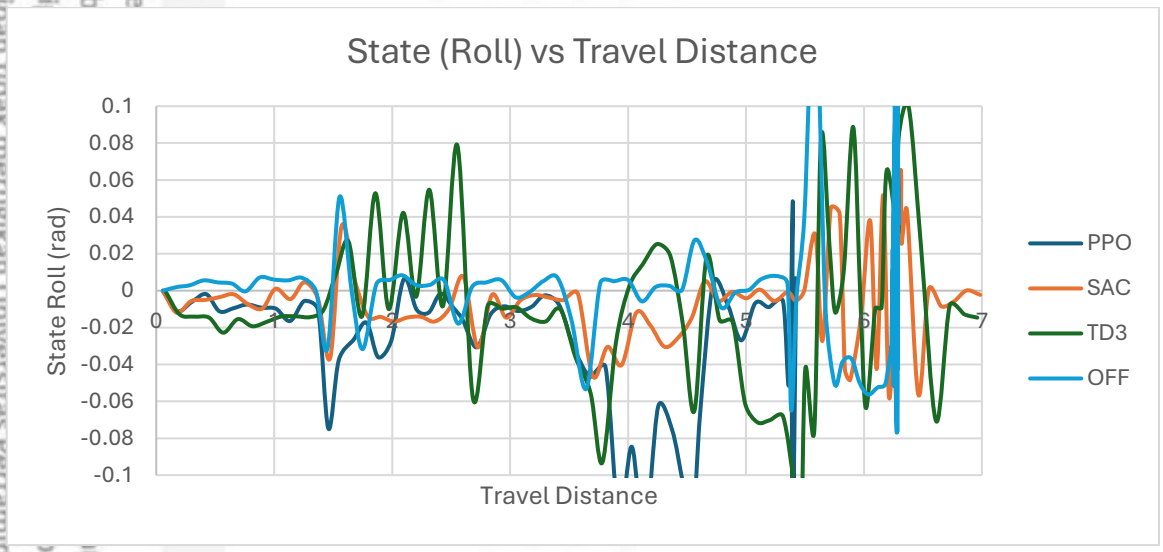
Di sisi lain, TD3 (Twin Delayed DDPG) berhasil menyelesaikan semua rintangan tetapi dengan total reward lebih rendah dibanding SAC. Data pengujian menunjukkan bahwa TD3 cenderung menghasilkan aksi yang lebih agresif, seperti perubahan mendadak pada *action*_pitch, yang mengakibatkan *stability_penalty* lebih tinggi. Meski demikian, TD3 tetap lebih unggul daripada PPO (Proximal Policy Optimization), yang gagal menghadapi rintangan ke-4 (tangga) karena ketidakmampuan menjaga keseimbangan. PPO menunjukkan lonjakan tajam pada *state_pitch* saat menghadapi medan sulit, mengindikasikan ketidakstabilan yang kritis sehingga tidak cocok digunakan dalam tahap pengujian selanjutnya yang akan diuji pada *real world* menggunakan hardware robot. Sementara itu, mode OFF (baseline) yang tidak menggunakan kontrol algoritma RL sama sekali gagal total, dengan robot terjungkal terus-menerus dan reward yang sangat negatif. Ini mempertegas pentingnya penggunaan algoritma RL untuk tugas lokomosi yang kompleks.

Dari segi grafik, SAC menampilkan osilasi yang terkontrol pada orientasi tubuh, sementara TD3 dan PPO cenderung fluktuatif. Grafik reward kumulatif SAC juga lebih stabil tanpa penurunan drastis, berbeda dengan PPO yang mengalami drop tajam saat gagal (pada rintangan tangga/travel distance 6.4 meter). Untuk implementasi nyata, SAC adalah pilihan terbaik karena stabilitas dan adaptabilitasnya, meski TD3 masih bisa dipertimbangkan dengan penyempurnaan pada efisiensi energi. Sedangkan algoritma PPO tidak direkomendasikan kecuali dilakukan modifikasi signifikan pada arsitekturnya. Langkah selanjutnya adalah menguji SAC dalam lingkungan fisik dengan untuk memvalidasi model dapat bekerja pada lingkungan real. Data hasil pengujian testing simulasi dapat dilihat pada Gambar 4.13, 4.14, dan 4.15.

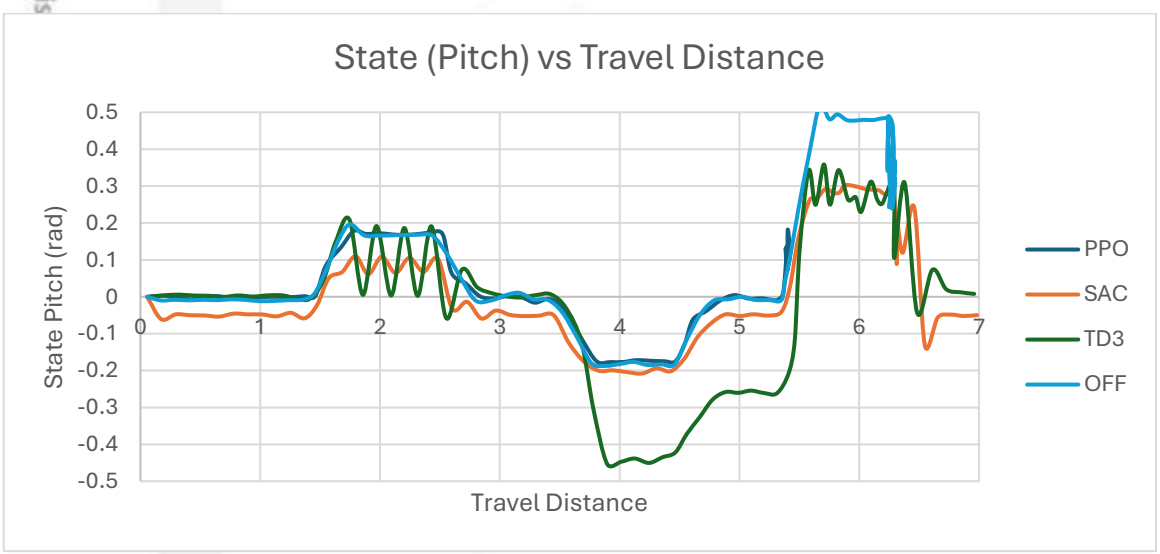
Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 13 Hasil Testing (PPO vs SAC vs TD3) terhadap Reward



Gambar 4. 14 Hasil Testing (PPO vs SAC vs TD3) Terhadap State (Roll)



Gambar 4. 15 Hasil Testing (PPO vs SAC vs TD3) Terhadap State (Pitch)

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Dapat dilihat pada Gambar 4.13 pada pengujian di lingkungan nyata, algoritma PPO menunjukkan performa yang kurang stabil, terutama saat robot dihadapkan pada medan dinamis seperti tangga. Kegagalan algoritma ini disebabkan oleh mekanisme pembaruan kebijakan PPO yang dibatasi oleh fungsi klip, sehingga perubahan yang terlalu ekstrem dari kebijakan lama akan ditekan. Secara matematis, PPO mengoptimasi fungsi objektif berikut:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r(\theta)\cdot \hat{A},\ \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)\cdot \hat{A}\right)\right] \quad (4.9)$$

- θ : parameter (isi dari model AI)
- $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$: perbandingan antara kebijakan baru dan lama
- $\hat{A}$: advantage (keuntungan aksi tertentu dibanding rata-rata)
- ϵ : batas toleransi perubahan (biasanya 0.1 sampai 0.2)
- clip(·) : fungsi pembatas, agar rasio tidak terlalu jauh dari 1
- $\mathbb{E}[\cdot]$: rata-rata dari semua data (ekspektasi)

Berdasarkan persamaan 4.9 yang merupakan simplifikasi dari algortima PPO, $r(\theta)$ adalah rasio antara kebijakan baru dan kebijakan lama, yang menunjukkan seberapa besar perubahan yang dilakukan terhadap aksi tertentu. Nilai $\hat{A}$ adalah advantage, yaitu ukuran seberapa menguntungkan suatu aksi dibandingkan dengan aksi rata-rata. Fungsi clip berperan untuk membatasi nilai $r(\theta)$ agar tidak terlalu jauh dari nilai 1, sebagai contoh hanya boleh berada dalam rentang 0.8 hingga 1.2 jika $\epsilon = 0.2$. Tujuannya adalah agar perubahan kebijakan yang dilakukan tidak terlalu ekstrem, sehingga proses pelatihan tetap aman dan tidak menyebabkan robot berperilaku tidak stabil. Fungsi *min* kemudian memilih nilai yang lebih kecil antara perubahan asli dan yang sudah dibatasi, untuk memastikan bahwa model tidak terlalu percaya diri terhadap aksi yang sangat berbeda dari kebijakan sebelumnya. Dengan pendekatan ini, PPO memungkinkan robot belajar secara bertahap dan menjaga kestabilan, meskipun hal ini kadang menjadi kelemahan saat menghadapi kondisi lingkungan yang berubah cepat. Hal ini yang menyebabkan algoritma PPO kurang baik untuk diterapkan di robot hexapod karena perubahan state yang cepat yang dibuktikan ketika menggunakan PPO, robot tidak berhasil untuk melakukan tugas menaiki tangga.

Sedangkan pada pengujian menggunakan TD3 menunjukkan kemampuan lebih baik dibanding PPO dalam menyelesaikan misi, karena mampu mengendalikan robot melewati semua rintangan. Namun, gerakan robot yang dihasilkan cenderung agresif dan kurang halus. TD3 menggunakan pendekatan deterministik dalam menghasilkan aksi dengan teknik clipped double Q-learning dan policy delay update yang mana tidak ada mekanisme eksplorasi berbasis entropi. Hal ini menyebabkan gerakan yang dihasilkan kadang berupa

perubahan sudut ekstrem, yang walaupun efektif, mengakibatkan konsumsi energi yang tinggi dan penurunan stabilitas. Persamaan inti dari TD3 dalam memperbarui nilai target adalah:

$$y = r + \gamma \cdot \min\big(Q_1(s', a'), Q_2(s', a')\big) \quad 4.10$$

- y : target nilai yang ingin dipelajari
- r : reward (nilai yang diperoleh dari aksi)
- γ : faktor diskonto (biasanya antara 0.9 - 0.99)
- Q_1, Q_2 : dua model yang memprediksi nilai (untuk mencegah overestimasi)
- $\min(\cdot)$: ambil nilai terendah dari dua prediksi
- s' : kondisi (state) berikutnya
- a' : aksi berikutnya

Pada persamaan 4.10 yang merupakan penyederhanaan dari algoritma TD3 ini, y adalah nilai target yang ingin dipelajari oleh model, yaitu nilai seberapa baik sebuah aksi jika diambil dari suatu kondisi tertentu. Nilai ini terdiri dari dua bagian: reward langsung r, yaitu nilai yang diterima setelah robot mengambil suatu aksi, dan nilai prediksi masa depan yang didiskon menggunakan faktor γ. Nilai γ adalah faktor diskonto yang menentukan seberapa jauh robot memperhatikan masa depan. Semakin besar nilai γ, semakin jauh robot mempertimbangkan dampak jangka panjang dari tindakannya. Kelebihan TD3 adalah menggunakan dua buah fungsi nilai, yaitu Q_1 dan Q_2, yang keduanya memprediksi nilai dari aksi berikutnya a' pada kondisi berikutnya s'. Nilai terendah dari dua prediksi tersebut yang digunakan untuk mencegah overestimasi, yaitu kondisi di mana model terlalu optimis terhadap nilai suatu aksi. Namun, dalam praktiknya, TD3 terkadang menghasilkan gerakan yang terlalu agresif. Hal ini dapat disebabkan oleh strategi eksplorasi berbasis noise atau karena policy yang terbentuk cenderung memilih aksi bernilai tinggi secara ekstrem, terutama ketika ruang aksi tidak dibatasi secara ketat.

Selanjutnya, SAC terbukti menjadi algoritma paling efektif dan stabil dalam pengujian ini. Robot yang dikendalikan dengan SAC menunjukkan gerakan yang halus, seimbang, dan hemat energi. Keunggulan SAC terletak pada penggunaan entropi sebagai bagian dari fungsi objektifnya, yang mendorong eksplorasi yang lebih aman dan tidak tergesa-gesa dalam mengambil keputusan. Dengan mempertimbangkan ketidakpastian dalam aksi yang diambil, robot lebih adaptif terhadap perubahan lingkungan tanpa mengorbankan stabilitas. Fungsi objektif SAC dirumuskan sebagai:

Pertamina
Hak Cipta milik Universitas Pertamina
1. Dilarang mengutip karya tulis ini, secara:
a. menyebutkan sumber asli dan tidak kepentingan
b. pengutipan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah atau penyusunan laporan
c. pengutipan tidak merugikan kepentingan Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

$$J(\pi) = \mathbb{E}\left[r + \alpha \cdot \mathcal{H}\left(\pi(\cdot\,|s)\right)\right] \quad 4.11$$

- $J(\pi)$: total tujuan yang ingin dimaksimalkan
- r: reward dari aksi
- α: bobot untuk bagian eksplorasi (entropi)
- $\mathcal{H}(\pi(\cdot\,|s))$: entropi, yaitu seberapa acak/beragam aksi dari kebijakan
- $\pi(\cdot\,|s)$: kebijakan (policy), yaitu pilihan aksi berdasarkan kondisi s

Persamaan ini menunjukkan bahwa tujuan utama SAC adalah memaksimalkan dua hal sekaligus: nilai reward r dari aksi yang diambil, dan nilai entropi H dari kebijakan π, yang mewakili keragaman atau ketidakpastian dalam pemilihan aksi. Dengan kata lain, SAC tidak hanya mengejar hasil yang baik (reward tinggi), tetapi juga mendorong agar kebijakan tetap fleksibel dan tidak terlalu cepat terpaku pada satu pola gerakan. Nilai α dalam persamaan berfungsi sebagai pengatur keseimbangan antara reward dan entropi. Semakin besar α, semakin besar dorongan eksplorasi terhadap aksi-aksi baru. Fungsi entropi ini sangat penting dalam situasi yang kompleks dan dinamis, karena membuat robot lebih mempertimbangkan aksi dan terbuka untuk mencoba berbagai kemungkinan sebelum memutuskan aksi terbaik. Dalam konteks pengujian, pendekatan ini membuat gerakan robot menjadi lebih halus, stabil, dan efisien, karena robot tidak hanya fokus menyelesaikan tugas secepat mungkin, tapi juga mempertimbangkan berbagai cara untuk melakukannya dengan aman dan hemat energi. Berikut merupakan Tabel 4.3 yang menunjukkan perbanding ketiga algoritma ini secara singkat.

Tabel 4. 4 Perbandingan PPO, TD3, dan SAC Serta Dampaknya ke Robot

Algoritma	Fokus Persamaan	Dampaknya ke Robot
PPO	Menjaga agar gerakan tidak berubah terlalu cepat dari sebelumnya	Stabil di kondisi biasa, tapi sulit adaptasi di medan ekstrim
TD3	Memilih aksi terbaik secara langsung, tapi pakai 2 Q-value untuk keamanan	Efektif dan cepat, tetapi gerakannya agresif
SAC	Memaksimalkan reward + keragaman aksi (entropy)	Gerakannya paling halus, stabil, dan efisien

Universitas Pertamina
Hak Cipta milik Universitas Pertamina
Pertamina Learning University
Universitas Pertamina
1. Dilarang mengutip karya tulis ini kecuali
a. menyebutkan sumber untuk kepentingan pendidikan, penelitian;
b. penggunaan hanya untuk keperluan pendidikan, penelitian;
c. penggunaan tidak merugikan kepentingan Universitas Pertamina.
2. Dilarang memperbanyak atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Selain itu, berdasarkan hasil pengujian yang dilakukan menggunakan fungsi *cost* untuk menghitung total reward dihasilkan tiap model yang ditentukan menggunakan persamaan sebagai berikut.

$$\text{Cost} = \int_{x=0}^{x=7} -\text{Reward}(x)\, dx \qquad 4.12$$

Fungsi ini digunakan untuk merepresentasikan efektivitas dari model yang diuji. Karena reward negatif menunjukkan penalti atau kondisi yang tidak diinginkan, maka semakin besar nilai negatif dari reward yang diterima selama perjalanan, semakin tinggi nilai cost yang dihasilkan. Melalui pendekatan ini, seluruh algoritma pembelajaran yang diuji termasuk PPO, SAC, TD3, dan OFF dapat dibandingkan secara kuantitatif berdasarkan akumulasi penalti yang mereka hasilkan selama proses evaluasi. Hasil perhitungan nilai cost untuk masing-masing algoritma ditampilkan dalam tabel berikut.

Tabel 4. 5 Perbandingan Reward PPO, TD3, dan SAC

Algoritma	Cost Reward
PPO	1551.89
TD3	280.13
SAC	22.53
OFF	3683.83

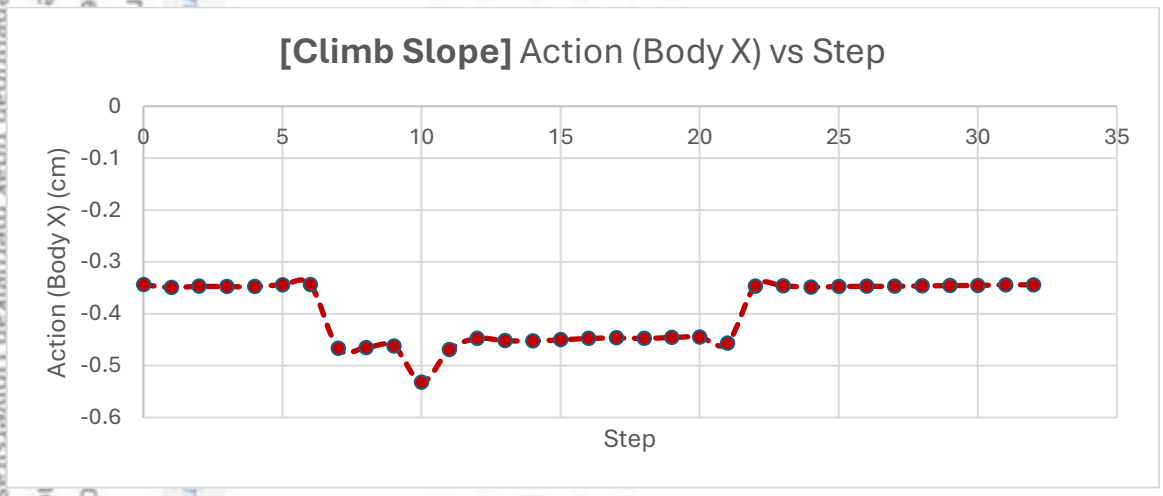
Meskipun reward SAC secara kumulatif lebih baik, terdapat fluktuasi negatif di awal pergerakan pada beberapa kondisi awal medan, seperti saat jalan datar. Hal ini dapat disebabkan oleh strategi eksplorasi agresif dari SAC yang memerlukan beberapa langkah adaptasi awal. Namun, setelah stabil, SAC menunjukkan konsistensi yang lebih tinggi. Untuk mengurangi penalti pada beberapa kondisi medan ini, tuning lebih lanjut pada beberapa parameter seperti entropi atau pre-training pada medan khusus dapat dilakukan sebagai langkah pengembangan selanjutnya.

Berdasarkan hasil analisis simulasi (*testing*) yang dilakukan, Soft Actor Critic (SAC) digunakan pada tahap implementasi karena memiliki performa terbaik diantara ketiga algoritma RL yang digunakan dibutktikan dengan *cost reward* paling minimum yaitu 22.53. Implementasi dilakukan menggunakan robot bumblebee dengan parameter yang telah disesuaikan dengan lingkungan simulasi yaitu jalan menanjak (*slope climbing*) dengan sudut

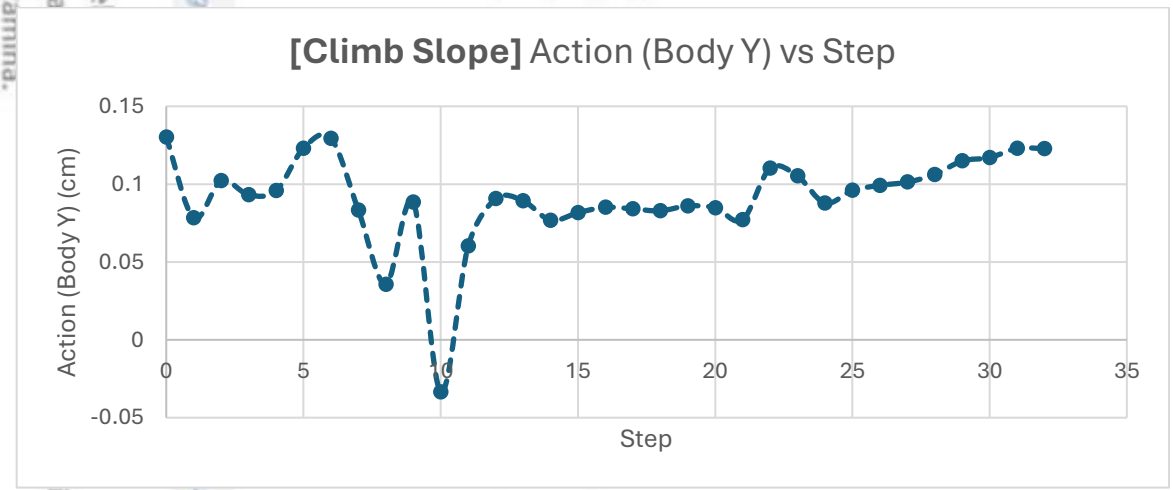
Hak Cipta milik Universitas Pertamina
Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

kemiringan 30 derajat, jalan menurun (*slope down*) dengan kemiringan 30 derajat, serta tangga (*climbing stairs*) dengan kemiringan 30 derajat. Data yang diperoleh dari hasil uji coba digunakan untuk menentukan performa robot seperti action, state, dan reward yang didapatkan robot.

Percobaan pada medan *slope climbing* menunjukkan performa yang cukup baik, robot berhasil menaiki tangga setelah 32 steps langkah. Data menunjukkan bahwa robot dapat menstabilkan body dan posisi, pada steps ke-5 robot mulai menanjak bidang miring yang dapat dilihat pada grafik *state_pitch* bahwa robot berusaha menstabilkan nilai *pitch*, kemudian steps ke-22 robot berhasil selesai menaiki tangga yang ditunjukkan nilai *pitch* yang kembali di titik *equilibrium*.

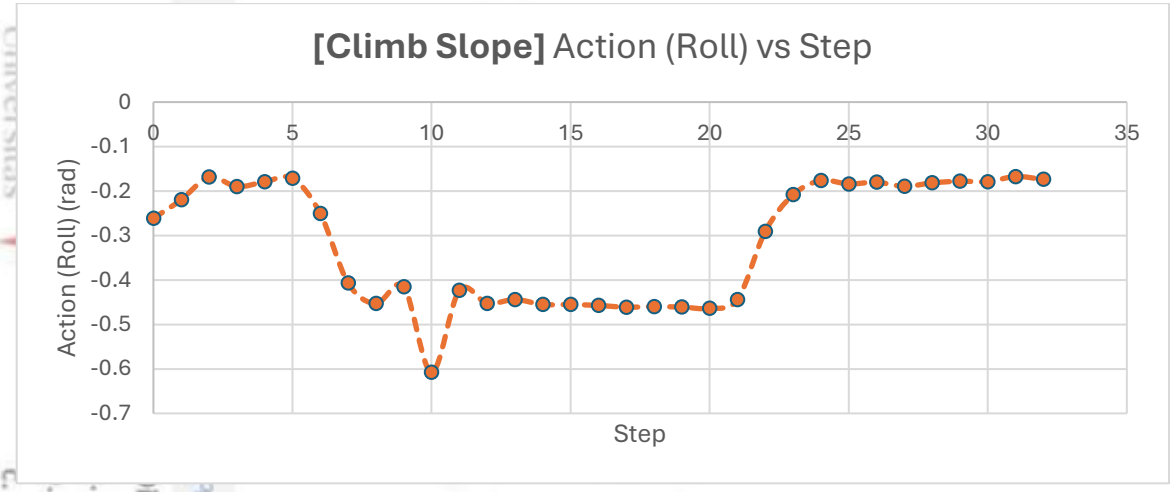


Gambar 4. 16 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope

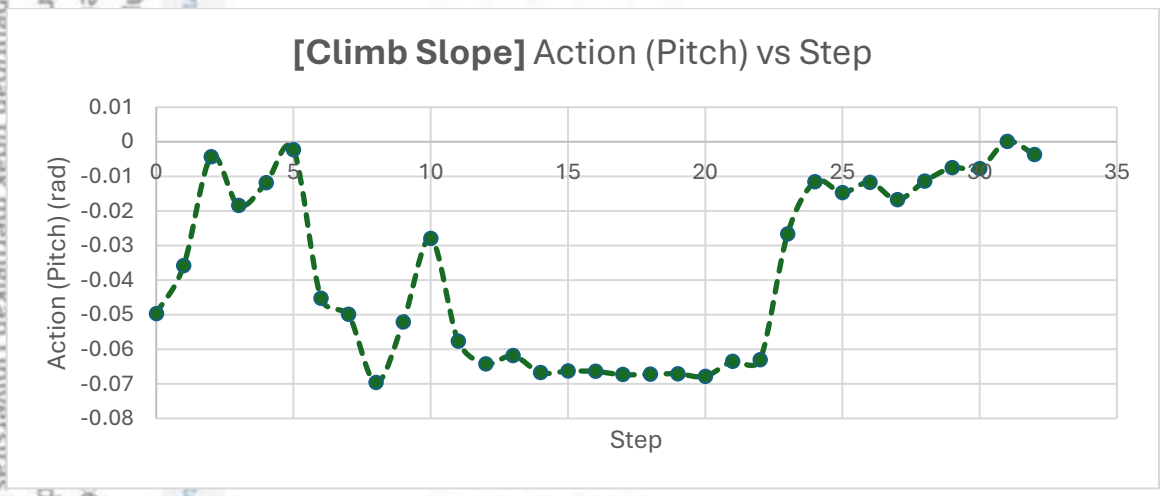


Gambar 4. 17 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope

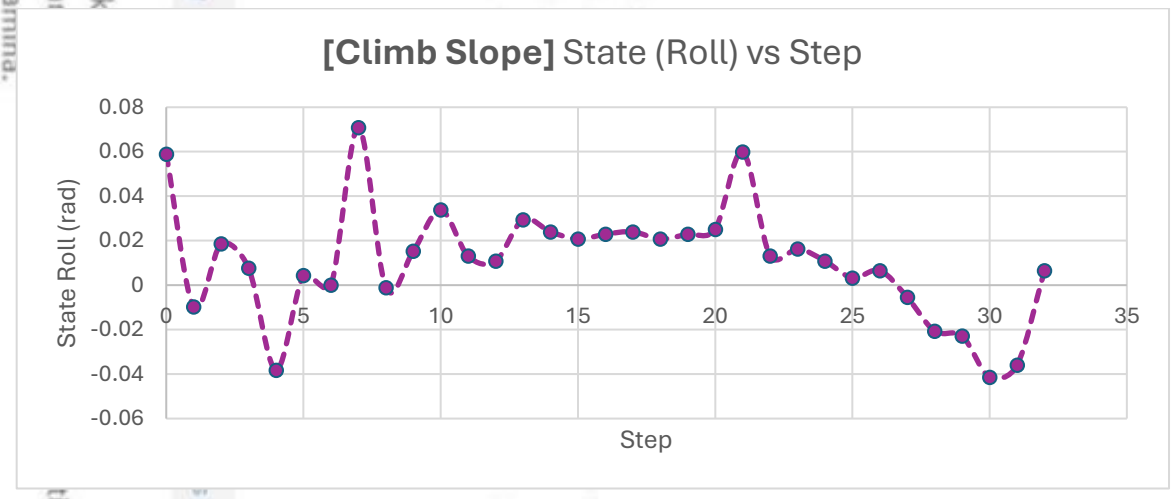
Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 18 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope

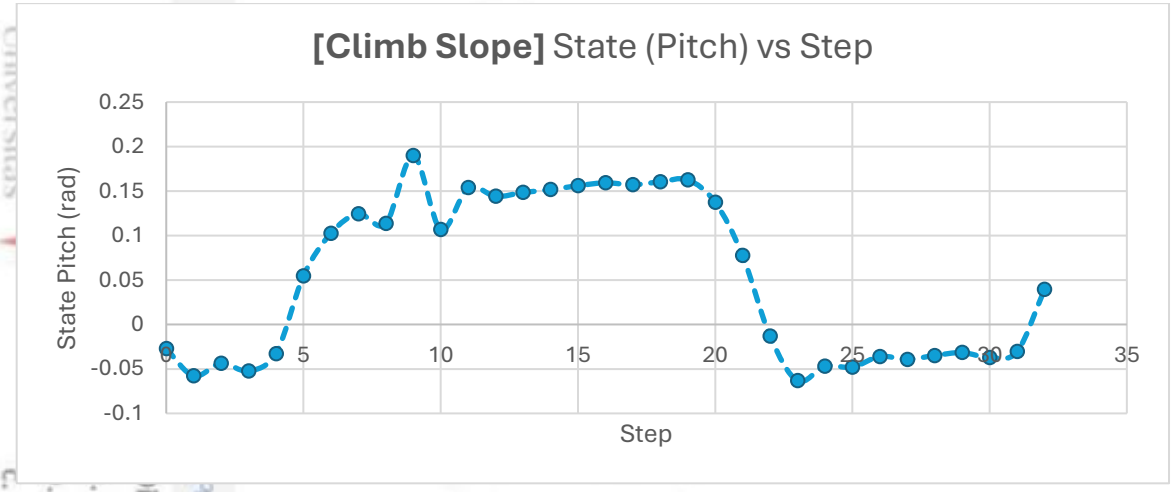


Gambar 4. 19 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope



Gambar 4. 20 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Slope

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

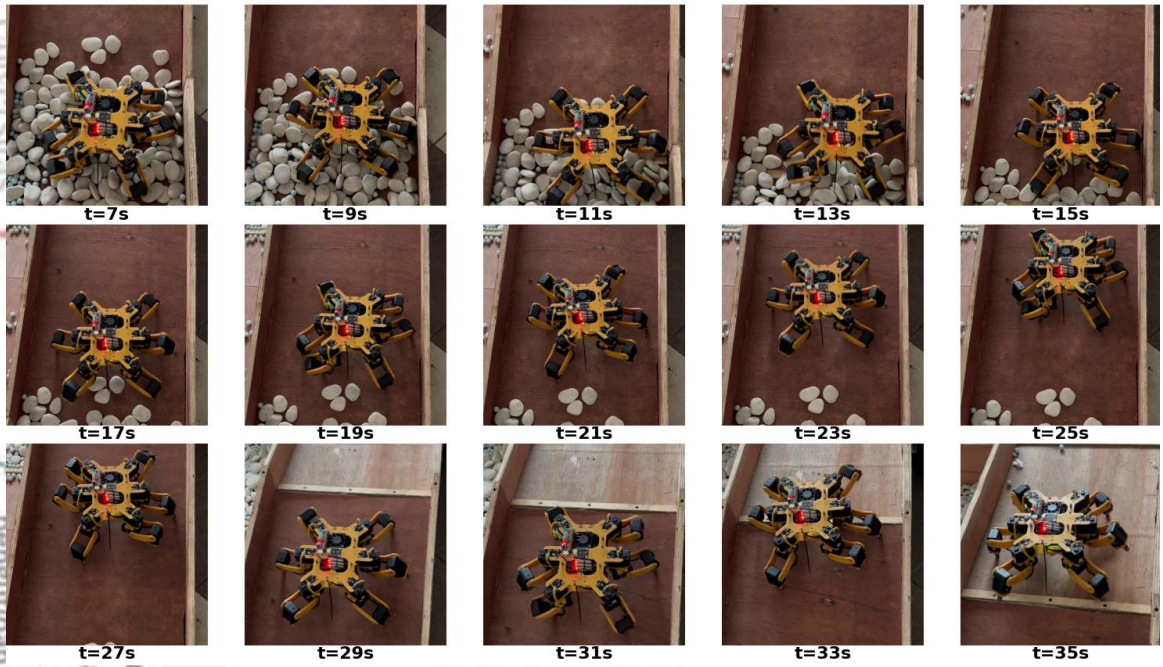


Gambar 4. 21 Hasil Pengujian Real Robot (State) pada Rintangan Climbing Slope

Berdasarkan grafik percobaan slope climbing, robot menunjukkan kemampuan adaptasi yang baik dalam menavigasi medan menanjak. Pada fase awal (step 0-5), robot mempertahankan gerakan maju yang stabil dengan nilai body X konsisten di sekitar -0.25 cm, sementara *state pitch* dan *roll* menunjukkan fluktuasi minimal yang disebabkan oleh jalan berbatu. Kemudian ketika mulai menanjak (step 5-22), robot secara progresif menyesuaikan posturnya dengan meningkatkan aksi pitch hingga -0.08 rad dan aksi body X hingga -0.5 cm, menunjukkan upaya aktif untuk mengatasi kemiringan. Nilai *state pitch* yang berosilasi antara -0.1 hingga -0.2 rad mencerminkan dinamika interaksi dengan medan, dengan puncak usaha terlihat pada step 15-20.

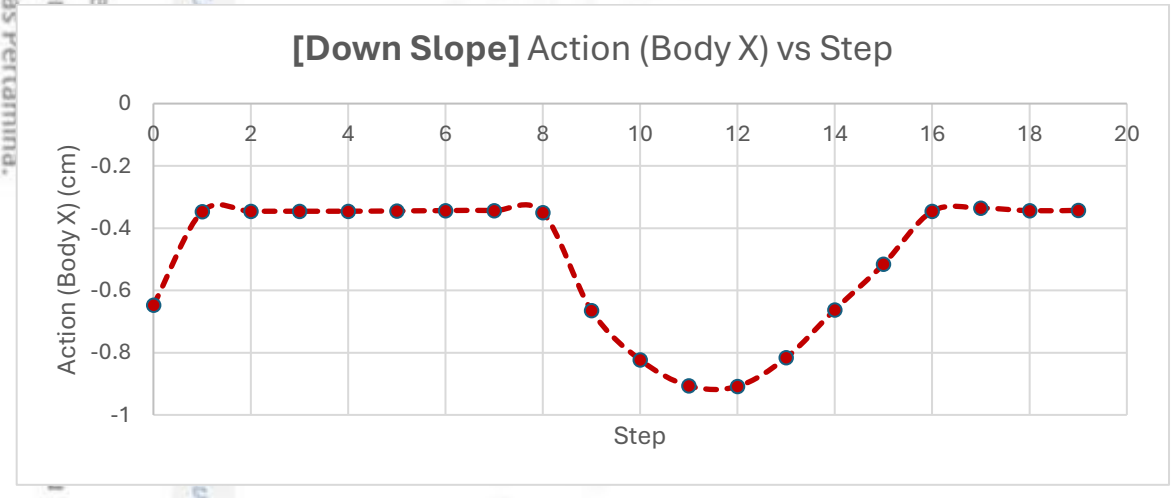
Setelah berhasil melewati bagian tersulit (step 22), robot dengan cepat menstabilkan diri, ditunjukkan oleh kembalinya nilai pitch mendekati equilibrium dalam 10 step terakhir. Namun, variasi aksi roll yang cukup besar (-0.1 hingga -0.5 rad) mengungkapkan sensitivitas berlebihan terhadap gangguan lateral, yang bisa menjadi fokus perbaikan. Secara keseluruhan, robot berhasil menyelesaikan tantangan slope climbing dalam 32 step, dengan kemampuan stabilisasi yang baik tetapi masih memiliki potensi untuk dioptimalkan, khususnya dalam mengurangi osilasi dan overshoot saat menghadapi kemiringan ekstrem.

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

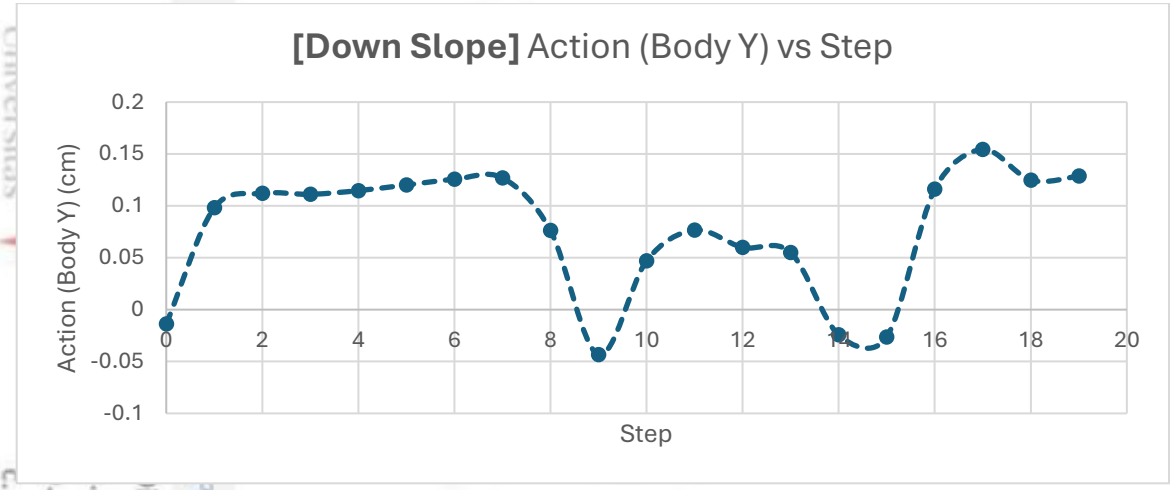


Gambar 4. 22 Dokumentasi Pengujian Real Robot pada Rintangan Climbing Slope

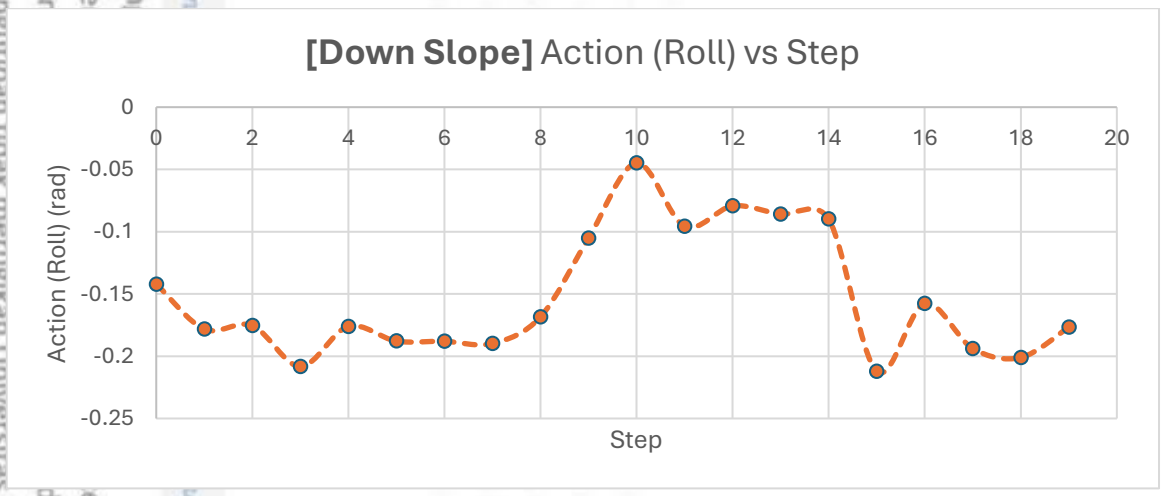
Sementara itu, pada medan lereng menurun, performa robot relatif stabil, meskipun terdapat beberapa tantangan yang muncul saat terjadi transisi medan. Dibandingkan dengan climbing slope pada pengujian sebelumnya, down slope terlihat lebih mudah dilewati oleh robot karena hanya membutuhkan 19 steps untuk dapat menyelesaikan rintangan. Berikut merupakan grafik yang diambil saat robot melakukan misi menuruni bidang miring (lereng).



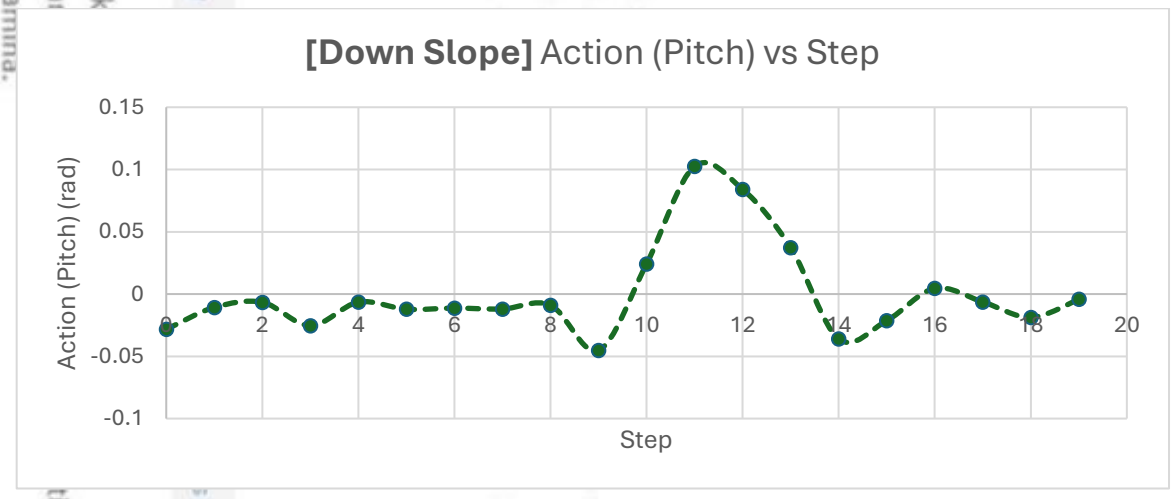
Gambar 4. 23 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope



Gambar 4. 24 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope

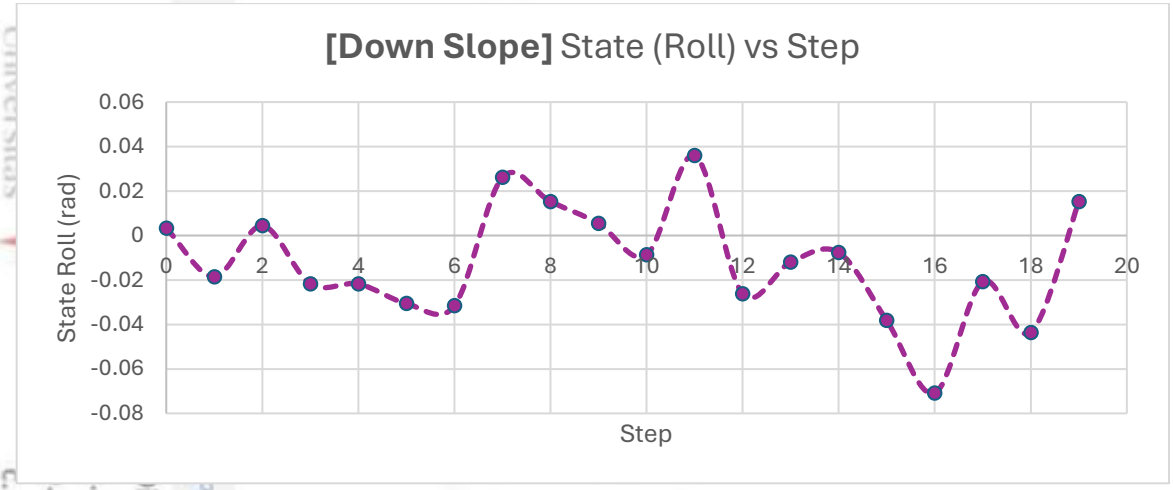


Gambar 4. 25 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope

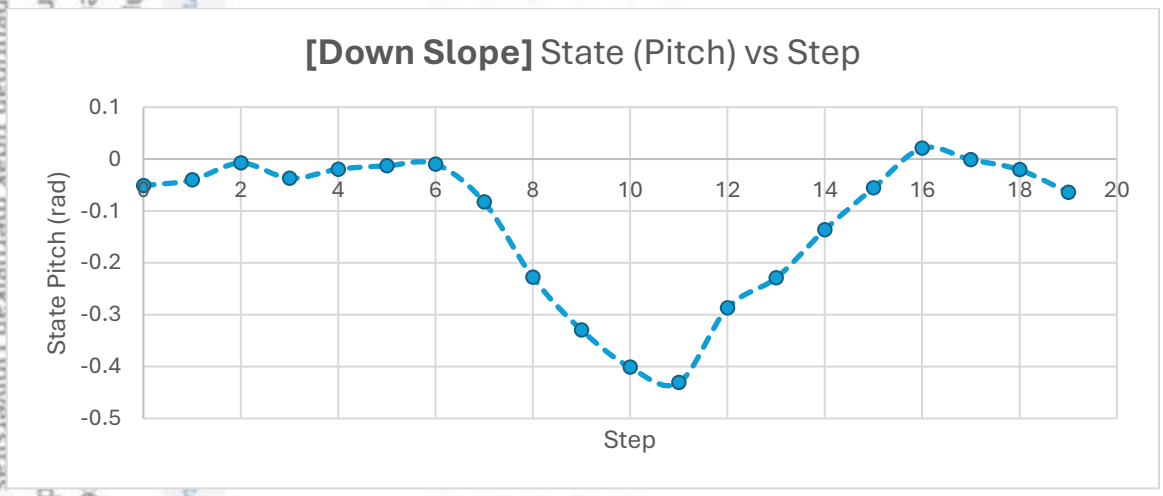


Gambar 4. 26 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 27 Hasil Pengujian Real Robot (Action) pada Rintangan Down Slope



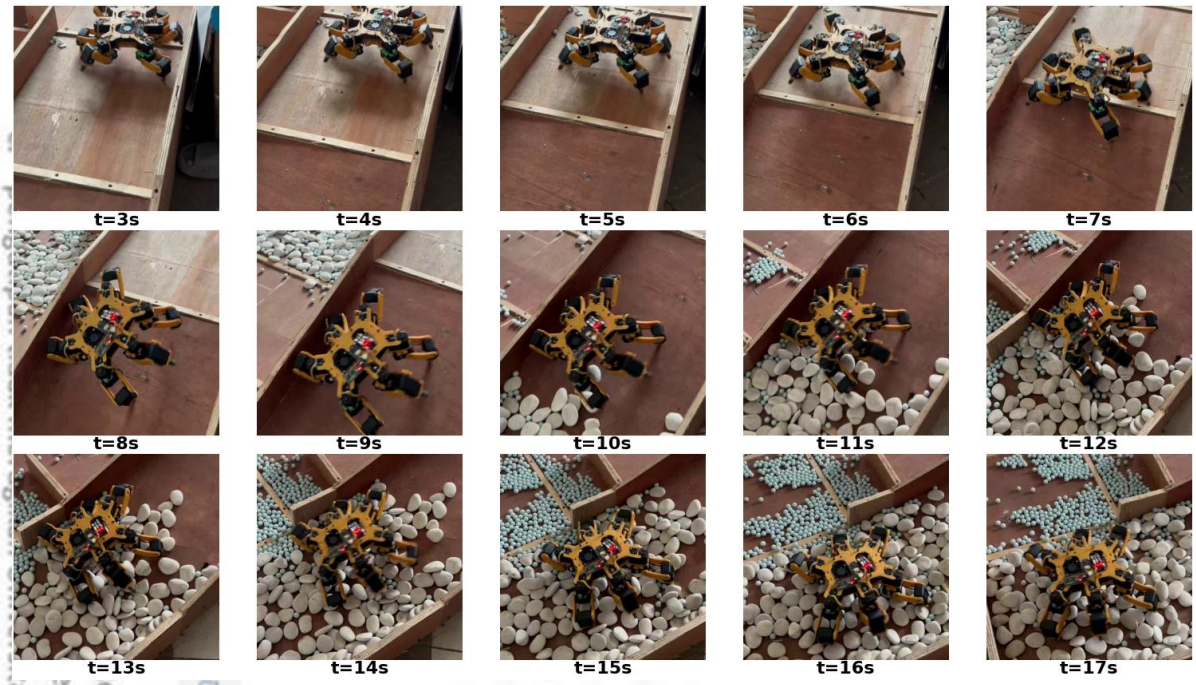
Gambar 4. 28 Hasil Pengujian Real Robot (State) pada Rintangan Down Slope

Berdasarkan analisis grafik down slope, robot menunjukkan kemampuan adaptasi yang baik dalam menavigasi medan menurun melalui serangkaian penyesuaian dinamis. Pada fase awal (step 1-5), robot secara proaktif melakukan pengereman dengan menurunkan nilai Body X hingga -0.6 cm sambil mempertahankan kemiringan pitch sekitar -0.1 rad untuk menjaga keseimbangan. Saat memasuki fase transisi (step 5-15), sistem kontrol menunjukkan respons aktif dengan mencapai nilai puncak Body X -0.8 cm di step 10 dan mengembangkan sudut pitch hingga -0.25 rad, menunjukkan upaya maksimal untuk mengendalikan kecepatan turun dan stabilitas. Yang menarik, meskipun aksi Roll menunjukkan variasi cukup besar (-0.25 hingga 0.1 rad), State Roll tetap stabil di kisaran ±0.04 rad, membuktikan efektivitas sistem dalam mengkompensasi gangguan lateral.

Pada fase akhir (step 15-20), semua parameter menunjukkan konvergensi yang baik menuju kondisi stabil, dengan Body X kembali ke -0.4 cm dan pitch menetap di -0.15 rad. Hasil ini mengindikasikan bahwa strategi kontrol saat ini sudah memadai namun masih

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

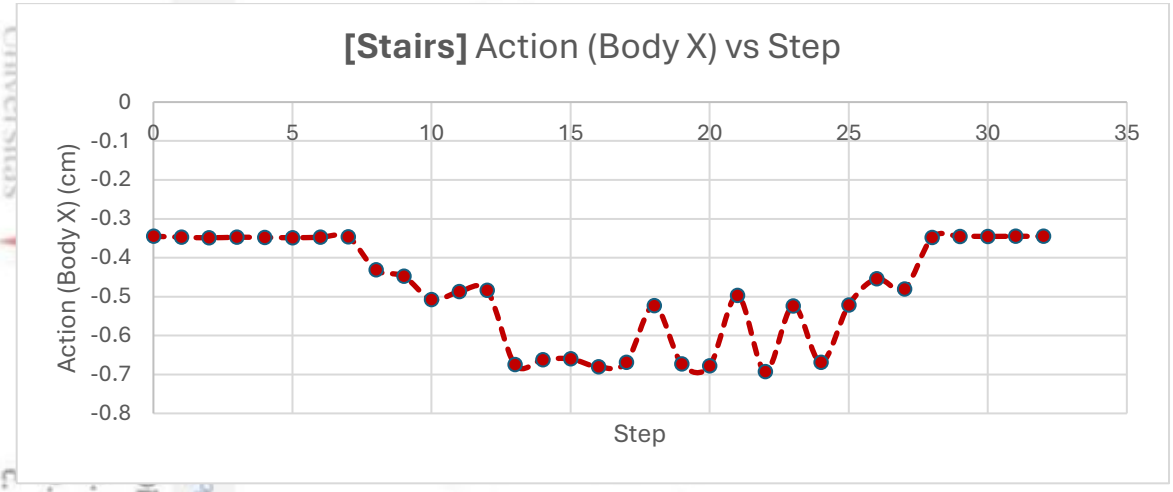
menyisakan ruang untuk optimasi, khususnya dalam mengurangi *overshoot* Body X dan variasi aksi Roll yang berlebihan. Untuk pengembangan selanjutnya, rekomendasi utama meliputi penyempurnaan algoritma kontrol kecepatan, kalibrasi gain controller yang lebih presisi, serta integrasi sensor inertial tambahan untuk prediksi kemiringan yang lebih akurat, yang secara bersama-sama diharapkan dapat meningkatkan efisiensi energi dan kelancaran gerakan dalam menghadapi medan menurun.



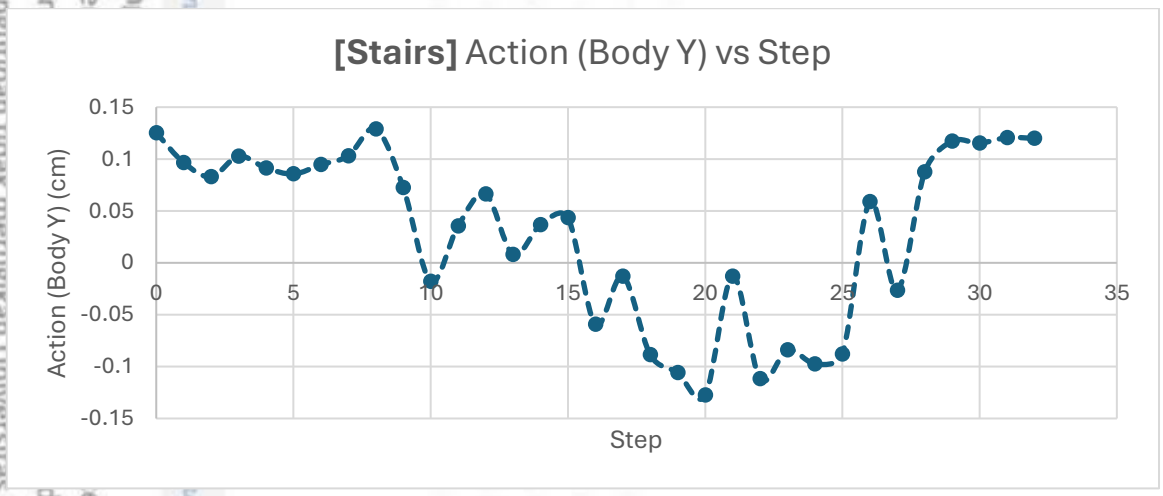
Gambar 4. 29 Dokumentasi Pengujian Real Robot pada Rintangan Down Slope

Pengujian selanjutnya yaitu mencoba pada medan bertangga menghasilkan performa yang paling menantang. Dapat dilihat pada Gambar 4.14 robot berhasil menaiki tangga dalam 32 steps. Terdapat beberapa steps yang mengalami fluktuasi yang mengindikasikan usaha robot dalam menyesuaikan diri terhadap perubahan ketinggian. Secara keseluruhan, medan tangga menjadi indikator utama bahwa algoritma RL yang digunakan masih memerlukan peningkatan, khususnya dalam hal perumusan fungsi *reward* yang dapat lebih menekankan pada selama pergerakan di medan bertingkat. Akan tetapi hasil yang sudah didapatkan dapat mencerminkan bahwa robot sudah dapat beradaptasi pada lingkungannya sesuaikan dengan objektif perancangan ini.

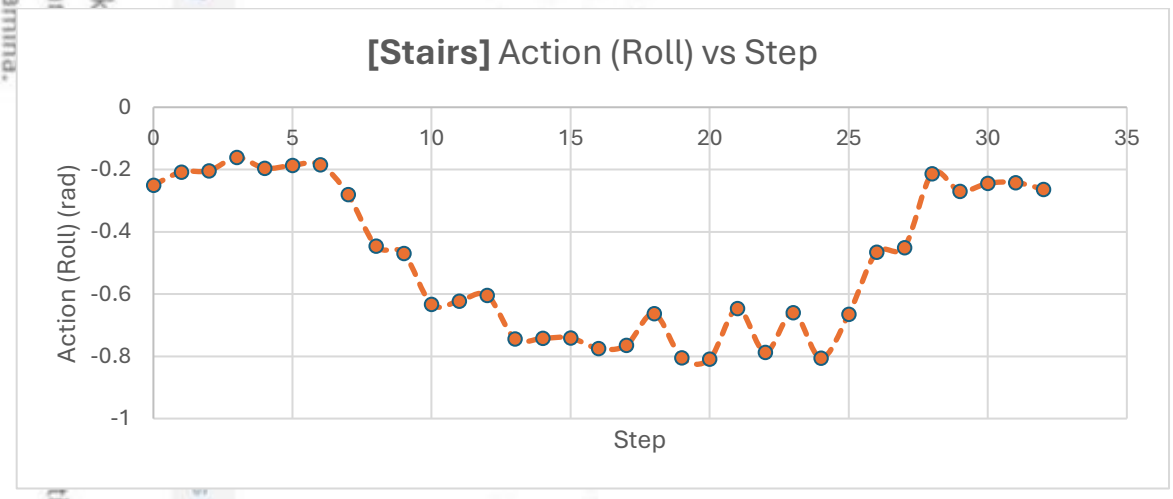
Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 30 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs

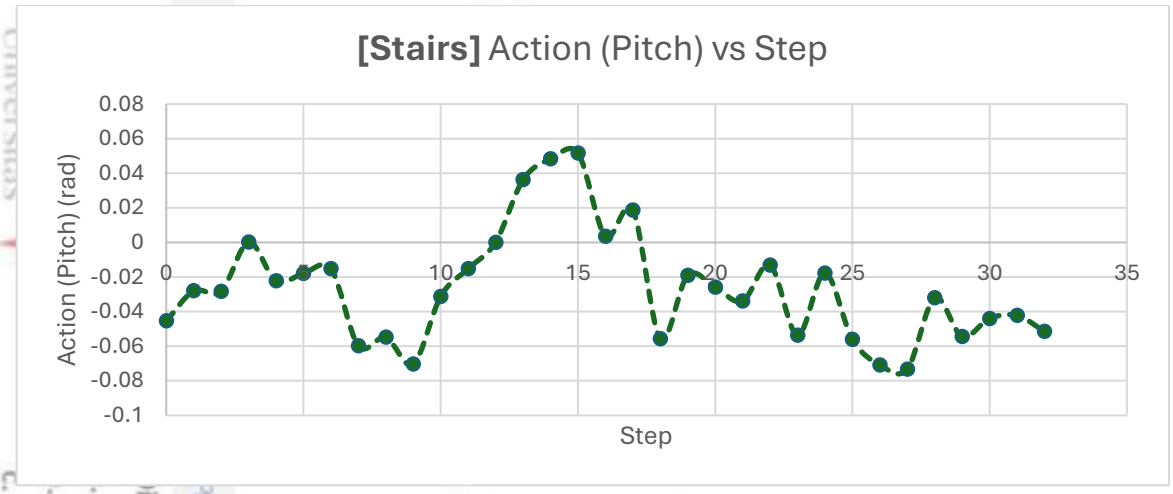


Gambar 4. 31 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs

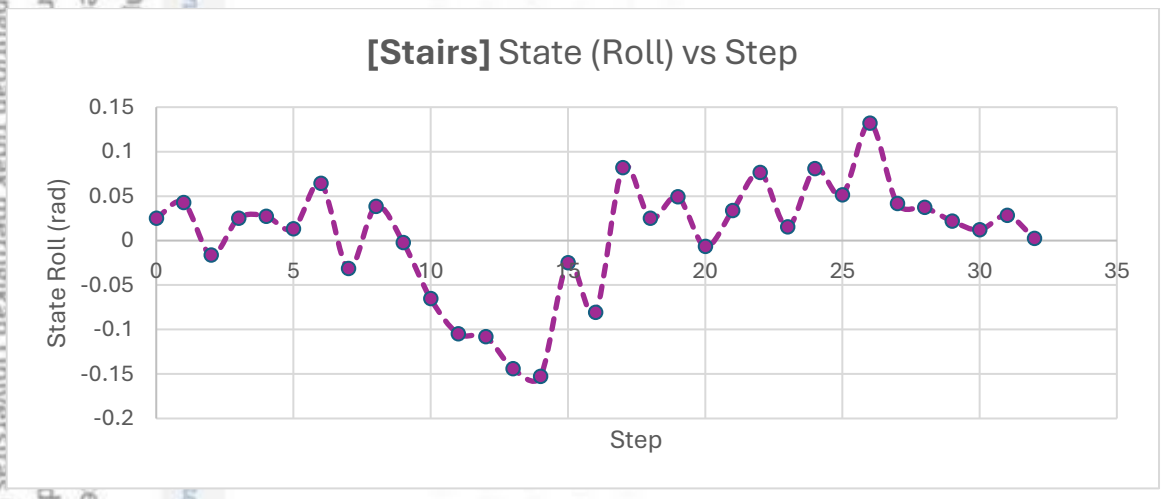


Gambar 4. 32 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs

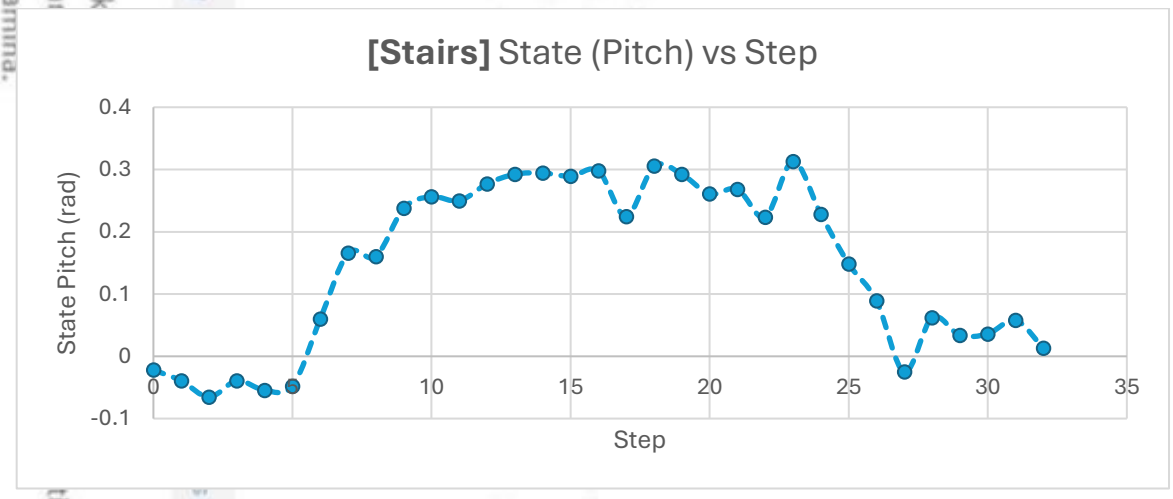
Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar 4. 33 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs



Gambar 4. 34 Hasil Pengujian Real Robot (Action) pada Rintangan Climbing Stairs

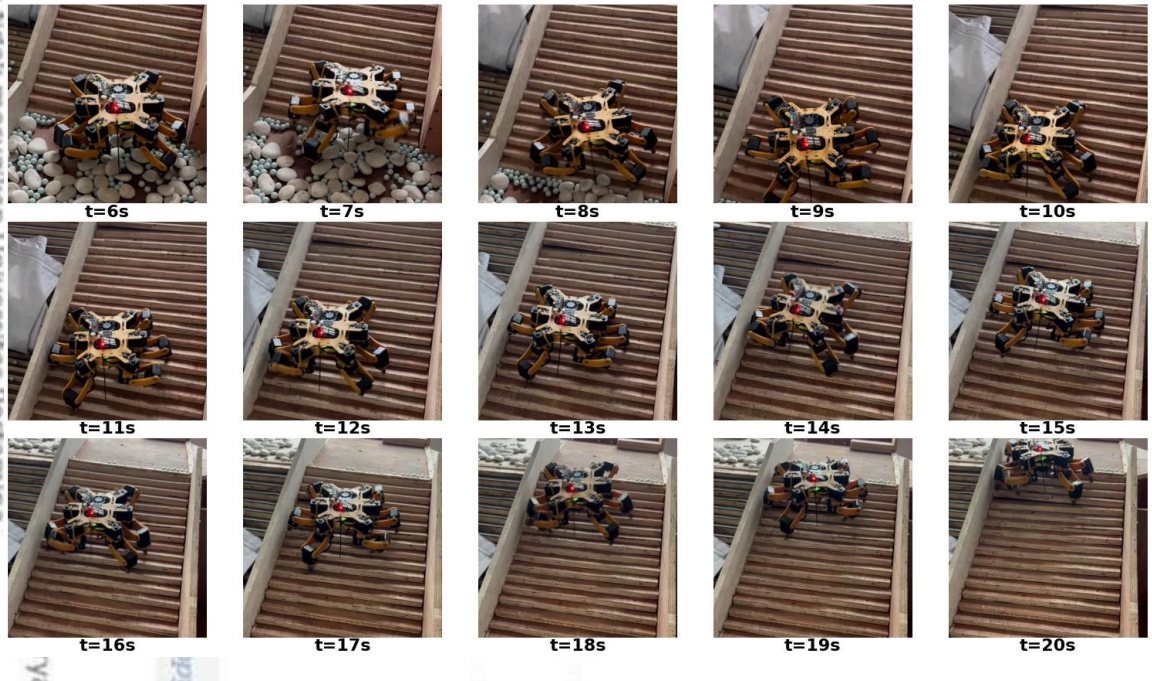


Gambar 4. 35 Hasil Pengujian Real Robot (State) pada Rintangan Climbing Stairs

Berdasarkan data percobaan, robot berhasil menaiki tangga dalam 32 langkah dengan pola gerakan yang menunjukkan adaptasi progresif terhadap medan bertingkat. Pada fase awal (step 0-10), robot secara konsisten meningkatkan daya dorong (Body X mencapai -0.8

cm) sambil mulai menyesuaikan kemiringan tubuh (State Pitch 0.2 rad). Fase transisi (step 10-25) menjadi momen kritis dimana robot menghadapi tantangan terberat, terlihat dari puncak State Pitch 0.4 rad di step 15 yang mengindikasikan usaha maksimal untuk mengatasi ketinggian tangga, sementara variasi Aksi Roll antara -1 hingga 0 rad menunjukkan upaya intensif menjaga keseimbangan lateral.

Meski berhasil menyelesaikan tugas, beberapa kelemahan teridentifikasi: (1) fluktuasi berlebihan pada State Pitch menunjukkan ketidakstabilan saat transisi antar anak tangga, (2) respons Roll yang terlalu agresif mengindikasikan kontrol lateral yang kurang halus, dan (3) pola gerakan yang belum sepenuhnya smooth antar step. Hasil ini membuktikan bahwa algoritma RL yang digunakan sudah mampu beradaptasi dengan lingkungan target, namun masih memerlukan penyempurnaan khususnya dalam hal formulasi fungsi reward yang lebih menekankan pada stabilitas gerakan vertikal dan efisiensi energi.



Gambar 4. 36 Dokumentasi Pengujian Real Robot pada Rintangan Climbing Stairs

Untuk pengembangan selanjutnya, disarankan tiga pendekatan utama: pertama, optimasi fungsi reward dengan penekanan pada smoothness gerakan dan penalti untuk osilasi berlebihan; kedua, penyempurnaan parameter kontrol melalui teknik auto-tuning; ketiga, implementasi arsitektur pembelajaran yang lebih canggih seperti SAC dengan memory buffer untuk meningkatkan konsistensi performa. Dengan berbagai penyempurnaan ini, robot diharapkan dapat menaklukkan medan tangga dengan lebih stabil dan efisien, sekaligus membuka kemungkinan untuk menghadapi medan tidak rata yang lebih kompleks di masa depan.

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1. Pengembangan metode *inverse kinematic*, *body kinematic*, dan *trajectory planning* telah berhasil diintegrasikan secara efektif ke dalam sistem robot hexapod. Kombinasi metode ini memungkinkan robot untuk menghasilkan gerakan yang lebih stabil, efisien, dan terkontrol dalam menghadapi berbagai kondisi medan.
2. Penerapan Reinforcement Learning (RL) berhasil meningkatkan kemampuan adaptif robot terhadap medan yang kompleks. Tiga algoritma RL diuji dalam simulasi PPO, TD3, dan SAC. Di antara ketiganya, SAC menunjukkan performa terbaik, dengan gerakan paling stabil, halus, dan efisien secara energi. Hal ini membuktikan bahwa RL dapat secara signifikan meningkatkan kemampuan kontrol robot secara adaptif dibanding pendekatan konvensional.
3. Uji coba performa robot di lingkungan nyata menunjukkan bahwa model RL yang telah dilatih mampu menghasilkan pola gerakan yang disesuaikan secara otomatis terhadap berbagai rintangan. Robot dapat mempertahankan keseimbangan dan menyelesaikan misi navigasi seperti menaiki slope dan tangga dengan sukses. Implementasi ini membuktikan bahwa pendekatan integratif RL-kinematika efektif untuk diterapkan dalam skenario dunia nyata seperti arena Kontes Robot SAR Indonesia (KRSRI) 2024.

5.2 Saran

Beberapa saran yang diajukan penulis untuk pengembangan lebih lanjut sehingga dapat dipertimbangkan adalah sebagai berikut:

1. Melakukan *hyperparameter tuning* lebih lanjut pada model SAC agar dapat tetap stabil dan tidak agresif saat melewati medan datar.
2. Penyempurnaan model RL sapat dilakukan dengan menambahkan variabel observation space seperti data LIDAR untuk deteksi rintangan lebih akurat.
3. Optimasi hardware dengan penggunaan material lebih ringan dan kuat (seperti serat karbon) untuk mengurangi beban servo. Selain itu juga dapat ditambahkan integrasi sensor (misalnya, force sensor pada kaki) untuk mendeteksi interaksi kaki dengan permukaan. Dengan penyempurnaan tersebut, diharapkan robot hexapod dapat menjadi solusi lebih matang untuk tantangan eksplorasi dan penyelamatan di masa depan.

Universitas Pertamina
Hak Cipta dilindungi Undang-Undang
1. Dilarang menyebarluaskan dan/atau mendistribusikan karya tulis ini, kecuali:
a. Pengutipan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah atau penyusunan laporan kerja profesional tidak merugikan kepentingan Universitas Pertamina;
b. Pengutipan tidak merugikan kepentingan Universitas Pertamina;
c. Pengutipan tidak merugikan kepentingan yang wajar bagi Universitas Pertamina.
2. Dilarang memublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

DAFTAR PUSTAKA

[1] T. Deepa, S. Angalaeswari, D. Subbulekshmi, S. Krithiga, S. Sujeeth, and R. Kathiravan, "Design and implementation of bio inspired hexapod for exploration applications," *Mater. Today Proc.*, vol. 37, no. Part 2, pp. 1603–1607, 2020, doi: 10.1016/j.matpr.2020.07.165.

[2] L. Zhang, D. Li, F. Yang, and C. Liu, "Development and attitude control of a Hexapod bionic-Robot," *2016 IEEE Int. Conf. Robot. Biomimetics, ROBIO 2016*, pp. 77–82, 2016, doi: 10.1109/ROBIO.2016.7866301.

[3] J. Coelho, F. Ribeiro, B. Dias, G. Lopes, and P. Flores, "Trends in the control of hexapod robots: A survey," *Robotics*, vol. 10, no. 3, pp. 1–22, 2021, doi: 10.3390/robotics10030100.

[4] W. Ouyang, H. Chi, J. Pang, W. Liang, and Q. Ren, "Adaptive Locomotion Control of a Hexapod Robot via Bio-Inspired Learning," *Front. Neurorobot.*, vol. 15, no. January, 2021, doi: 10.3389/fnbot.2021.627157.

[5] S. Muslimin and D. Istardi, "Inverse Kinematics Analysis for Motion Prediction of a Hexapod Robot," *Proc. 2018 Int. Conf. Appl. Eng. ICAE 2018*, pp. 1–5, 2018, doi: 10.1109/INCAE.2018.8579402.

[6] I. A. Kurniawan, Feriyonika, and S. Pramono, "Inverse dan Body Kinematics pada Robot Hexapod," *Ind. Res. Work. Natl. Semin.*, vol. 9, pp. 115–123, 2018.

[7] F. Tedeschi and G. Carbone, "Hexapod walking robot locomotion," *Mech. Mach. Sci.*, vol. 29, pp. 439–468, 2015, doi: 10.1007/978-3-319-14705-5_15.

[8] H. Xia, X. Zhang, and H. Zhang, "A new foot trajectory planning method for legged robots and its application in hexapod robots," *Appl. Sci.*, vol. 11, no. 19, 2021, doi: 10.3390/app11199217.

[9] S. Ji, W. Wei, X. Liu, and J. Wu, "Optimization Control of Attitude Stability for Hexapod Robots Based on Reinforcement Learning," *IEEE Access*, vol. 12, no. November, pp. 156363–156377, 2024, doi: 10.1109/ACCESS.2024.3485506.

[10] Q. Yang, Y. Gao, and S. Li, "Terrain-Adaptive Central Pattern Generators with Reinforcement Learning for Hexapod Locomotion," *Proc. - 2023 China Autom. Congr. CAC 2023*, pp. 1017–1022, 2023, doi: 10.1109/CAC59555.2023.10451414.

[11] D. Li, W. Wei, and Z. Qiu, "Combined Reinforcement Learning and CPG Algorithm to Generate Terrain-Adaptive Gait of Hexapod Robots," *Actuators*, vol. 12, no. 4, pp. 1–12, 2023, doi: 10.3390/act12040157.

Universitas Pertamina

2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian; pengutipan tidak merugikan Universitas Pertamina

[12] B. Kusumoputro *et al.*, "Pedoman Kontes Robot Indonesia (Kri) Pendidikan Tinggi Tahun 2024," *Kementeri. Pendidikan, Kebudayaan, Riset, dan Teknol.*, pp. 1–164, 2024.

[13] D. Toth and G. Parker, "Evolving Gaits for the Lynxmotion Hexapod II Robot," *Proc. 7th World Multiconference ...*, no. May, 2003, [Online]. Available: http://oak.conncoll.edu/parker/papers/SCI2003.pdf

[14] S. I. Beaber, A. S. Zaghloul, M. A. Kamel, and W. M. Hussein, "Dynamic modeling and control of the hexapod robot using Matlab simmechanics," *ASME Int. Mech. Eng. Congr. Expo. Proc.*, vol. 4A-2018, no. January 2019, 2018, doi: 10.1115/IMECE2018-88226.

[15] P. Cizek, M. Zoula, and J. Faigl, "Design, Construction, and Rough-Terrain Locomotion Control of Novel Hexapod Walking Robot with Four Degrees of Freedom per Leg," *IEEE Access*, vol. 9, pp. 17866–17881, 2021, doi: 10.1109/ACCESS.2021.3053492.

[16] M. Bjelonic, T. Homberger, N. Kottege, P. Borges, M. Chli, and P. Beckerle, "Autonomous navigation of hexapod robots with vision-based controller adaptation," *Proc. - IEEE Int. Conf. Robot. Autom.*, no. March, pp. 5561–5568, 2017, doi: 10.1109/ICRA.2017.7989655.

[17] ROBOTIS, "Dynamixel AX-12A Specifications," https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/. [Online]. Available: https://www.generationrobots.com/media/Dynamixel-AX-12-user-manual.pdf?srsltid=AfmBOooeUE-Nh8MgulclkxVjUMiZhieo89j_7NAWf2lW5kl6TwVN8xiU

[18] ROBOTIS, "Dynamixel AX-18F / AX-18A Specifications," https://emanual.robotis.com/docs/en/dxl/ax/ax-18a/. [Online]. Available: https://www.crustcrawler.com/motors/AX-18F/docs/AX-18A Actuator Technical Reference Guide.pdf

[19] S. Valladares, M. Toscano, R. Tufiño, P. Morillo, and D. Vallejo-Huanga, "Performance Evaluation of the Nvidia Jetson Nano Through a Real-Time Machine Learning Application," no. January, pp. 343–349, 2021, doi: 10.1007/978-3-030-68017-6_51.

[20] SLAMTEC, "RPLIDAR C1 Low Cost 360 Degree Laser Range Scanner Development Kit User Manual."

[21] S. D. Varanasi, M. Tammana, and R. K. Megalingam, "Robotic Navigation Unveiled :

Pertamina
Hak Cipta Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
c. penggunaan tidak merugikan Universitas Pertamina.
b. penggunaan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;

A Comprehensive Study of GMapping , Hector Slam , and Cartographer," *2024 3rd Int. Conf. Innov. Technol.*, no. November, pp. 1–6, 2024, doi: 10.1109/INOCON60754.2024.10511508.

[22] B. Sensortec, "Intelligent 9-axis absolute orientation sensor." [Online]. Available: https://cdn-shop.adafruit.com/datasheets/BST_BNO055_DS000_12.pdf

[23] E. Coumans and Y. Bai, "PyBullet Quickstart Guide," *PyBullet*, vol. 1, no. 1, pp. 1–84, 2023.

[24] J. Panerati, H. Zheng, S. Q. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to Fly-A Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control," *IEEE Int. Conf. Intell. Robot. Syst.*, pp. 7512–7519, 2021, doi: 10.1109/IROS51168.2021.9635857.

[25] M. Quigley *et al.*, "ROS: an open-source Robot Operating System." [Online]. Available: http://stair.stanford.edu

[26] G. Nam, "Bio-Inspired Control of Hexapod for Navigating Confined Spaces in Disaster Recovery," no. February, 2022, doi: 10.13140/RG.2.2.18529.58729.

[27] H. Yu, W. Guo, J. Deng, M. Li, and H. Cai, "A CPG-based locomotion control architecture for hexapod robot," *IEEE Int. Conf. Intell. Robot. Syst.*, pp. 5615–5621, 2013, doi: 10.1109/IROS.2013.6697170.

[28] M. Luneckas *et al.*, "Hexapod robot gait switching for energy consumption and cost of transport management using heuristic algorithms," *Appl. Sci.*, vol. 11, no. 3, pp. 1–13, 2021, doi: 10.3390/app11031339.

[29] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement Learning : A Survey," pp. 237–285, 1996.

[30] Y. Li, B. We, D. Q-network, and D. R. Learning, "DEEP REINFORCEMENT LEARNING: AN OVERVIEW," pp. 1–85, 2018.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," pp. 1–12, 2017, [Online]. Available: http://arxiv.org/abs/1707.06347

[32] H. Lee, M. K. Dahouda, and I. Joe, "Character Behavior Automation Using Deep Reinforcement Learning," *IEEE Access*, vol. 11, pp. 101435–101442, 2023, doi: 10.1109/ACCESS.2023.3313737.

[33] S. Fujimoto, H. Van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," *35th Int. Conf. Mach. Learn. ICML 2018*, vol. 4, pp. 2587–2601, 2018.

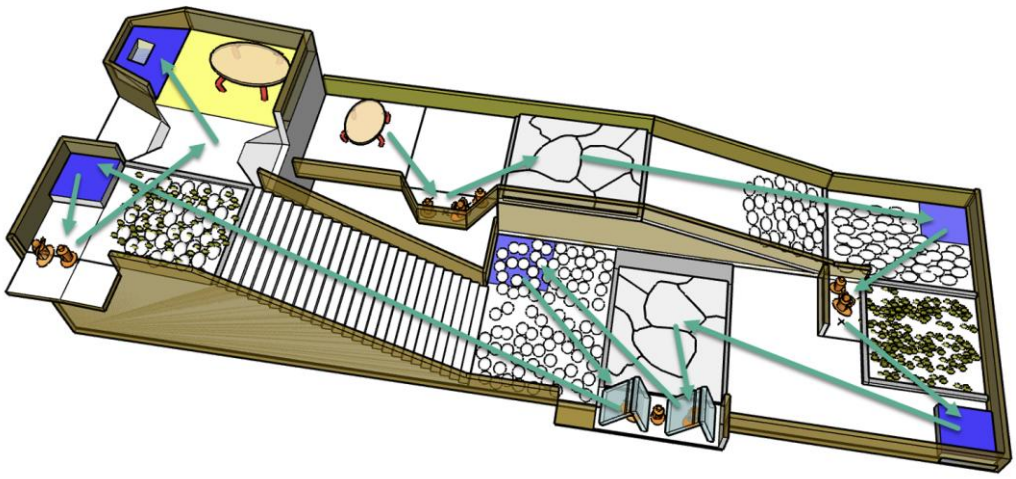
Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.


Universitas
Pertamina

Lampiran 1. Lapangan Kontes Robot SAR Indonesia (KRSRI) 2024



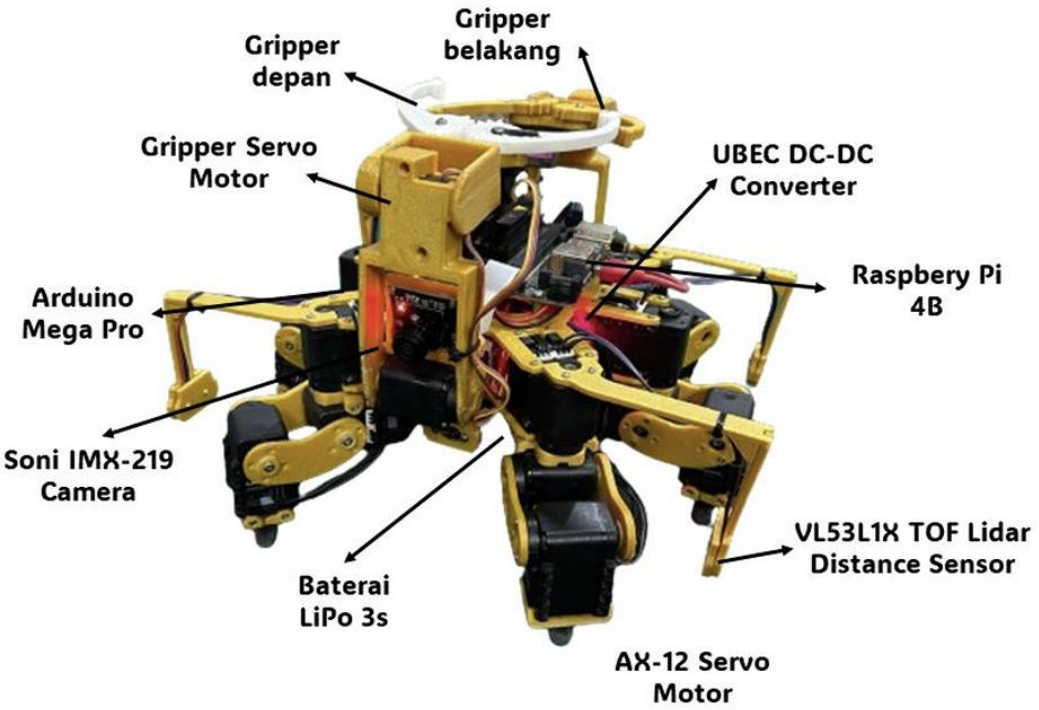
Gambar Lampiran 1.1 Lapangan Kontes Robot SAR Indonesia (KRSRI) 2024



Gambar Lampiran 1.2 Lapangan Kontes Robot SAR Indonesia (KRSRI) 2024 (Sketchup)

Universitas Pertamina
Hak Cipta Dilindungi Undang-Undang
1. Dilarang mengutip karya tulis ini, kecuali:
a. menyebutkan sumber sesuai kaidah kecend
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian,
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Lampiran 2. Robot Hexapod Bumblebee



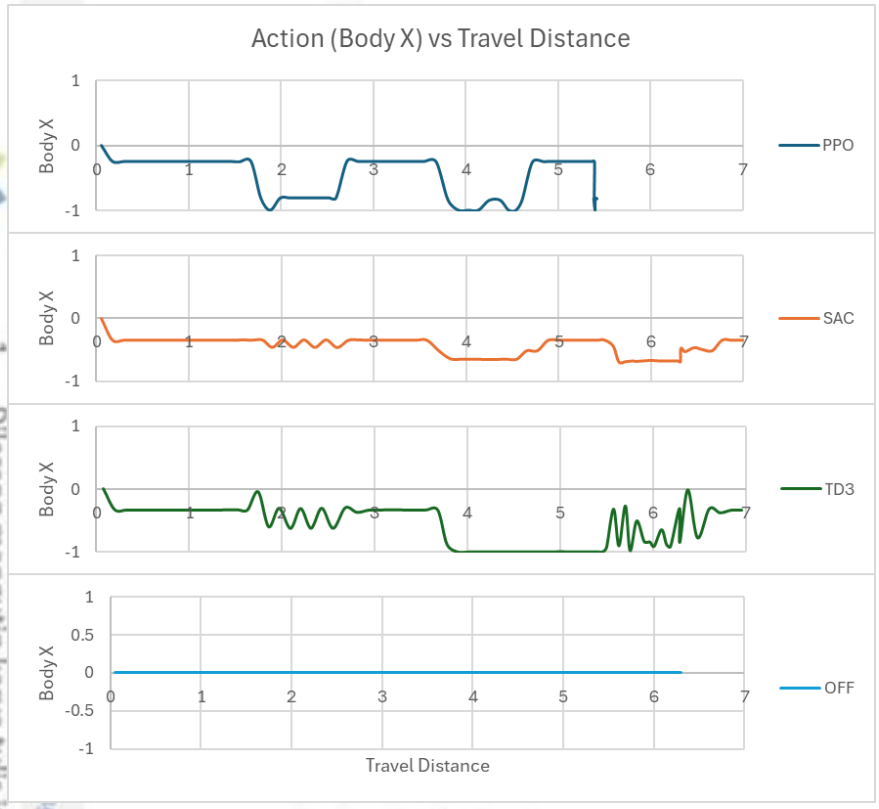
Gambar Lampiran 2.1 Robot Hexapod dan Komponen Robot



Gambar Lampiran 2.2 Robot Hexapod Tampak Segala Sisi

Universitas Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Lampiran 3. Hasil Testing Model *Reinforcement Learning*

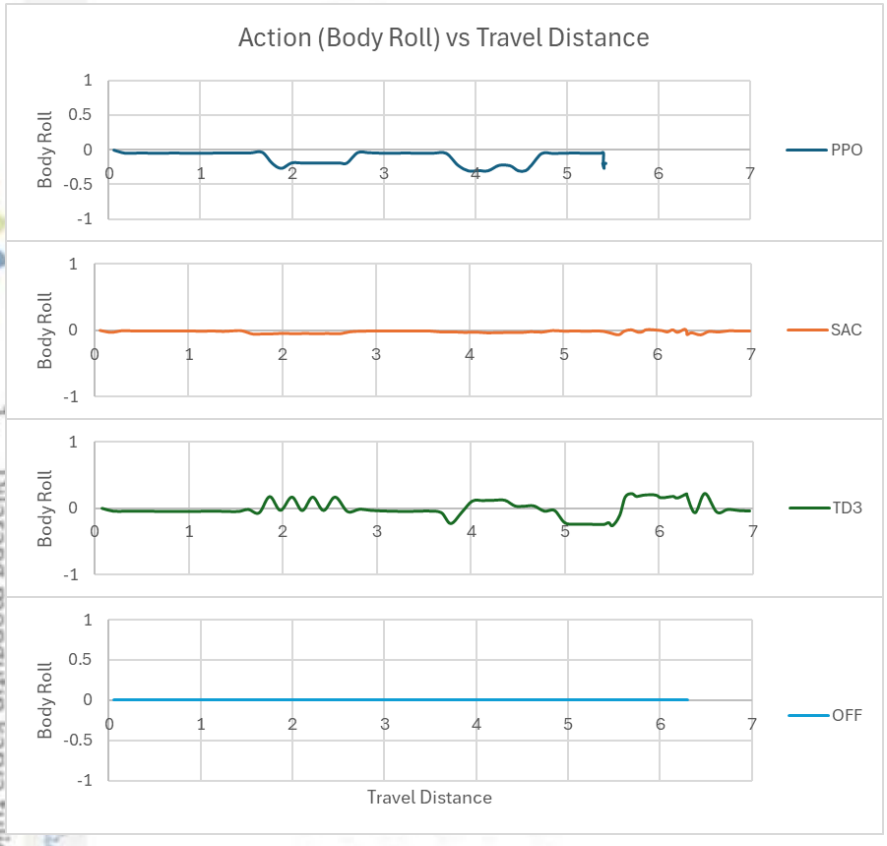


Gambar Lampiran 3.1 Action (Body X) vs Travel Distance

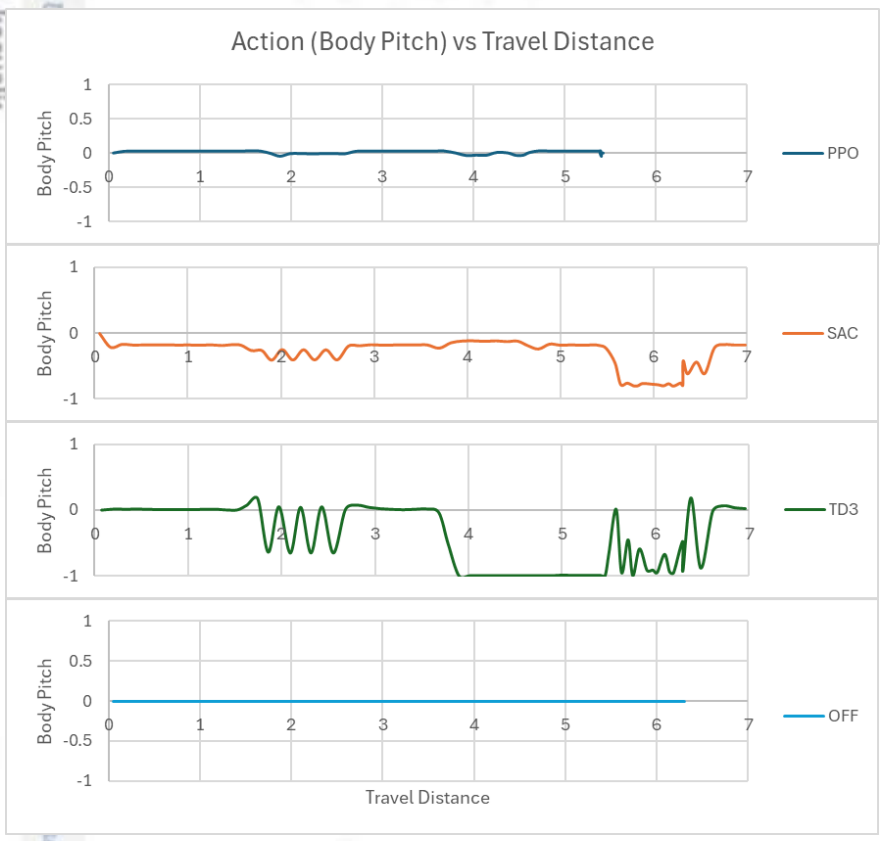


Gambar Lampiran 3.2 Action (Body Y) vs Travel Distance

Universitas Pertamina
1. Dilarang mengutip karya tulis ini, kecuali:
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



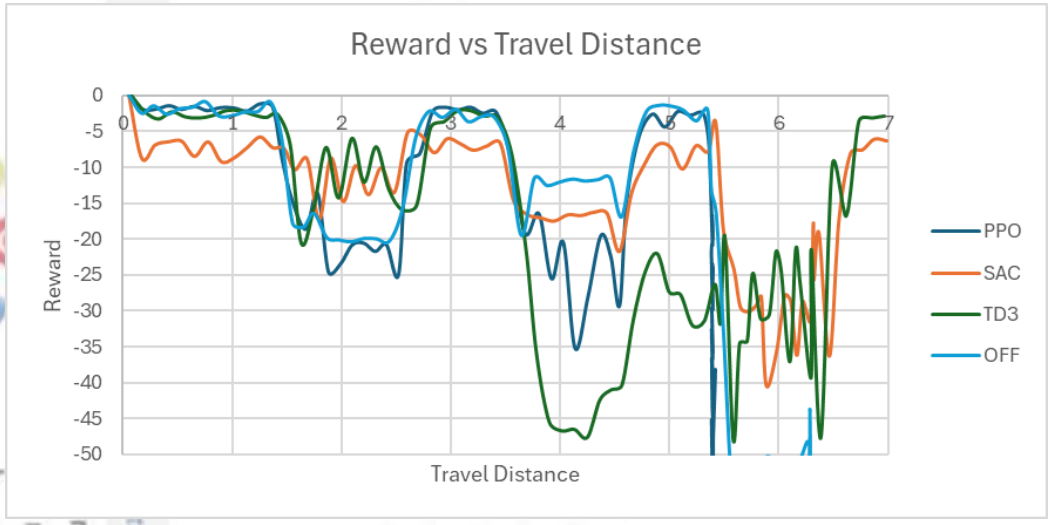
Gambar Lampiran 3.3 Action (Body Roll) vs Travel Distance



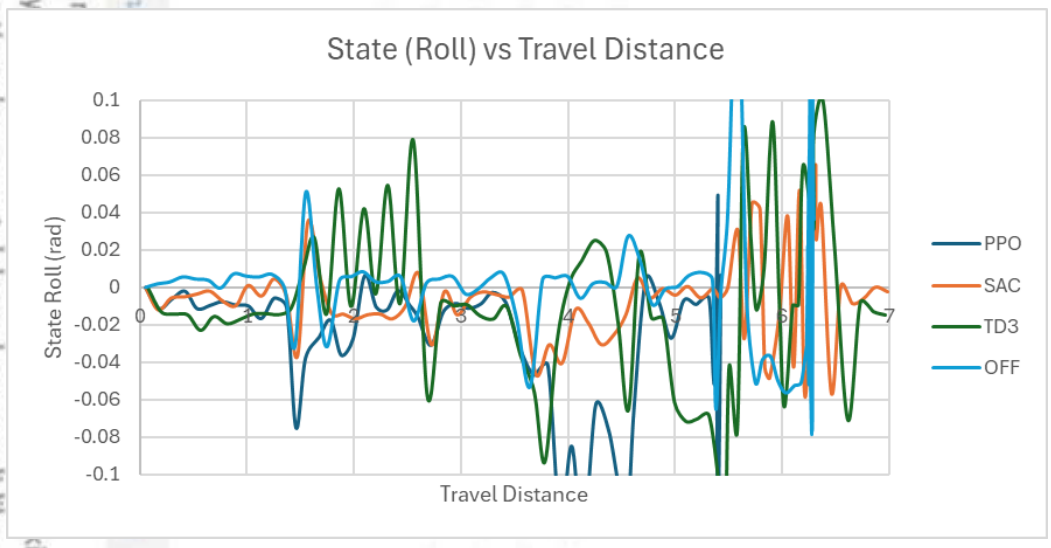
Gambar Lampiran 3.4 Action (Body Pitch) vs Travel Distance

Universitas Pertamina

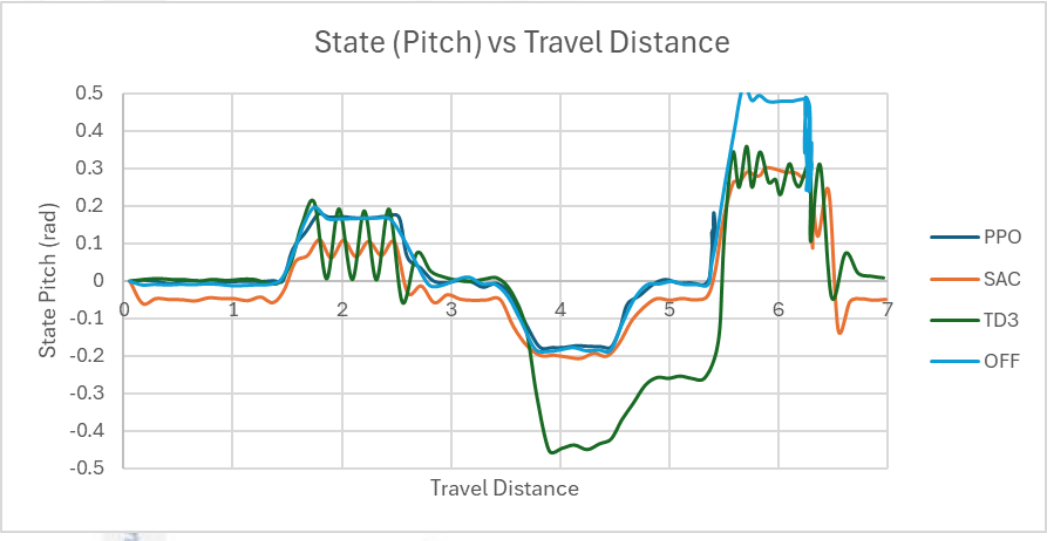
1. Dilarang mengutip karya tulis ini, kecuali:
   a. menyebutkan sumber sesuai kaidah kecendekiaan;
   b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
   c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.



Gambar Lampiran 3.5 Reward (Total) vs Travel Distance



Gambar Lampiran 3.6 State (Roll) vs Travel Distance



Gambar Lampiran 3.7 State (Pitch) vs Travel Distance

Universitas Pertamina
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Lampiran 4. Program Environment Reinforcement Learning

```python
"""
Hexapod Robot Simulation Environment for Reinforcement Learning

This module implements a PyBullet-based simulation environment for training a hexapod robot
using reinforcement learning. The environment includes physics simulation, robot control,
and reward calculation for stable locomotion.

Author: Muhammad Harish
Institution: Universitas Pertamina
Date: 23 June 2025
"""

import gym
from gym import spaces
import pybullet as p
import pybullet_data
import time
import math
import numpy as np
import threading
from lib.hexapod_constant import *
from hexapod_control import *
from terrain import add_terrain, add_field

# Simulation Constants
HOME_POSITION = [1, 0, 0.5]
TARGET_POSITION = [-4, 0, 0.1]  # Target position for navigation

# Camera Configuration
CAMERA_CONFIG = {
    'width': 512,
    'height': 512,
    'fov': 90,
    'aspect': 1.0,
    'near': 0.02,
    'far': 15,
    'offset': [0, 0, 0.5]
}

class HexapodEnv(gym.Env):
    """Custom Gym environment for hexapod robot simulation."""

    def __init__(self, is_train=False):
        """
        Initialize the hexapod environment.

        Args:
            is_train (bool): Whether the environment is for training (disables rendering)
        """
        self.train = is_train
        self.progress = 0
        self.progress_old = 0
        self.stuck_counter = 0
        self.land_clear = True
        self.start_time = time.time()
```

Pertamina University
Hak Cipta Universitas Pertamina
1. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
2. Penggunaan untuk kepentingan pendidikan, penulisan karya ilmiah atau penelitian;

```
        self.time_elapsed = 0
        self.global_position = np.array([0, 0, 0])
        self.global_orientation = np.array([0, 0, 0, 0])

        # Gait parameters
        self.h_step = 0.05  # Step height
        self.d_step = 0.5   # Step duration
        self.gait = 2       # Gait pattern

        # Initialize visualization thread if not in training mode
        if not self.train:
            robot_camera_thread = threading.Thread(
                target=self._robot_camera_thread,
                daemon=True
            )
            robot_camera_thread.start()

        # Initialize parent class
        super(HexapodEnv, self).__init__()

        # Set up PyBullet simulation
        self._setup_simulation()

        # Define action and observation spaces
        self.action_space = spaces.Box(
            low=np.array([-1, -1, -20, -20]),
            high=np.array([1, 1, 20, 20]),
            dtype=np.float32
        )

        self.observation_space = spaces.Box(
            low=np.array([-np.inf, -np.inf]),
            high=np.array([np.inf, np.inf]),
            dtype=np.float32
        )

    def _setup_simulation(self):
        """Initialize PyBullet simulation environment."""
        self.client = p.connect(p.DIRECT if self.train else p.GUI)

        if self.train:
            # Disable visualization during training
            p.configureDebugVisualizer(p.COV_ENABLE_RENDERING, 0)
            p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)

        # Load physics engine settings
        p.setAdditionalSearchPath(pybullet_data.getDataPath())
        p.setGravity(0, 0, -9.8)
        p.setRealTimeSimulation(1)

        # Load environment
        self._load_environment()
        self.robot = self._load_robot()

    def _load_environment(self):
        """Load the simulation environment including terrain."""
        ground = p.loadURDF("plane.urdf")
```

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
1. Dilarang mengunggah karya tulis ini, kecuali

```
        terrain = add_field()
        p.changeDynamics(ground, -1, lateralFriction=5)

    def _load_robot(self):
        """Load the hexapod robot model."""
        model_path = "models/hexapod.urdf"
        robot_id = p.loadURDF(
            model_path,
            basePosition=HOME_POSITION,
            baseOrientation=p.getQuaternionFromEuler([0, 0, 0])
        )
        return robot_id

    def _robot_camera_thread(self):
        """Thread function for robot's point-of-view camera."""
        while True:
            t_start = time.time()

            # Get camera parameters from config
            width = CAMERA_CONFIG['width'] // 2
            height = CAMERA_CONFIG['height'] // 2
            fov = CAMERA_CONFIG['fov']
            aspect = CAMERA_CONFIG['aspect']
            near = CAMERA_CONFIG['near']
            far = CAMERA_CONFIG['far']
            offset = CAMERA_CONFIG['offset']

            # Get robot's current position and orientation
            pos, orientation = self.global_position, self.global_orientation
            orientation_matrix = np.array(
                p.getMatrixFromQuaternion(orientation)
            ).reshape(3, 3)

            # Adjust camera position
            x, y, z = pos
            pos = x-0.2, y, z-0.4

            # Calculate camera position and target
            camera_pos = np.array(pos) + np.dot(orientation_matrix, offset)
            camera_target = np.array(pos) + np.dot(
                orientation_matrix,
                [-5, 0, -1]  # Forward offset
            )

            # Compute view matrices
            view_matrix = p.computeViewMatrix(
                camera_pos.tolist(),
                camera_target.tolist(),
                [0, 0, 1]
            )
            projection_matrix = p.computeProjectionMatrixFOV(
                fov, aspect, near, far
            )

            # Render camera image
            p.getCameraImage(
                width, height,
```

Pertamina
1. Dilarang mengutip karya tulis ini, kecuali:
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

```
            view_matrix,
            projection_matrix,
            shadow=True,
            renderer=p.ER_BULLET_HARDWARE_OPENGL
        )

        # Control update rate
        time.sleep(1/240)  # 240 Hz update rate

    def reset(self):
        """Reset the simulation environment."""
        print("Robot RESET...")
        p.resetSimulation()
        self._setup_simulation()
        self.stuck_counter = 0
        self.start_time = time.time()
        return np.array(self._get_state(), dtype=np.float32)

    def _get_state(self):
        """Get current state of the robot."""
        pos, orn = p.getBasePositionAndOrientation(self.robot)
        euler = p.getEulerFromQuaternion(orn)
        roll, pitch, _ = euler

        # Update global position and orientation
        self.global_position = np.array(pos)
        self.global_orientation = np.array(orn)

        return [roll, pitch]

    def step(self, action):
        """
        Execute one time step in the environment.

        Args:
            action: The action to be executed

        Returns:
            tuple: (observation, reward, done, info)
        """
        # Apply action to the robot
        while True:
            scaled_action = action  # Scale action if needed
            if self._apply_action(scaled_action):
                break

        # Get new state
        state = self._get_state()
        roll, pitch = state

        # Calculate reward and check termination conditions
        reward = self._calculate_reward(state)
        done, info = self._check_termination(state)

        return np.array(state, dtype=np.float32), reward, done, info

    def _apply_action(self, action):
```

Pertamina
Hak Cipta milik Universitas Pertamina
Dilindungi Undang-Undang
Copyright of Universitas Pertamina
1. Dilarang menggunakan karya tulis ini, kecuali untuk:
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

```
        """
        Apply control action to the robot.

        Args:
            action: The control action to apply

        Returns:
            bool: True when action cycle is complete
        """
        t = time.time() - self.start_time

        # Get current robot state
        pos, orn = p.getBasePositionAndOrientation(self.robot)
        euler = p.getEulerFromQuaternion(orn)
        x_pos, y_pos, z_pos = pos
        yaw_pos = euler[2]

        # Calculate heading alignment
        target_angle = math.atan2(
            TARGET_POSITION[1] - y_pos,
            TARGET_POSITION[0] - x_pos
        )
        angle_diff = (target_angle - yaw_pos) % (2 * math.pi) - math.pi
        vz = angle_diff * 0.1  # Angular velocity for alignment

        # Check if action cycle is complete
        if self.progress_old > self.progress:
            self.progress_old = self.progress
            return True

        # Parse action components
        x, y, roll, pitch = action
        z = yaw = 0  # Fixed parameters
        vx, vy = [0.07, 0]  # Base velocities

        # Generate body movement and leg positions
        body_movement = [vx, vy, vz]
        leg_pos_body = body_kinematics([x,y,z], [roll, pitch, yaw])

        # Update progress tracking
        self.progress_old = self.progress

        # Control each leg
        for leg_index in range(6):
            # Compute trajectory
            pos, progress = generate_movement(
                t, self.gait, self.d_step, self.h_step,
                body_movement, leg_index
            )

            if leg_index == 0:
                self.progress = progress

            # Calculate target leg position
            leg_base = leg_pos_body[leg_index]
            leg_pos = (
                leg_base[0] + pos[0],
```

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
Hak Cipta milik Universitas Pertamina
© Copyright Universitas Pertamina

```
            leg_base[1] + pos[1],
            leg_base[2] + pos[2]
        )

        # Compute inverse kinematics
        joint_index = leg_index * 3 + 1
        if joint_index >= 10:  # Mirror for left side legs
            leg_pos = (-leg_pos[0], -leg_pos[1], leg_pos[2])

        coxa_angle, femur_angle, tibia_angle = inverse_kinematics(*leg_pos)

        # Set joint positions
        p.setJointMotorControl2(
            self.robot, joint_index,
            p.POSITION_CONTROL,
            targetPosition=coxa_angle
        )
        p.setJointMotorControl2(
            self.robot, joint_index + 1,
            p.POSITION_CONTROL,
            targetPosition=femur_angle
        )
        p.setJointMotorControl2(
            self.robot, joint_index + 2,
            p.POSITION_CONTROL,
            targetPosition=tibia_angle
        )

        # Step simulation
        p.stepSimulation()
        if not self.train:
            time.sleep(1 / 240)  # Real-time rendering

    def _check_termination(self, state):
        """
        Check termination conditions for the episode.

        Args:
            state: Current state of the robot

        Returns:
            tuple: (done, info) where done indicates episode termination
        """
        roll, pitch = state
        pos, _ = p.getBasePositionAndOrientation(self.robot)
        distance = math.sqrt(
            (TARGET_POSITION[0] - pos[0])**2 +
            (TARGET_POSITION[1] - pos[1])**2
        )

        # Check movement stagnation
        if not hasattr(self, "prev_position"):
            self.prev_position = pos[:2]

        current_position = pos[:2]
        distance_moved = math.dist(self.prev_position, current_position)

```

Pertamina
Hak Cipta milik Universitas Pertamina
Dilindungi Undang-Undang
1. Dilarang mengutip karya tulis ini, kecuali untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah, penyusunan laporan, penulisan kritik atau tinjauan suatu masalah;
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

```
        if distance_moved < 0.01:  # Minimal movement
            self.stuck_counter += 1
        elif distance_moved > 0.05:
            if self.land_clear:
                self.stuck_counter = max(0, self.stuck_counter - 1)

        self.prev_position = current_position

        # Check body contact with ground
        contacts = p.getContactPoints(bodyA=self.robot)
        body_touching_ground = any(c[3] == 0 for c in contacts)  # Link 0 is body

        # Check termination conditions
        done = False
        termination_reason = ""

        if abs(math.degrees(roll)) > 90 or abs(math.degrees(pitch)) > 90:
            termination_reason = f"Robot flipped (roll: {math.degrees(roll):.1f}°"
            done = True
        elif self.time_elapsed < -120:
            termination_reason = "Timeout (exceeded 120 seconds)"
            done = True
        elif self.stuck_counter > 20:
            termination_reason = f"Stuck (counter: {self.stuck_counter})"
            done = True
        elif body_touching_ground:
            termination_reason = "Robot body touching ground"
            done = True
        elif distance < 0.1:  # Success condition
            termination_reason = "Target reached!"
            done = True

        if done and termination_reason:
            print(f"Episode terminated: {termination_reason}")

        return done, {'termination_reason': termination_reason}

    def _calculate_energy_consumption(self):
        """Calculate current energy consumption of all joints."""
        total_power = 0.0
        for joint_index in range(p.getNumJoints(self.robot)):
            joint_state = p.getJointState(self.robot, joint_index)
            torque = joint_state[3]  # Applied torque
            velocity = joint_state[1]  # Joint velocity
            total_power += abs(torque * velocity)  # Instantaneous power

        # Approximate energy consumption
        time_step = 1 / 240  # Simulation timestep
        return total_power * time_step

    def _calculate_reward(self, state):
        """
        Calculate reward based on current state.

        Args:
            state: Current state of the robot
```

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;

```
        Returns:
            float: Calculated reward value
        """
        roll, pitch = state
        pos, _ = p.getBasePositionAndOrientation(self.robot)

        # Calculate distance to target (squared for stronger gradient)
        distance = math.sqrt(
            (TARGET_POSITION[0] - pos[0])**2 +
            (TARGET_POSITION[1] - pos[1])**2
        )**2

        # Stability penalty (based on orientation)
        stability_penalty = -(abs(math.degrees(roll)) + abs(math.degrees(pitch)))
        stability_penalty = np.clip(stability_penalty, -50, 50)

        # Energy consumption penalty
        energy_penalty = -self._calculate_energy_consumption()
        energy_penalty = np.clip(energy_penalty, -50, 50)

        # Time penalty
        self.time_elapsed = -(time.time() - self.start_time)

        # Distance reward components
        distance_reward = 0
        if distance < 0.1:
            distance_reward = 100  # Large reward for reaching target
        elif distance < 0.5:
            distance_reward = 20
        elif distance < 1.0:
            distance_reward = 10
        elif distance < 1.5:
            distance_reward = 5

        # Stagnation penalty
        stagnation_penalty = -self.stuck_counter

        # Adaptive gait control based on terrain
        if abs(math.degrees(roll)) < 5 and abs(math.degrees(pitch)) < 5:
            self.land_clear = True
            if self.stuck_counter <= 1:
                # Fast gait for flat terrain
                self.d_step = 0.3
                self.h_step = 0.03
                self.gait = 2
        elif ((5 <= abs(math.degrees(roll)) < 10) or \
            ((5 <= abs(math.degrees(pitch)) < 10)) and \
            (self.stuck_counter <= 5):
            # Stable gait for moderate terrain
            self.land_clear = False
            self.d_step = 0.5
            self.h_step = 0.05
            self.gait = 3
        else:
            # Slow, stable gait for difficult terrain
            self.land_clear = False
            self.d_step = 0.8
```

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;

```
            self.h_step = 0.08
            self.gait = 2

        # Weighted reward components
        reward = (
            0.2 * stability_penalty +
            0.1 * energy_penalty +
            1.0 * distance_reward +
            1.0 * stagnation_penalty
        )

        # Clip final reward
        reward = np.clip(reward, -50, 50)

        # Debug output
        print(
            f"Reward components:\n"
            f"  Orientation: roll={math.degrees(roll):+06.2f}°"
            f"  Stability: {stability_penalty:+06.2f}\n"
            f"  Energy: {energy_penalty:+06.2f}\n"
            f"  Stuck counter: {self.stuck_counter:+06.2f}\n"
            f"  Time elapsed: {self.time_elapsed:+06.2f}s\n"
            f"  Distance: {distance:+06.2f}m\n"
            f"  Total reward: {reward:+06.2f}"
        )

        return reward

    def close(self):
        """Clean up and close the environment."""
        p.disconnect()
```

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
a. menyebutkan sumber sesuai kaidah kecendekiaan;
b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
Universitas Pertamina.
Hak Cipta Universitas Pertamina
Undang-Undang
© Copyright of Universitas Pertamina

Research Model Canvas

Tugas Akhir

Oleh:
Muhammad Harish
102121014

FAKULTAS TEKNOLOGI INDUSTRI
PROGRAM STUDI TEKNIK ELEKTRO
UNIVERSITAS PERTAMINA
2025

Masalah/Gap	Tujuan	Metode
Robot Hexapod kesulitan dalam melewati rintangan secara efektif dalam lingkungan yang dinamis. Sehingga diperlukan pengembangan kemampuan adaptasi gerakan robot terhadap perubahan situasi yang terjadi.	Mengembangkan robot hexapod yang dapat beradaptasi pada kondisi lingkungan seperti klasifikasi rintangan dan gerak robot yang dinamis	Penerapan kontrol konvensional yaitu kinematika robot hexapod seperti *inverse kinematic, body kinematic, trajectory, dan gait* yang diintegrasikan dengan ***Reinforcement Learning*** **(RL)**. RL digunakan untuk menciptakan gait yang dinamis dan stabil untuk menghasilkan gerakan yang adaptif
Pustaka Utama	**Teori dan Mata Kuliah Pendukung**	**Topik Peta Jalan PSTE**
Shichang Huang, Minhua Zheng, Zhongyu Hu, Peter Xiaoping Liu, **"Enhancing Hexapod Robot Mobility on Challenging Terrains: Optimizing CPG-Generated Gait with Reinforcement Learning"**, Neurocomputing, Vol. 622, p. 129328, 2025.	Dasar Pemrograman dan Algoritma [Python dan C++], Sistem Kendali [Model matematika dan kendali], Mekatronik [Sensor, aktuator, kontroller], Kecerdasan Buatan [Reinforcement Learning], Robotik [Teknologi robotik]	Teknologi Informasi dan Komunikasi Mekatronika
Judul		
Implementasi Reinforcement Learning pada Robot Hexapod		

Abstrak

Robot hexapod adalah solusi inovatif untuk eksplorasi lingkungan yang sulit dijangkau dan berbahaya, dengan pendekatan bio-inspired yang memungkinkan mobilitas tinggi melalui jumlah kaki dan derajat kebebasan yang dimilikinya. Namun, kompleksitas tinggi dalam pengaturan gerak menjadi tantangan utama. Penelitian ini berfokus pada pengembangan robot hexapod yang mampu mendeteksi rintangan dan beradaptasi terhadap lingkungan dengan mengintegrasikan kontrol konvensional berbasis kinematika seperti inverse kinematic, body kinematic, trajectory, dan gait, yang dikombinasikan dengan Reinforcement Learning (RL). RL digunakan untuk menciptakan pola gerak (gait) yang dinamis, stabil, dan adaptif terhadap perubahan kondisi lingkungan. Proses pengembangan dilakukan mulai dari simulasi, desain hardware robot, hingga integrasi sistem yang diuji dalam simulasi dan lingkungan nyata pada arena Kontes Robot SAR Indonesia (KRSRI) 2024. Diharapkan hasil penelitian ini dapat menghasilkan robot hexapod yang dapat beradaptasi dengan berbagai rintangan dan kondisi lingkungan, serta memberikan kontribusi dalam pengembangan teknologi robotik yang mendukung penyelesaian permasalahan di sektor industri dan mitigasi bencana.

Universitas Pertamina

b. pengutipan hanya untuk keperluan pendidikan, penulisan karya ilmiah atau penelitian;
c. pengutipan tidak merugikan Universitas Pertamina.
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

LEMBAR BIMBINGAN TUGAS AKHIR

Nama	: Muhammad Harish	NIM	: 102121014
Nama Pembimbing	: Dr. Eng. Muhammad Abdillah, S.T., M.T.	NIP	: 116153

No.1	Hari/Tanggal: 31 Januari 2025

Diskusi terkait pemilihan judul dan bedah judul Tugas Akhir
Pengembangan Robot Hexapod untuk Deteksi Rintangan dan Adaptasi Gerakan Berbasis Visual Language Model dan Reinforcement Learning
- Pengembangan Robot Hexapod: Pembuatan robot berkaki enam.
- Deteksi Rintangan: Kemampuan mengenali lingkungan.
- Adaptasi Gerakan: Penyesuaian pola gerak pada berbagai rintangan
- Visual Language Model: Pemrosesan visual untuk mengetahui rintangan yang akan dihadapi
- Reinforcement Learning: Pembelajaran dari pengalaman robot untuk menghasilkan gerakan yang stabil di berbagai rintangan.

Paraf Dosen Pembimbing:

No.2	Hari/Tanggal: 05 Februari 2025

Bimbingan terkait draft proposal Tugas Akhir untuk persiapan submit draft proposal ke program studi
- Jangan menggunakan DRL (Deep Reinforcement Learning) karena perlu data yang sangat banyak dan cukup sulit untuk dikerjakan. Solusinya mengganti DRL dengan RL (Reinforcement Learning)

Paraf Dosen Pembimbing:

No.3	Hari/Tanggal: 27 Februari 2025

Pembimbingan dan perbaikan proposal tugas akhir final
- Pengubahan judul tugas akhir dengan menghilangkan komponen RL (Reinforcement Learning) karena dinilai tugas akhir terlalu kompleks. Solusinya fokus mengembangan bagian VLM (Visual Language Model)
- Judul tugas akhir baru: Pengembangan Robot Hexapod Berbasis Visual Language Model untuk Kendali Gerakan Adaptif dan Navigasi Otonom

Paraf Dosen Pembimbing:

Pertamina
2. Dilarang memublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

No.4	Hari/Tanggal: 10 Maret 2025
Presentasi progres tugas akhir dan diskusi mengenai permasalahan alur kerja Visual Language Model (VLM) yang cukup rumit dalam menghasilkan JSON. Diusulkan penggunaan NLP untuk mengonversi VLM ke JSON agar prosesnya lebih mudah. Selain itu, dibahas juga mengenai hardware komputasi robot, di mana Jetson Nano dinilai kurang andal untuk VLM, dengan alternatif yang lebih potensial yaitu Jetson Orin Nano. Paraf Dosen Pembimbing:	
No.5	Hari/Tanggal: 19 Maret 2025
Perubahan judul tugas akhir yang sebelumnya menggunakan Visual Language Model menjadi Reinforcement Learning Paraf Dosen Pembimbing:	
No.6	Hari/Tanggal: 07 April 2025
Penambahan sumber gambar pada laporan, jika bisa lebih baik membuat gambarnya sendiri. Karena seharusnya jika memakai gambar orang lain sebaiknya izin ke penulisnya. Penambahan penelitian terkait/terdahulu pada BAB II Tinjauan Pustaka Laporan Tugas Akhir. Paraf Dosen Pembimbing:	
No.7	Hari/Tanggal: 16 Mei 2025
Kemajuan robot hexapod dengan pengujian hardware. Pengujian ini menggunakan input parameter robot seperti velocity x,y dan body kinematics robot yang dikontrol menggunakan stick PS4. Bertujuan untuk mengetahui kinematika robot sudah berjalan dengan baik. Diskusi terkait pengambilan jadwal sidang tugas akhir, disarankan untuk maju di sidang batch 1 (akhir Juni) Paraf Dosen Pembimbing:	

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

No.8	Hari/Tanggal: 26 Mei 2025
Progress tugas akhir sudah mencapai tahap pengujian, setelah proses training dan testing pada simulasi, maka langkah selanjutnya adalah menguji pada lingkungan real menggunakan lapangan krsri 2024 yang terdiri dari jalan menanjak, menurun, dan tangga. robot hexapod berhasil melewati rintangan dan hasil yang didapatkan sudah cukup baik, sehingga langka selanjutnya adalah melakukan analisis terhadap data robot saat melewati rintangan Paraf Dosen Pembimbing:	
No.9	Hari/Tanggal: 16 Juni 2025
Pada bimbingan kali ini, pembimbing memberikan masukan terkait hasil analisis data pengujian robot di lapangan. Bagian hasil dan pembahasan dinilai masih kurang mendalam, terutama dalam menjelaskan hubungan antara data dan performa robot saat melewati rintangan seperti tanjakan, turunan, dan tangga. Pembimbing menyarankan agar pembahasan dibuat lebih analitis, dilengkapi grafik dan visualisasi data, serta tidak hanya bersifat deskriptif. Selain itu, disarankan agar hasil dan pembahasan dibuat minimal 20 halaman agar lebih lengkap dan sesuai dengan standar laporan tugas akhir. Paraf Dosen Pembimbing:	

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.
of Universitas Pertamina

Universitas Pertamina

LEMBAR BIMBINGAN TUGAS AKHIR

Nama	: Muhammad Harish	NIM	: 102121014
Nama Pembimbing	: Novanto Yudistira, Dr.Eng., S.Kom., M.Sc.	NIP	: 20120183B1101001

No.	Hari/Tanggal: 1-3-2025
penentuan topik dan judul penelitian Paraf Dosen Pembimbing:	
No.	Hari/Tanggal: 3-3-2025
perlu diperhitungkan VLM yg digunakan agar dapat diaplikasikan di edge device. mencari model yg efisien Paraf Dosen Pembimbing:	

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Pertamina

2. Dilarang memublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Universitas Pertamina

LEMBAR BIMBINGAN TUGAS AKHIR

Nama	: Muhammad Harish	NIM	: 102121014
Nama Pembimbing	: Dr. Eng. Novanto Yudistira, S.Kom., M.Sc.	NIP	: 20120183311101001

No. 3	Hari/Tanggal: Jumat, 7 Maret 2025

Hal yang perlu diperhatikan:

- Impelemntasi VLM (Visual Language Model) memerlukan komputasi yang besar, sehingga pengaplikasian VLM pada robot hexapod menjadi tantangan.
- Spesifikasi hardware yang diperlukan harus memiliki GPU untuk processing yang realtime seperti Jetson Orin



Paraf Dosen Pembimbing:

No. 4	Hari/Tanggal: Jumat, 14 Maret 2025

Hal yang perlu diperhatikan:

- Percobaan impementasi VLM dan finetuning sudah dicoba untuk dilakukan. Terdapat beberapa kendala dalam finetuning VLM diantaranya:
  1. VLM yang digunakan untuk finetuning adalah BLIP Image Captioning, kendala yang didapatkan adalah BLIP Image Captioning tidak dirancangan untuk dapat generate text berbasis JSON yang hal ini dibutuhkan untuk menggerakkan robot
  2. Solusi yang dicoba adalah dengan mengintegrasikan dengan NLP sehingga hasil deskripsi text dari VLM dapat diubah menjadi text berbasis JSON
  3. Akan tetapi hal ini sulit dicapai dan membutuhkan waktu serta komputasi yang cukup besar.
- Solusi yang diberikan adalah dengan mengubah metode VLM dengan metode lain seperti CNN+LSTM atau CNN+Reinforcement Learning.



Paraf Dosen Pembimbing:

No. 5	Hari/Tanggal: Jumat, 21 Maret 2025

Hal yang perlu diperhatikan:

- Laporan progress terkait penerapan Reinforcement Learning pada robot hexapod menggunakan simulasi Pybullet, didapatkan hasil bahwa robot daapat bergerak dengan baik.
- Selanjutnya dilakukan pengembangan mendalam pada Reinforcement Learning dengan mencoba beberapa metode seperti PPO dan SAC untuk kemudian dilakukan Hyperparameter Tuning. Selain itu disarankan juga untuk mengintegrasikan dengan CNN sebagai klasifikasi rintangan.



Paraf Dosen Pembimbing:

No. 6	Hari/Tanggal: Jumat, 11 April 2025

Hal yang perlu diperhatikan:

- Menambahkan referensi penelitian sebelumnya yang terkait untuk mengetahui perkembangan teknologi robot hexapod. Beberapa referensi jurnal yang dapat diambil yaitu dari IEEE dan IROS.



Paraf Dosen Pembimbing:

Pertamina 2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

No. 7	Hari/Tanggal: Jumat, 16 Mei 2025

Hal yang perlu diperhatikan:

- Mencoba algoritma SAC terbaru yaitu Langevin SAC https://github.com/hmishfaq/LSAC.git, diharapkan dapat lebih baik dibandingkan SAC sebelumnya.
- Jika ada waktu tambahan dan memungkinkan, dapat ditambahkan deteksi rintangan berupa visual yang dapat menambah efektifitas model reinforcement learning. Untuk deteksi rintangan dapat dengan menggunakan seperti CNN, YOLO, dll.

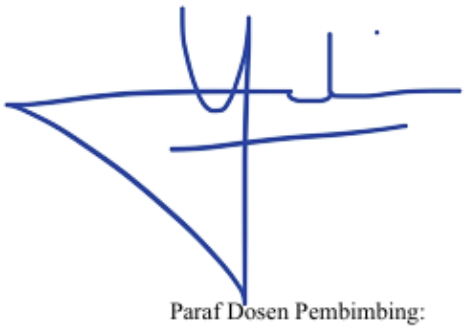


Paraf Dosen Pembimbing:

No. 8	Hari/Tanggal: Jumat, 24 Mei 2025

Hal yang perlu diperhatikan:

- Hasil pengujian robot secara real sudah di implementasikan dan dapat berjalan dengan baik.
- Robot sukses melewati beberapa rintangan seperti slope down, slope climbing, dan climbing stairs.
- Pengujian dilakukan menggunakan algoritma RL Soft Actor Critic.
- Jika ada waktu bisa dicoba untuk implementasi algoritma lain seperti TD3 yang sudah selesai di training



Paraf Dosen Pembimbing:

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Pertamina
2. Dilarang mempublikasikan atau memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa adanya izin dari Universitas Pertamina.

Form TA-4 Pendaftaran Sidang Tugas Akhir

FAKULTAS TEKNOLOGI INDUSTRI
PROGRAM STUDI TEKNIK ELEKTRO

Nama	:	Muhammmad Harish
NIM	:	102121014
Tema Tugas Akhir	:	Implementasi Reinforcement Learning pada Robot Hexapod
Tanggal Seminar Kemajuan TA	:	25 April 2025

Bersama Form ini disertakan pula:

No.	Uraian Kelengkapan	Keterangan*
1.	Tiga Salinan *draft* Laporan Tugas Akhir (empat Salinan bila Pembimbing 2 orang)	
	Softcopy draft Laporan Tugas Akhir (menggantikan no. 1 apabila kondisi Pandemi masih menyebabkan kegiatan pembelajaran dilakukan secara *online*)	
2.	Bukti bahwa Laporan Tugas Akhir telah bebas plagiasi**	
3.	Transkrip nilai resmi mata kuliah wajib minimal 139 sks (kecuali TA) telah lulus (nilai minum C) dengan IPK minimal 2,00	
4.	Form Bimbingan Tugas Akhir (dengan jumlah bimbingan minimal 8 kali)	
5.	Form bebas pinjam dari laboratorium	
6.	Bukti lunas SPP	
7.	Transkrip Penilaian Ekstrakulikuler Mahasiswa minimal score 1300	

(* beri tanda ✓ jika sudah ada oleh Tenaga Kependidikan Program Studi)
(** maksimal kesamaan/*similarity* 30 %)

Jakarta, 24 Juni 2025
Pendaftar,

Nama : Muhammad Harish
NIM : 102121014

Menyetujui,

Dosen Pembimbing I,	Dosen Pembimbing II,
Nama : Dr. Eng. Muhammad Abdillah, S.T., M.T.	Nama : Dr. Eng. Novanto Yudistira, S.Kom., M.Sc
NIP : 116153	NIP : 20120183311101001